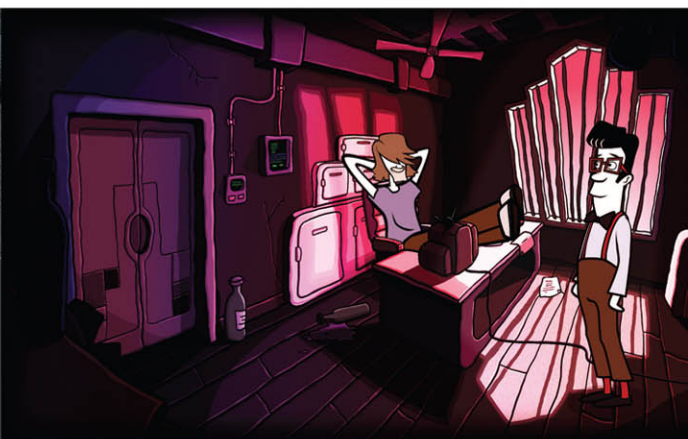
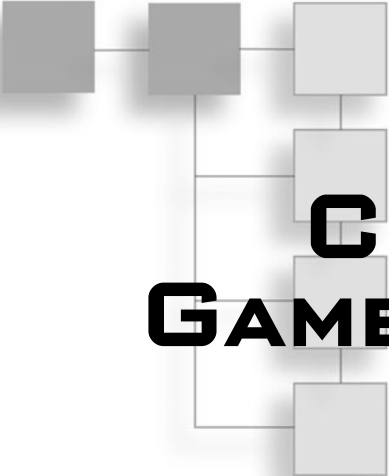


ANIMATING 2D CHARACTERS FOR GAMES IN BLENDER™





ANIMATING 2D CHARACTERS FOR GAMES IN BLENDER™

ALAN THORN

Cengage Learning PTR

 **CENGAGE
Learning®**

Professional • Technical • Reference

Animating 2D Characters for Games in Blender™

Alan Thorn

Publisher and General Manager,
Cengage Learning PTR: Stacy L. Hiquet

Associate Director of Marketing:
Sarah Panella

Manager of Editorial Services:
Heather Talbot

Senior Product Manager: Emi Smith

Project Editor: Kate Shoup

Technical Reviewer: Michael Duggan

Copy Editor: Kate Shoup

Interior Layout Tech: MPS Limited

Cover Designer: Mike Tanamachi

Proofreader: Kelly Talbot Editing
Services

© 2015 Cengage Learning PTR.

CENGAGE and CENGAGE LEARNING are registered trademarks of Cengage Learning, Inc., within the United States and certain other jurisdictions.

ALL RIGHTS RESERVED. No part of this work covered by the copyright herein may be reproduced, transmitted, stored, or used in any form or by any means graphic, electronic, or mechanical, including but not limited to photocopying, recording, scanning, digitizing, taping, Web distribution, information networks, or information storage and retrieval systems, except as permitted under Section 107 or 108 of the 1976 United States Copyright Act, without the prior written permission of the publisher.

For product information and technology assistance, contact us at
Cengage Learning Customer & Sales Support, 1-800-354-9706.

For permission to use material from this text or product,
submit all requests online at **cengage.com/permissions.**

Further permissions questions can be emailed to
permissionrequest@cengage.com.

Blender is a trademark of The Blender Foundation. All other trademarks are the property of their respective owners.

All images © Cengage Learning unless otherwise noted.

ISBN-13: 978-1-305-50184-3

ISBN-10: 1-305-50184-5

eISBN-10: 1-305-50184-5

Cengage Learning PTR

20 Channel Center Street

Boston, MA 02210

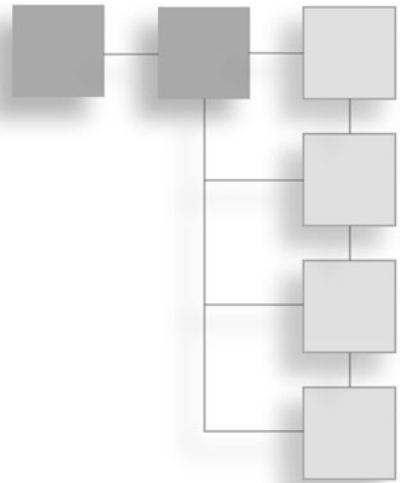
USA

Cengage Learning is a leading provider of customized learning solutions with office locations around the globe, including Singapore, the United Kingdom, Australia, Mexico, Brazil, and Japan. Locate your local office at:
international.cengage.com/region.

Cengage Learning products are represented in Canada by Nelson Education, Ltd.

For your lifelong learning solutions, visit **cengageptr.com.**

Visit our corporate website at **cengage.com.**

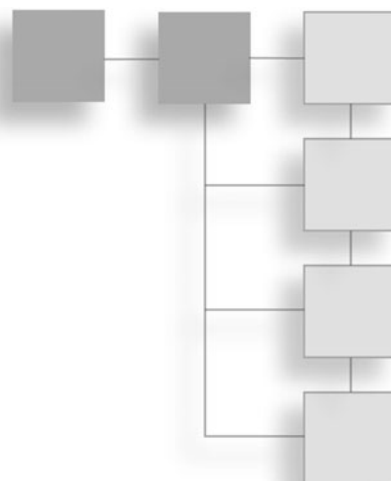


ACKNOWLEDGMENTS

Writing a book is no simple matter. Although I'm listed as the author, this book would not have been possible without the valuable help and fine work of many people, including Emi Smith, Kate Shoup, Michael Duggan, and many others. I'd like to say a big thank you to them, and also to you, the reader, for investing time to improve your skills. I hope this book proves helpful to you.

Alan Thorn, 2014

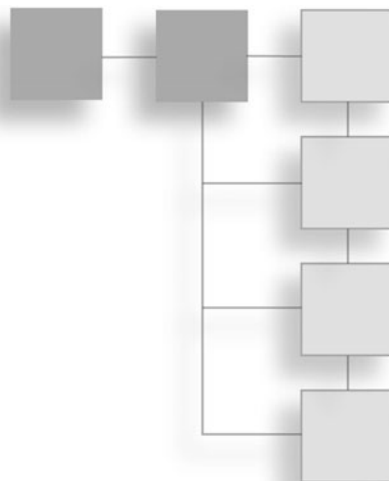
London



ABOUT THE AUTHOR

Alan Thorn is a game developer, freelance programmer, and author with more than 13 years of industry experience. He founded Wax Lyrical Games in 2010 and is the creator of the award-winning game *Baron Wittard: Nemesis of Ragnarok*. He is the author of 10 video training courses and 11 books on game development, including *Unity 4 Fundamentals*, *UDK Game Development*, and *Pro Unity Game Development with C#*. He is also a visiting lecturer for the game development master's degree at the National Film and Television School, London. Alan has worked on a freelance basis on more than 500 projects, including games, simulators, kiosks, serious games, and augmented reality software for game studios, museums, and theme parks worldwide. He is currently working on an adventure game, *Mega Bad Code*, for desktop computers and mobile devices.

CONTENTS



Introduction

Chapter 1 Preparation and Modeling

Step 1: Drawing Your Character

Step 2: Importing Character References into Blender

Step 3: Modeling from References

Step 4: Optimizing Topology for Games and Animation

Step 5: Housekeeping

Conclusion

Chapter 2 UV Mapping and Texturing

Step 6: Removing Doubles

Step 7: Creating the Character Atlas Texture

Step 8: UV Preparation and Projection

Step 9: Creating UV Islands and Mapping

Step 10: Completing the Mapping

Conclusion

Chapter 3 Rigging, Bones, and Weights

Step 11: Creating Armatures

Step 12: Creating Bones

Step 13: Skinning

Step 14: Adjusting the Weighting

Step 15: Applying Inverse Kinematics
Conclusion

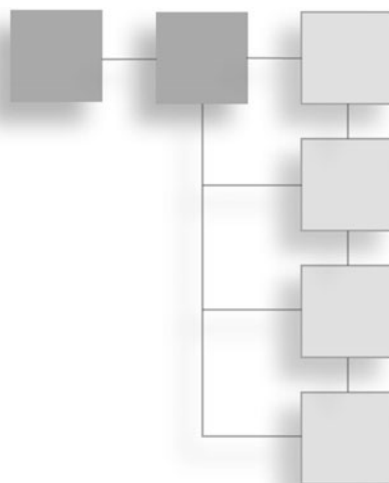
Chapter 4 Animation

Step 16: Preparing for Animation
Step 17: Blocking In the Walk Cycle
 The Contact Pose
 The Passing Pose
 The Down Pose
 The Up Pose
Step 18: Moving the Arms
Conclusion

Chapter 5 Exporting and Testing

Step 19: Exporting Objects from Blender
Step 20: Importing Objects into Unity
Step 21: Configuring the Mesh
Step 22: Testing the Mesh
Conclusion

INTRODUCTION



This eBook is aimed at readers looking to use Blender to create animated 2D characters for video games. It'll be especially relevant to developers working with Unity, as I provide examples for importing characters into Unity. The main focus of this eBook, however, is on Blender's tools and techniques for animating a 2D character through a complete looping walk cycle animation. It covers subjects such as modeling from reference, texture mapping and unwrapping, rigging, animation, and finally export-import workflows. Further, it outlines the basic mechanical stages of a walk cycle, which apply to most bipedal characters.

In describing this eBook, it's important to stress several issues:

- This eBook covers Blender, which is freely available software. It doesn't cover alternatives such as 3DS Max or Maya. This, however, doesn't mean the general workflow and concepts covered here don't apply to those applications, because they do. It means only that all software-specific steps will be given in relation to Blender and its UI.
- This eBook focuses on 2D characters as opposed to the more common 3D type. Consequently, this eBook is especially relevant for developers creating cartoon or stylized characters for their games.
- In covering character animation, this eBook assumes you have a working familiarity with the Blender basics such as the UI, a control scheme, and a solid grasp of foundational concepts like box modeling, mesh editing, and object transforming. This eBook avoids unnecessary assumptions where possible, introducing concepts

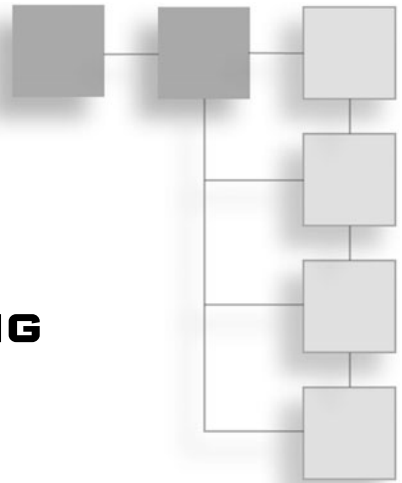
and tools as though you were encountering them for the first time, but it doesn't assume you're totally new to Blender.

Tip

If you've never used Blender before, or if you're new to 3D modeling generally, I recommend checking out many of the books and tutorials on the Blender basics before getting started with this eBook. Good starting points include *The Complete Guide to Blender Graphics* by John M. Blain and *Blender Foundations* by Roland Hess.

With those points in mind, let's get started animating 2D characters with Blender!

CHAPTER 1



PREPARATION AND MODELING

Blender is renowned for creating 3D graphics and animation—the kind found in real-time 3D games and movie shorts like *The Elephant’s Dream* and *Big Buck Bunny*. For this reason, 2D character animation—the kind found in cartoons—may be regarded as an unconventional use of Blender. Indeed, many avoid Blender altogether for this sort of animation. They often turn instead to Adobe Flash, Toon Boom Animate, Anime Studio Pro, or Synfig Studio. But this eBook will show you how to creatively adapt and reuse Blender’s 3D functionality to make professional-grade 2D character animation for games. Being able to do this is useful. It means Blender users can continue to use their free and cross-platform software and its familiar toolset to get their versatile work done. This can save time, effort, and money.

This eBook is project-based. It focuses on the specific, real-world case of creating a fully animated character in Blender that can walk in a loopable cycle, as shown in Figure 1.1. You’ll work through the complete animation process, from start to end—from drawing board to a final game-ready model.

The character to be created is named Pendragon, a carefree, clumsy homicide detective in my upcoming game *Mega Bad Code*. One of Pendragon’s main design qualities is to provide comic relief for a dark and brooding storyline. In creating Pendragon, however, it’s important to be aware of the wider context and applicability of the knowledge contained here. Certainly, in finishing this eBook and using the companion files, you’ll re-create the animated Pendragon character. But you’ll have achieved much more than this—you’ll have seen a complete animation toolset in Blender and gained first-hand experience of

how it can be adapted and reused to make your own characters for your own projects according to your own needs. Helping you develop that power and awareness is the core purpose of this eBook. So let's begin.



Figure 1.1

Meet Pendragon, the character to model, texture, rig and animate using Blender.

Source: The Blender Foundation.

Note

This eBook uses Blender 2.70 and applies generally throughout the 2.7 range of releases. However, much of the information contained here also applies to earlier versions, from 2.5 upward.

STEP 1: DRAWING YOUR CHARACTER

The first step in character creation is to create concept sketches, playing around with ideas. After that, you'll draw your character using illustration software such as Adobe Illustrator or Inkscape. (See Figure 1.2.) The exact details of sketching and drawing techniques are not the central focus of this eBook. Here, we're concerned with Blender and animation specifically, and so I'll assume you already have a character design and illustration in place that you want to animate for games. Nevertheless, there are important animation considerations to remember when drawing your characters.

Note

You can create 2D characters using a range of software. Some free solutions include Inkscape (<http://www.inkscape.org>), GIMP (<http://www.gimp.org/>), MyPaint (<http://mypaint.intilinux.com/>), and Krita (<https://krita.org/>).

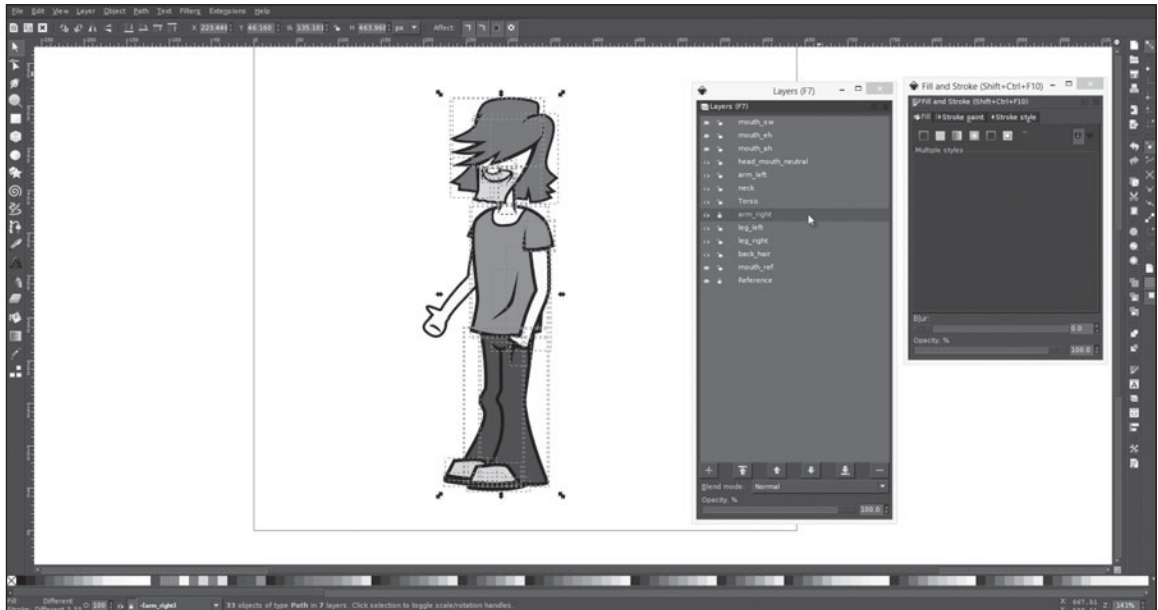


Figure 1.2

Start by drawing and inking your character. Arrange your character so that each limb or element is separate.

Source: The Inkscape Project.

Specifically, before drawing your final character, you'll want to be sure to identify all or most of the final animations you'll need: walk cycles, runs, jumps, punches, and more. Work out which limbs on the body must move (translate, rotate, and scale) to accommodate all your animation needs. Typically, these will be arms, legs, body, and head, but other character designs could involve more elements including costume props (such as hats) and weapons (for example, rifles and swords).

After identifying all movable things in the design, create your character in software so that each separate limb or element can easily be saved to an independent image file. To achieve this, simply draw each element in a separate layer or group that can be shown or hidden individually. The Pendragon character created here will walk. To achieve this, you'll need to be able to move his legs, head, body, and arms. These limbs have been drawn on separate layers in Inkscape, as shown in Figure 1.2, and each element has been exported to a

separate image file (using File > Export Bitmap), as shown in Figure 1.3. These images will be imported into Blender shortly as reference material.

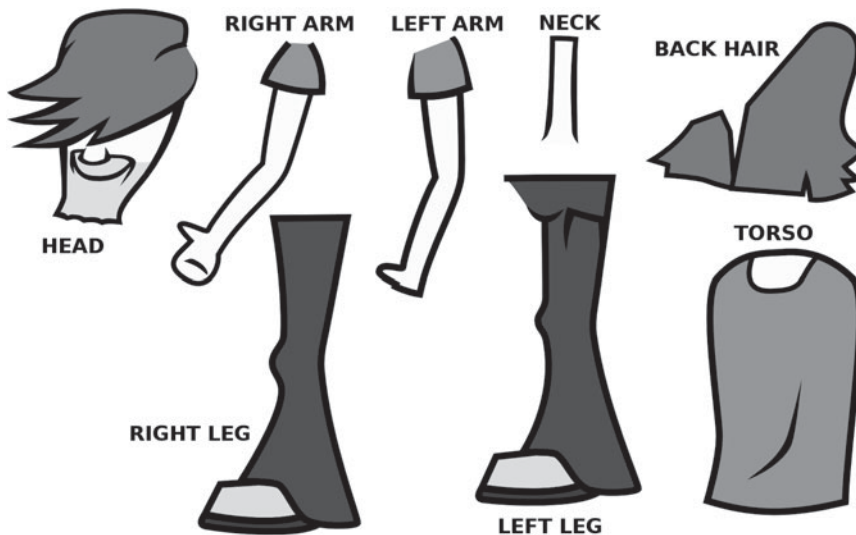


Figure 1.3

Each animated element is separated into a unique image file. The Pendragon files include head, left arm, right arm, left leg, right leg, torso, neck, and back hair.

Source: The Inkscape Project.

Note

The Pendragon element files are included in the eBook companion files in the Chapter01 folder.

Notice from Figure 1.3 that each element is divided into a separate file based on whether it moves independently during the animation, as well as its depth order in the overall composition. For example, the arms will swing and bend separately from the body and legs and other limbs, so they should be exported to separate images. That's to be expected. But the back hair is also exported as a separate image even though it'll move along with the rest of the head as it rotates. The back hair is not exported with the head because it appears behind the elongated neck, and both the neck and head can rotate separately around their own pivot points.

STEP 2: IMPORTING CHARACTER REFERENCES INTO BLENDER

If you've separated your character elements into separate images, then you're ready to start creating an animated character in Blender. The basic process is to import your character images into Blender and then to model from them as references, as shown in Figure 1.4. Let's see how this works in more detail, starting from an empty scene file.

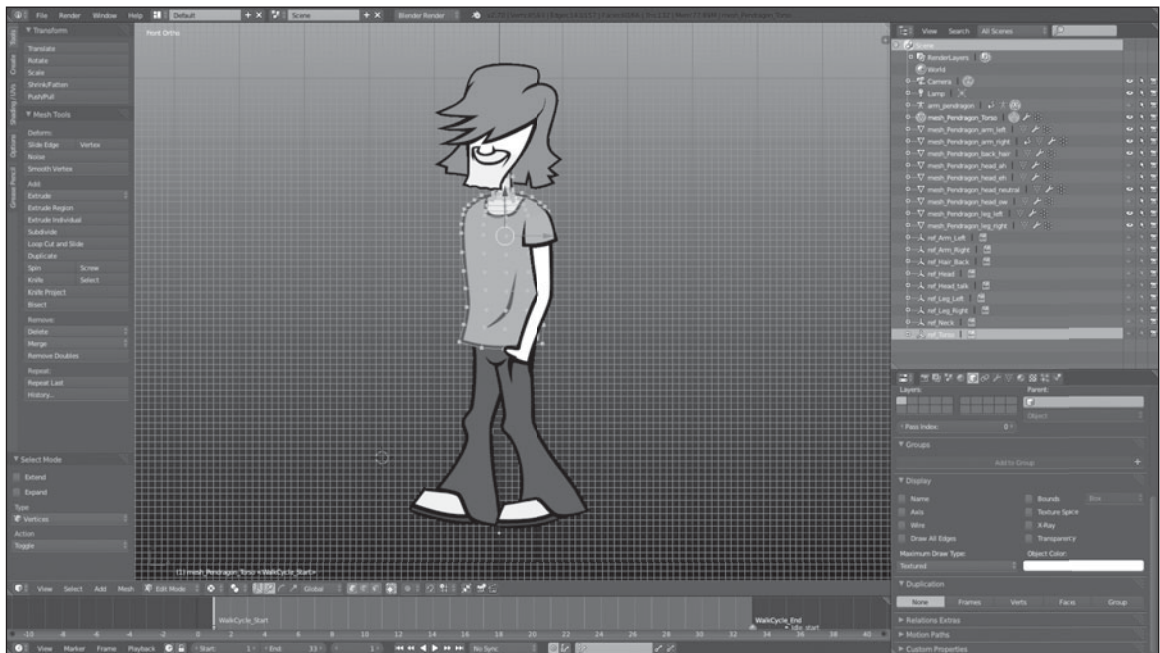


Figure 1.4
Modeling a character from reference in Blender.

Source: The Blender Foundation.

1. Switch to a front orthographic viewport in Blender (press Ctrl+1 using the Maya keys). This gives you a precise 2D view of the scene.
2. Create a new empty object at the origin. To do so, switch to the Create tab and click the Empty button. (See Figure 1.5.) This object will become your image reference, as you'll see. If the plane axes are not aligned to the front view, be sure to select the Align to View checkbox in the Create panel found in the bottom-left corner of the screen.

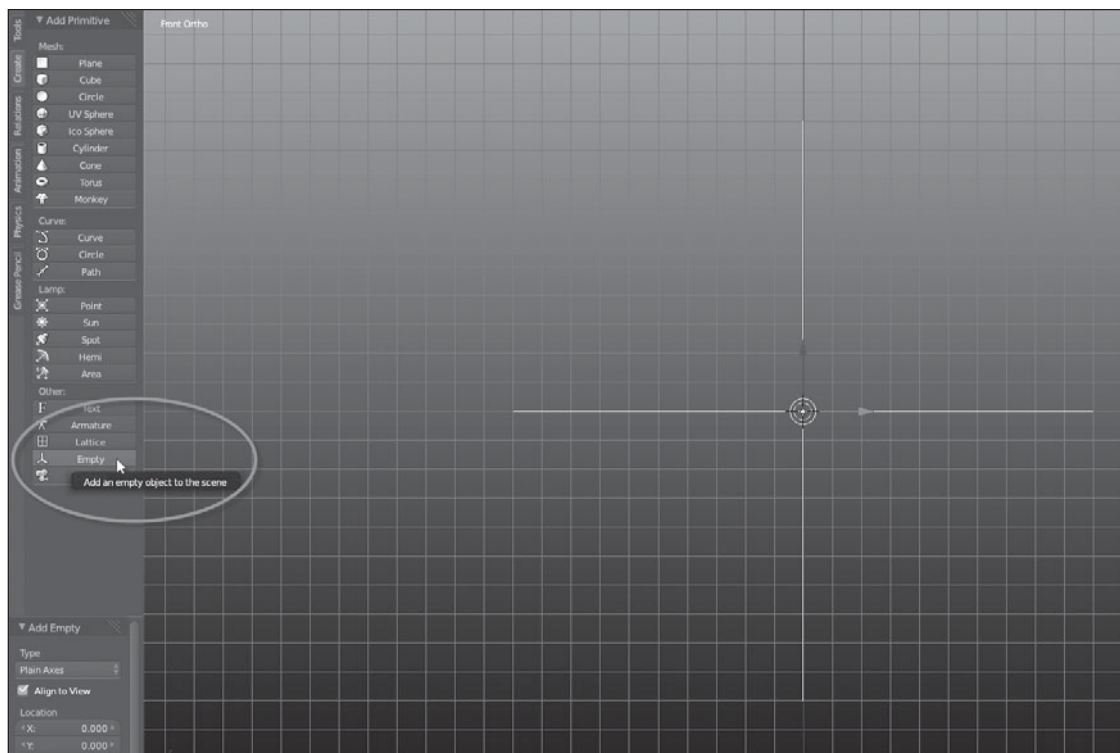


Figure 1.5
Creating a new empty object in the scene.

Source: The Blender Foundation.

Note

Newly created objects will be added at the position of the 3D cursor, not the world origin. To move the 3D cursor to the world origin, choose Object > Snap > Cursor to Center from the 3D viewport menu. For more information on the 3D cursor in Blender, see http://wiki.blender.org/index.php/Doc:2.6/Manual/3D_interaction/Transform_Control/Pivot_Point/3D_Cursor.

3. Select the newly created empty object and, in the Object Data tab in the Properties panel, open the Display drop-down list in the Empty section and choose Image. (See Figure 1.6.) This converts the empty object into an image plane.

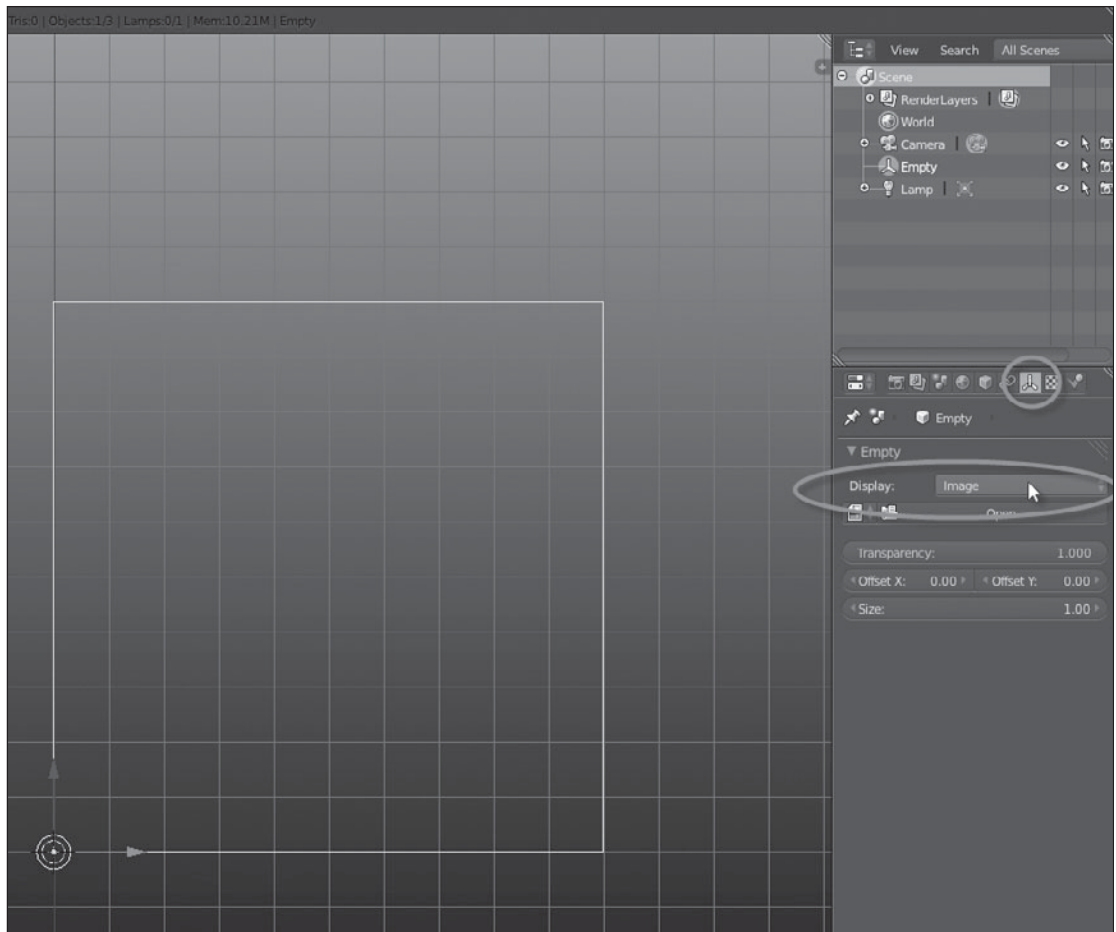


Figure 1.6

Converting an empty objects into an image plane.

Source: The Blender Foundation.

4. The image begins empty. The next step is to load a reference image into it. To achieve this, click the Open button in the Properties panel and select one of the character limb reference images from your hard drive. I've started with the head, as shown in Figure 1.7. The reference image should appear in the viewport.



Figure 1.7

Loading a reference image into the viewport.

Source: The Blender Foundation.

5. When a reference is loaded, reduce its transparency to around 0.5 in the Properties panel. This weakens its visual intensity in the scene, enabling you to concentrate on modeling.
6. In the Outliner window, disable object selection for the reference. This prevents you from accidentally selecting and moving it. To do so, click the mouse pointer icon in the Outliner to toggle it off for the reference object. See Figure 1.8.

**Figure 1.8**

Adjusting reference transparency and selectability.

Source: The Blender Foundation.

7. Repeat this process for each successive reference image for your character until all references are loaded into the scene. You can show and hide each reference individually by toggling its visibility from the Outliner using the eye icon. (See Figure 1.9.) This enables you to isolate and view only the reference you need at any one time in the viewport.

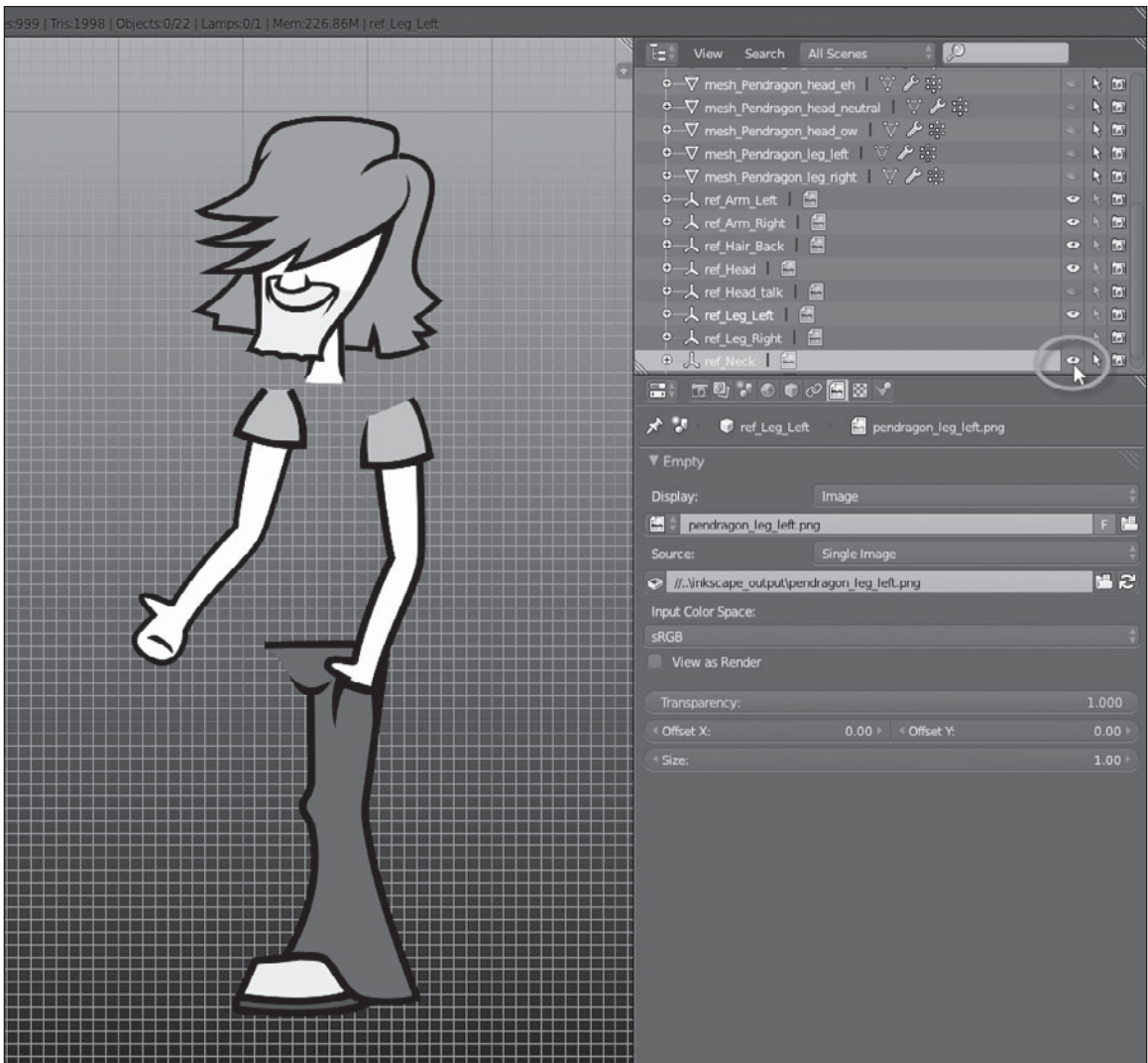


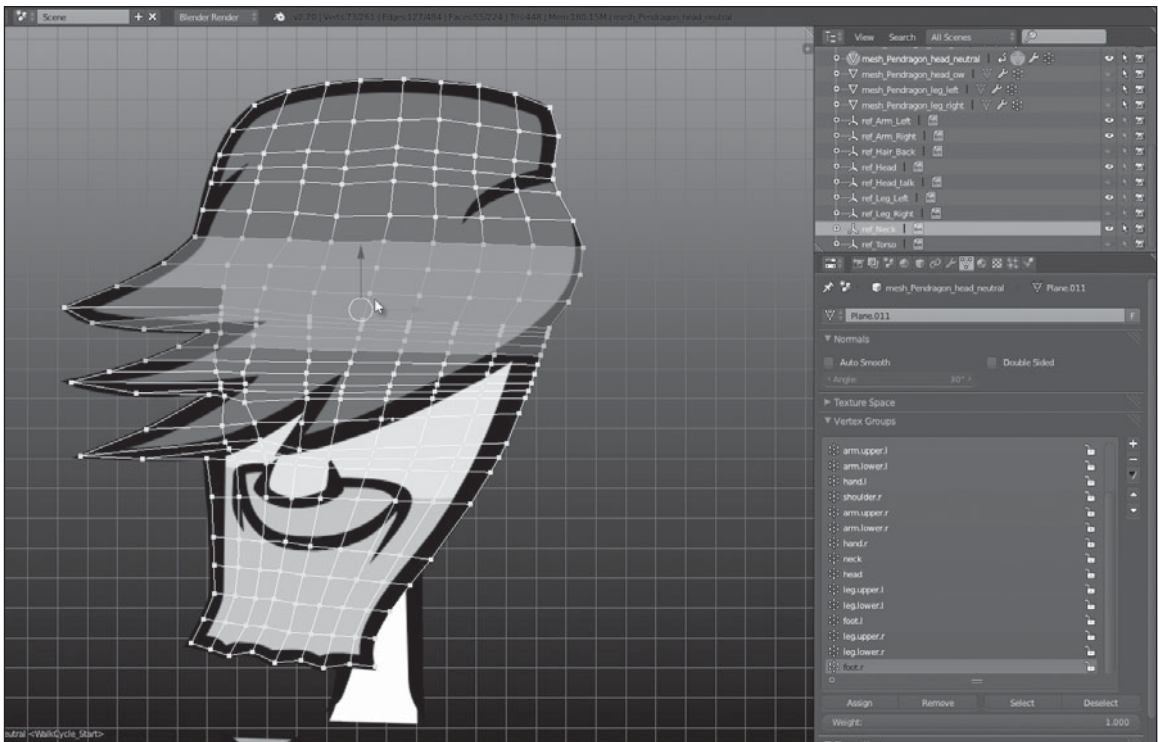
Figure 1.9

Loading all your reference images into Blender.

Source: The Blender Foundation.

STEP 3: MODELING FROM REFERENCES

The loaded reference images will act only as guides for creating polygonal models that truly conform to the reference. When creating a 2D character for animation, the modeling process involves shaping a flat-plane primitive to the reference image using standard modeling techniques like extrude and subdivide, as shown in Figure 1.10. This modeling approach may seem like overkill, but typically it's necessary.

**Figure 1.10**

Generate a plane and model over the reference image.

Source: The Blender Foundation.

Note

It might seem that a simpler solution would be to use only quad objects (planes with four corner vertices)—to map the reference images as textures onto the quads, using a transparent background in the texture to hide any pixels that are not part of the character. That way, you could stick with quad meshes and wouldn't need to add additional vertices. But quads usually fail to deform and bend as intended when animated, causing unwanted distortions on your character. So it's often best to avoid using quads.

1. Model each limb or element one at a time, hiding and showing the references in the viewport as necessary. To begin, create a new and separate mesh for each element. I often start with the character's head element. Center the head reference in the front viewport, and then create a new plane object in the scene using the Create panel. (See Figure 1.11.)

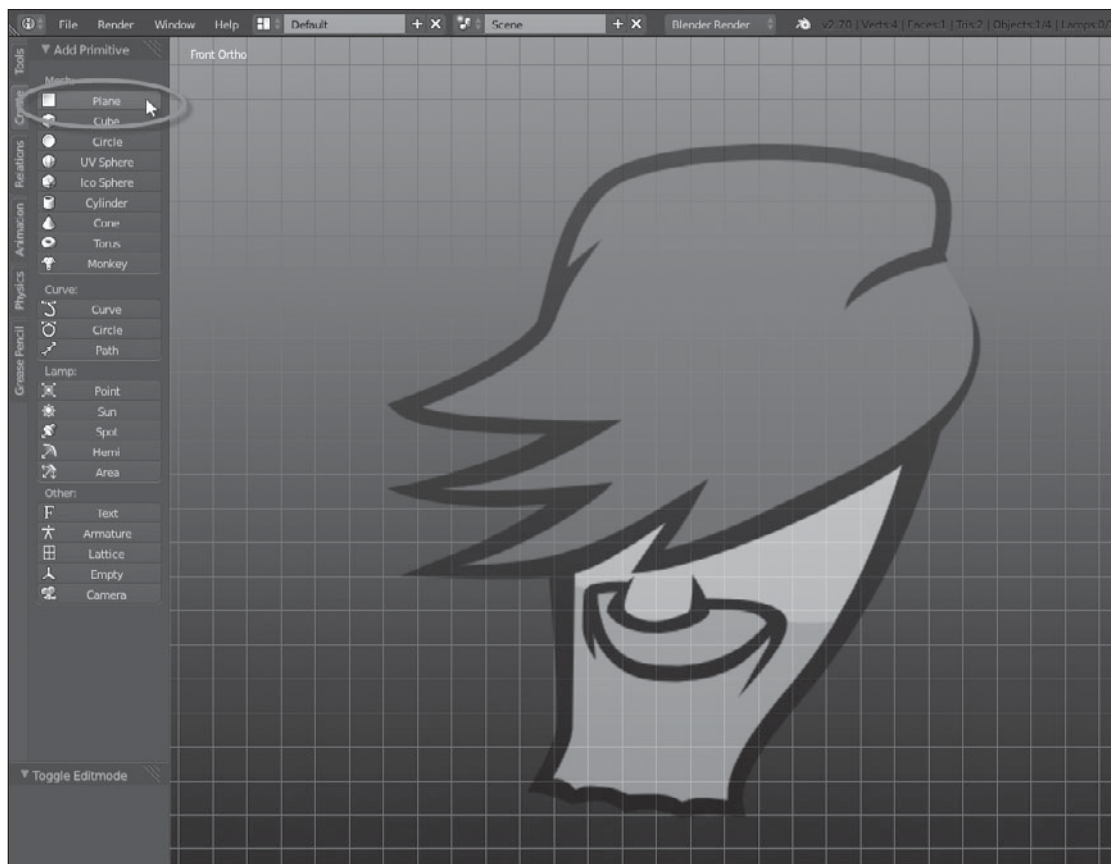


Figure 1.11

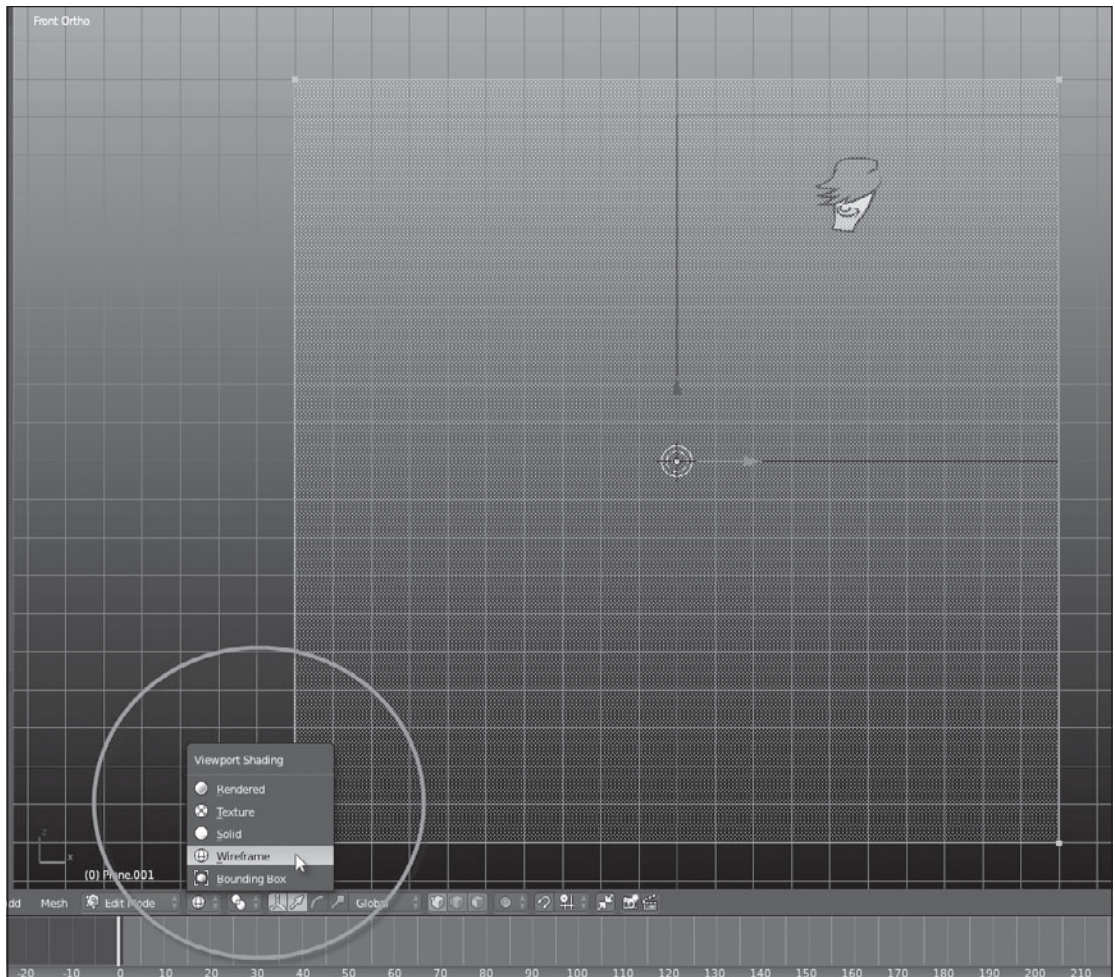
The modeling process often begins with a flat plane object.

Source: The Blender Foundation.

Note

Be sure to model your 2D characters inside orthographic viewports, such as a front, top, or side view, as opposed to a perspective view. This helps your vertices stay within the same XY plane.

2. Newly created objects like planes and cubes may appear in a solid or textured display in the viewport, covering or concealing the reference image behind them. It's important to see both your model and reference at the same time, however. For this reason, choose Wireframe from the Viewport Shading menu, as shown in Figure 1.12. This renders your plane object as a wireframe or cage while still showing the image reference.

**Figure 1.12**

Use Wireframe mode to view your meshes and references together.

Source: The Blender Foundation.

3. To start the modeling process, shape the plane object to the general extents of the reference image, encompassing its total surface area. To do this, enter Edit mode for the mesh object (press the Tab key) and then translate the mesh vertices into place. If your character is in the cartoon style and has a thick, stroked outer border, like Pendragon, then position your vertices to the outside edge of the border, allowing as much of the reference image to be contained inside the mesh as possible. See Figure 1.13.

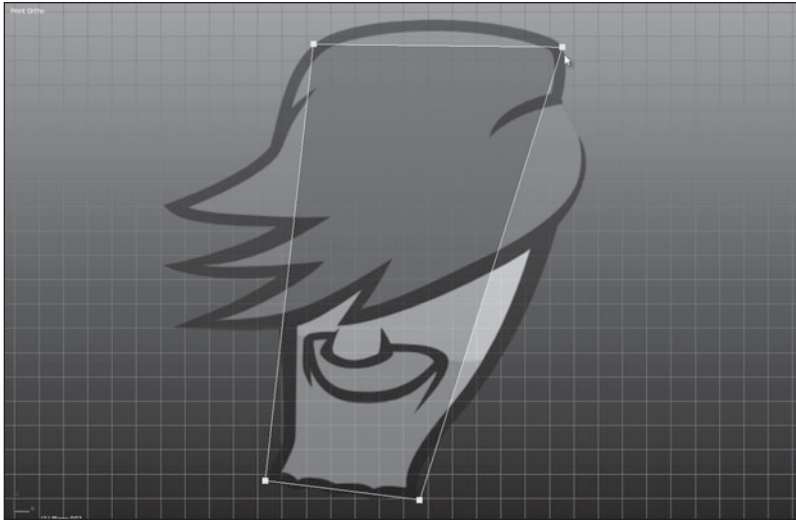


Figure 1.13

Building the extents of the mesh using the image reference.

Source: The Blender Foundation.

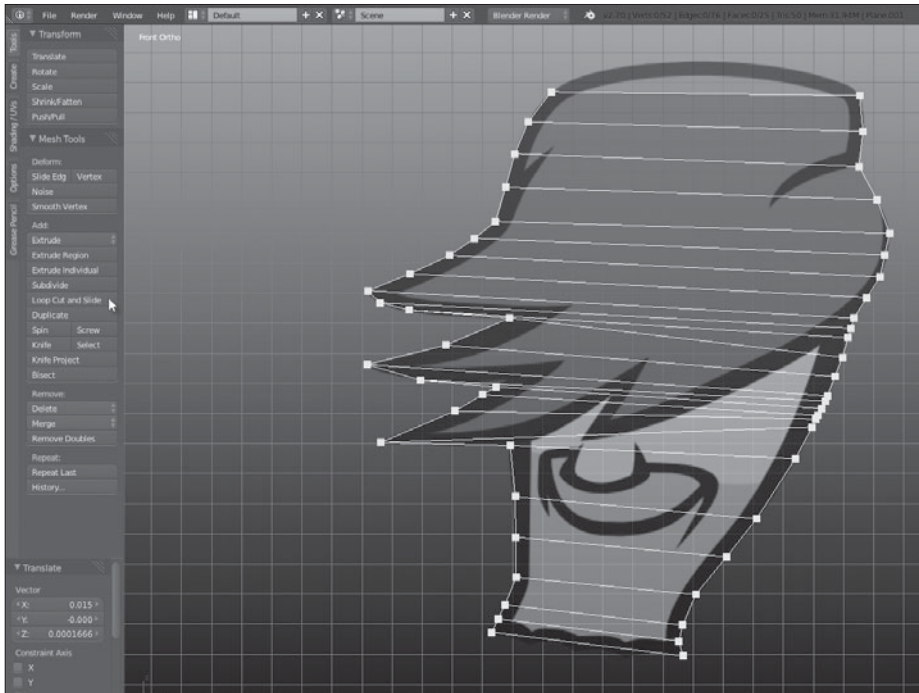
4. A quad mesh is not detailed enough to approximate the overall shape of the head. Neither will it be enough for nearly any element of a character. For this reason, more detail must be added to the mesh. You achieve this by using the Loop Cut and Slide tool, available from the Tools panel only while in Edit mode. After you click the Loop Cut and Slide tool button, move your cursor over the mesh. Blender will display a preview for a new edge loop that'll be inserted into the mesh. Click your mouse to fix the new edge loop at the previewed position and then press Enter to confirm the operation, adding in more detail. (See Figure 1.14.)

**Figure 1.14**

The Loop Cut and Slide tool lets you insert additional detail into a model for shaping and finalizing.

Source: The Blender Foundation.

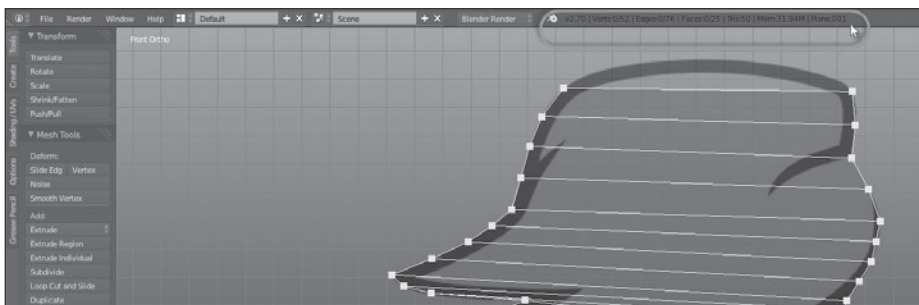
5. Continue to use the Loop Cut and Slide tool to insert more edge loops horizontally across the model, shaping and positioning each vertex as you go in line with the reference image. (See Figure 1.15.) At the end of the process, you should have a mesh that closely matches the reference except for vertical detail. I typically add vertical detail in a second pass. Remember to keep vertices positioned at the outer extremes of the reference while maintaining the shape and profile of the reference as closely as possible.

**Figure 1.15**

Insert additional edge loops to approximate horizontal detail in the mesh.

Source: The Blender Foundation.

- Because you're modeling for games, it's important to be aware of your model's vertex and face counts. You may be working within specific limits set by your team or you may simply be keeping vertices as low as possible, within reasonable limits, to respect your ultimate creative vision. To view the vertex and face counts for the selected mesh in Blender, see the Information panel, which appears at the top of the screen by default. (See Figure 1.16.)

**Figure 1.16**

Viewing vertex and face counts for the selected mesh.

Source: The Blender Foundation.

7. For a second pass, use the Loop Cut and Slide tool to insert additional vertical edge loops downward through the mesh, again shaping and positioning vertices to match the reference. Use the vertices to shape the top and bottom of the Pendragon head. When modeling, try to keep vertices and faces roughly equal in size and distance apart, conforming to the general shape and flow of the reference. This will ensure the best results when creating animations later. (See Figure 1.17.)

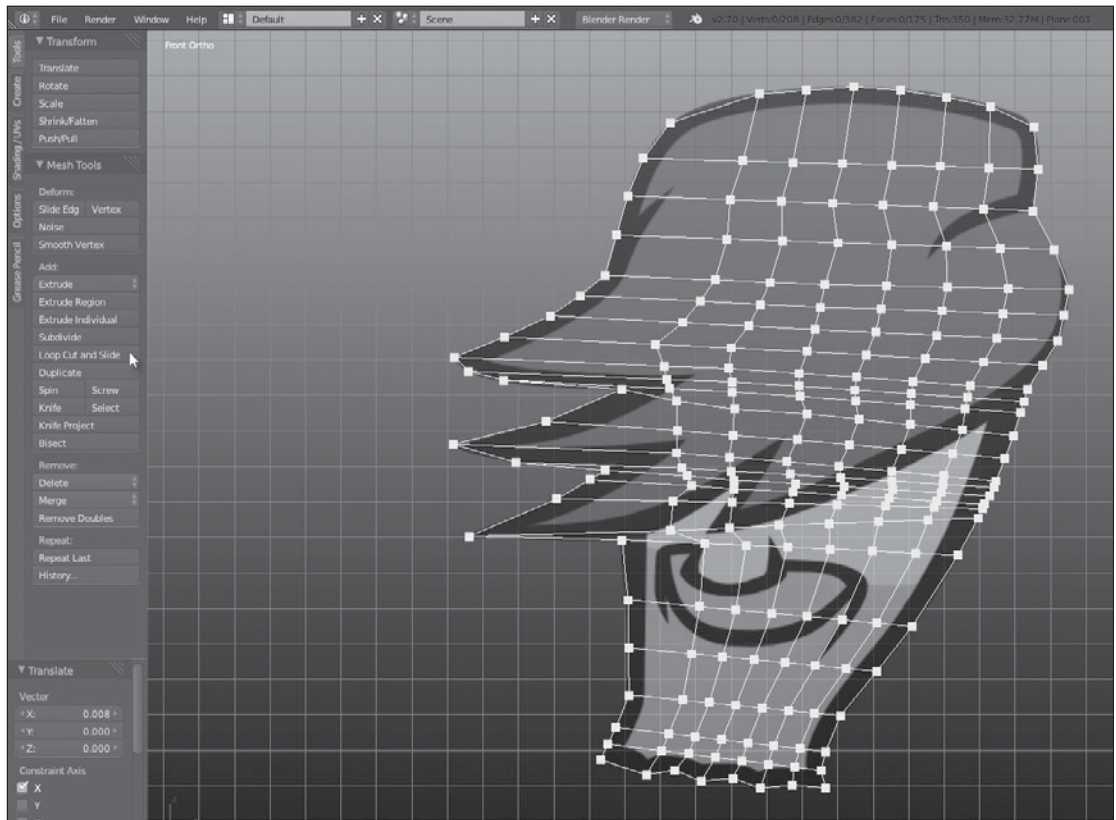


Figure 1.17

Inserting vertical edge loops to shape the head.

Source: The Blender Foundation.

8. Repeat this process for all other limbs and elements in the character, making sure to use a separate mesh for each. Use the hiding and showing feature for objects in the Outliner so you can concentrate on just one object at a time.

STEP 4: OPTIMIZING TOPOLOGY FOR GAMES AND ANIMATION

When modeling for games generally, and also for 2D characters, you should use only triangles or quads—that is, three- or four-sided polygons. (See Figure 1.18.) Polygons with more than four sides, known as *n-gons*, typically do not render correctly when imported into game engines. If you stick to using plane objects and the Loop Cut and Slide tool, it's unlikely you'll ever introduce *n-gons* into your model. But there are times when it can happen—for example, when using other tools like the Knife tool. In these cases, it's useful to detect and eliminate *n-gons* in your meshes before finalizing and exporting them to game engines. Follow these steps:

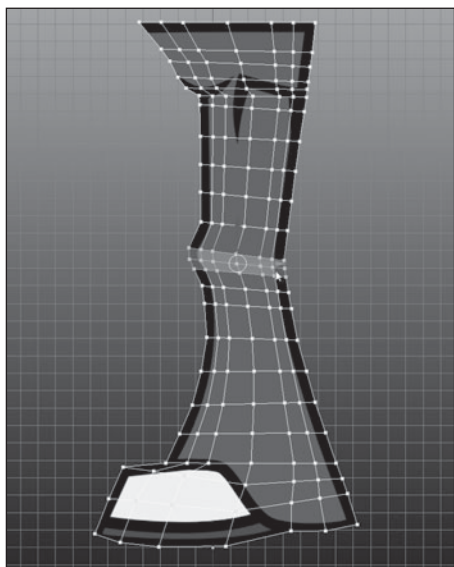
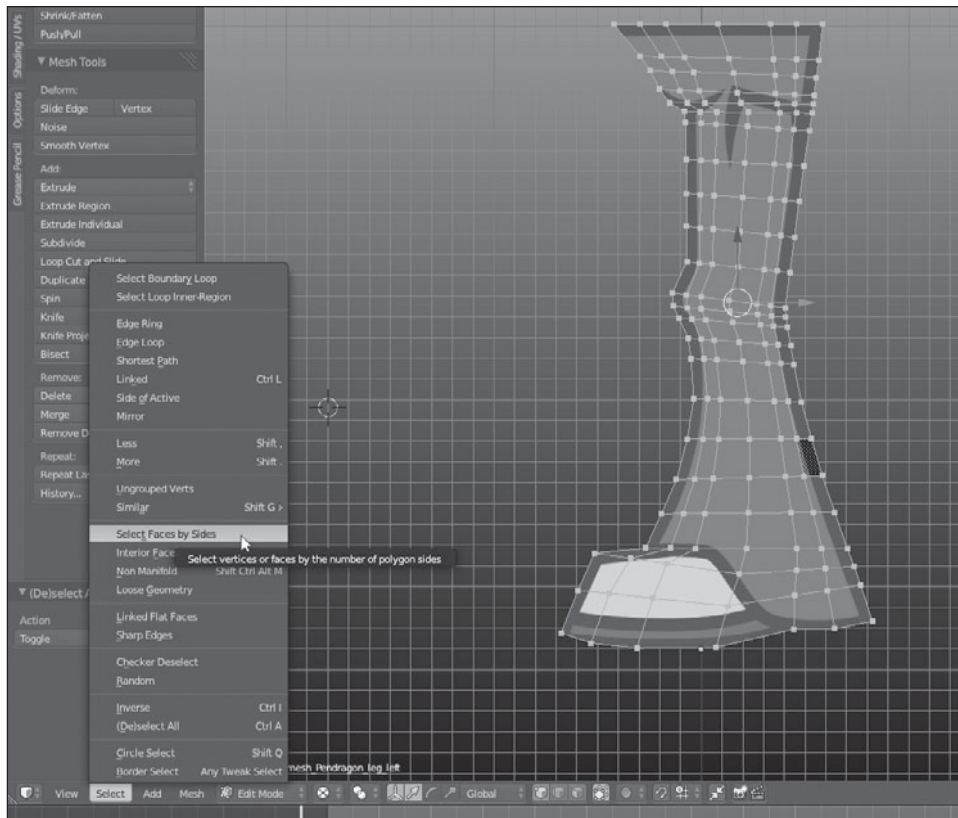


Figure 1.18

Use three- or four-sided polygons only, and increase resolution wherever the mesh will bend.

Source: The Blender Foundation.

1. Enter Edit mode for your mesh (press the Tab key) and choose Select > Select Faces by Sides from the 3D View menu. (See Figure 1.19.)

**Figure 1.19**

Detecting n-gons in a mesh using the Select Faces by Sides tool.

Source: The Blender Foundation.

2. Choosing Select Faces by Sides displays selection options in the toolbox on the left side of the screen, as shown in Figure 1.20. Deselect the Extend checkbox, enter 4 in the Number of Vertices Edit field, and choose Greater Than from the Type drop-down list. When these options are chosen, Blender will immediately and automatically select all faces in the mesh with more than four sides. (Of course, selecting the n-gons doesn't eliminate them from the mesh. It just indicates where they are. But this can make your work simpler, as you'll see.)

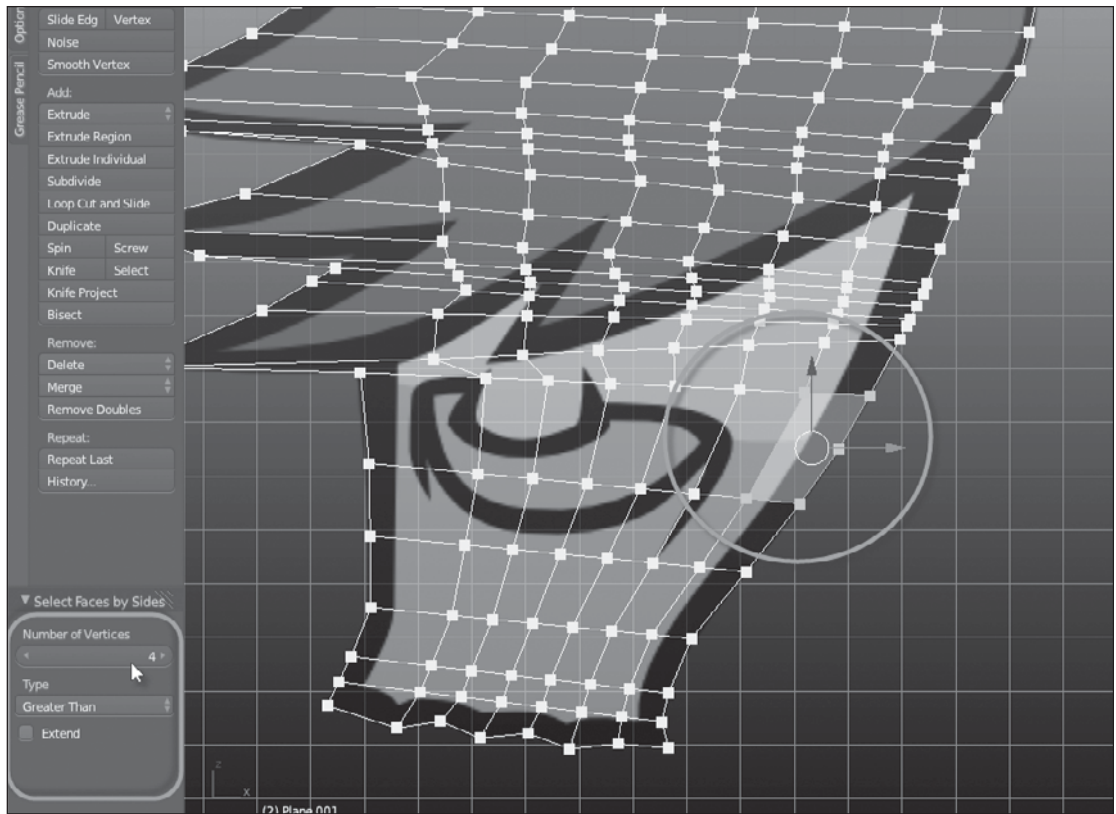
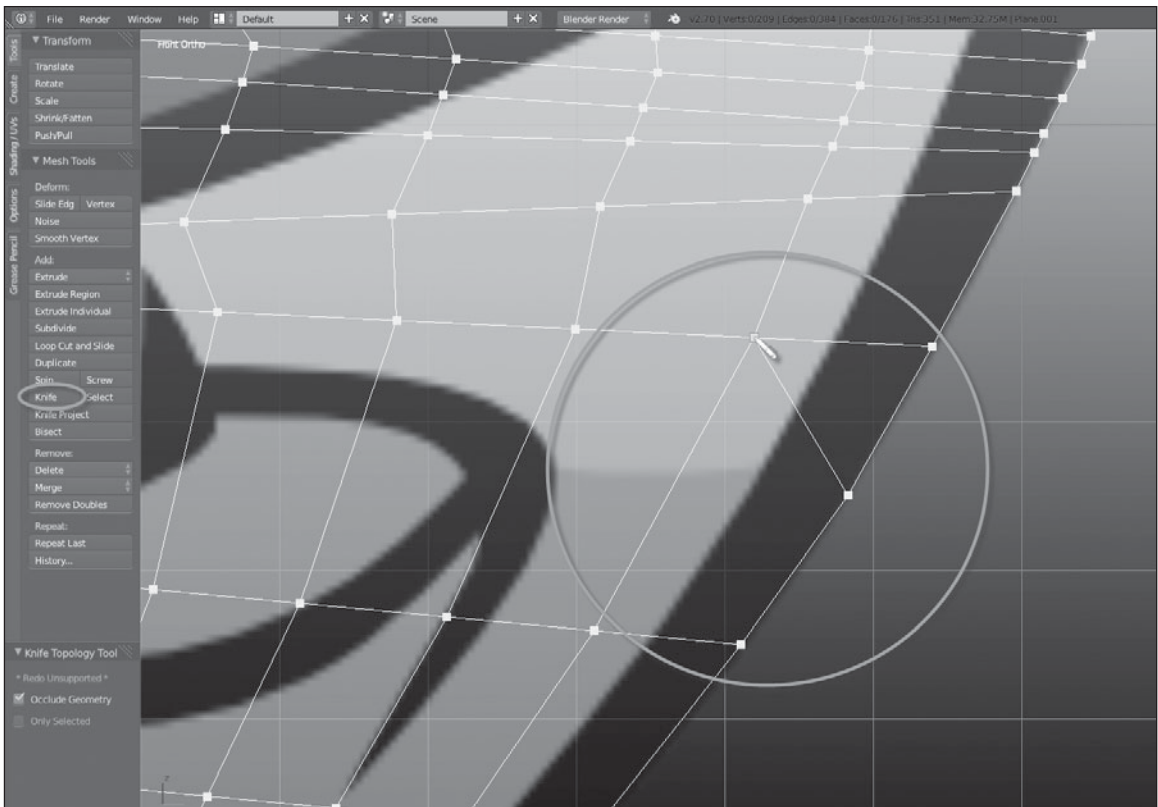


Figure 1.20

Selecting n-gons in a mesh. In this example, there's one n-gon on the face.

Source: The Blender Foundation.

3. One way to eliminate an n-gon is to use the Knife tool, rerouting mesh topology by cutting in new edge loops. You can access the Knife tool from the Tools panel when your mesh is in Edit mode. Simply click the Knife Tool button and move your cursor into the viewport, clicking on a start and end vertex where a new cut (edge loop) should be inserted. Be sure to press the Enter key when you're finished to confirm the changes and insert the cut. The idea is to insert new detail into the model to dissect the n-gon into either a triangle or a quad. See Figure 1.21 where the Knife tool splits an n-gon into one quad and a triangle.

**Figure 1.21**

Using the Knife tool to cut an n-gon into one quad and a triangle.

Source: The Blender Foundation.

If your 2D characters were never going to be animated, then n-gons would perhaps be your biggest enemy. But animation brings more threats and new guidelines. Specifically, whenever a mesh, like a leg or arm, bends, it affects the underlying mesh geometry, which also affects the nature of the bend. A mesh can only bend and deform wherever there are vertices. For this reason, arms and legs especially should be regularly tessellated. That is, they should have equally spaced edges and vertices to support a smooth and predictable deformation during animation, like a walk cycle. In addition, be sure to insert extra edge loops around hinged areas like joints such as the knee and hip. If you don't, the mesh will look sharp and rigid at the area of the bend. See Figure 1.22 to see how I've configured edge loops around the knee area, inserting additional detail to support a smoother bend deformation.

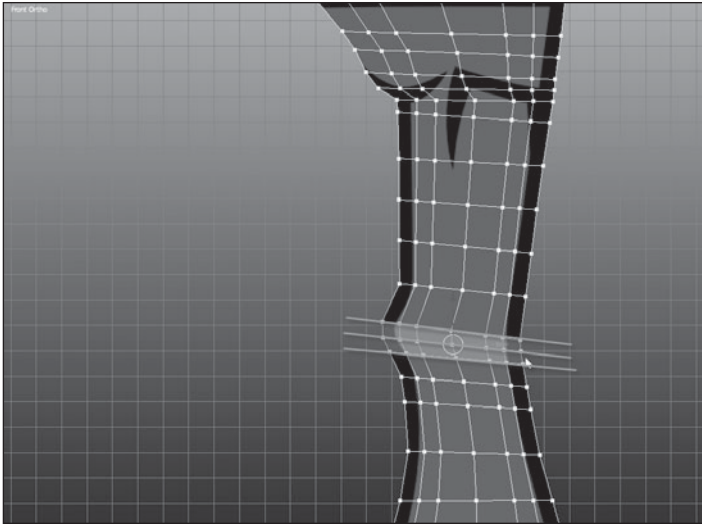


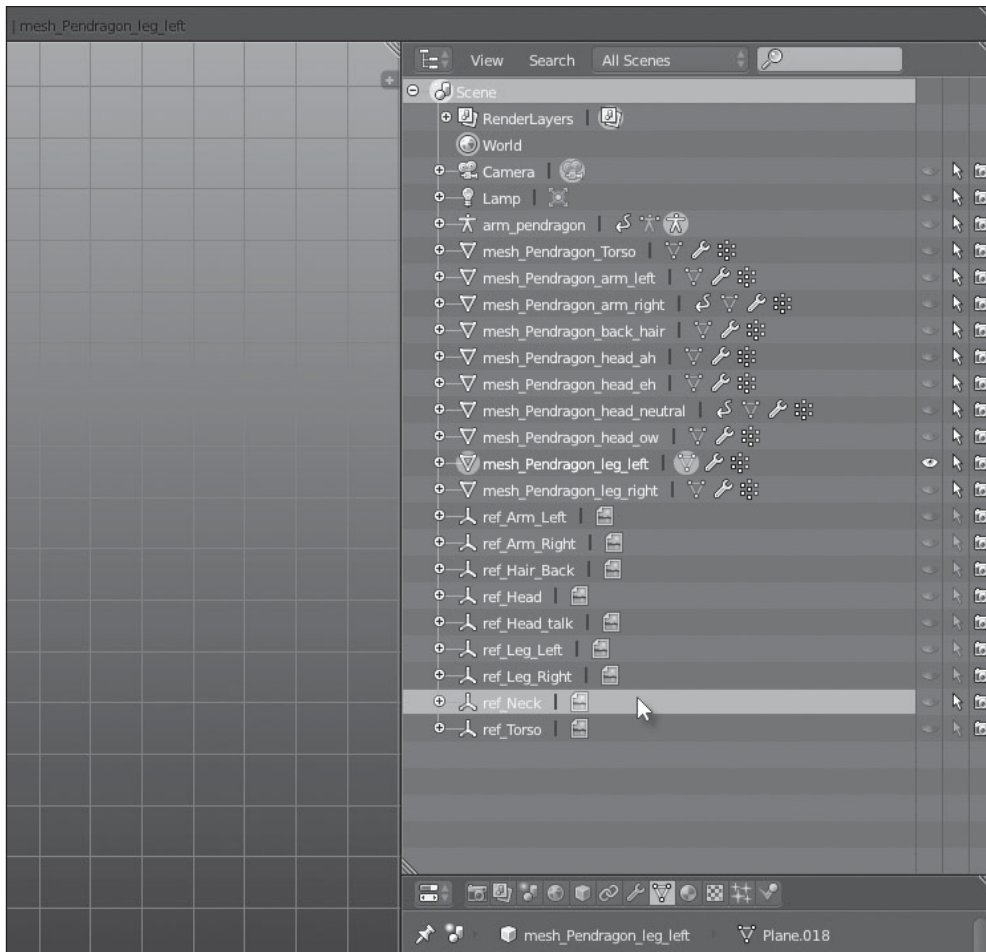
Figure 1.22

Adding extra edge loops around joints and hinges where a bend deformation may occur during animation.

Source: The Blender Foundation.

STEP 5: HOUSEKEEPING

After you've modeled everything for your character mesh, be sure to assign each element a meaningful name. Consider adopting a naming convention, using established prefixes and suffixes that suit your project. For example, you might prefix mesh objects with `mesh_`, reference objects with `ref_`, etc. In short, a user should be able to view only the Outliner panel and ascertain what objects are in the scene on the basis of object names alone. If the name is not descriptive in this way, then reconsider object names sooner rather than later. The names themselves are only for your benefit (and that of other team members), but they can significantly affect how much time you spend searching for objects. To rename an object, you can double-click its name in the Outliner window, as shown in Figure 1.23, and then type a new name. Alternatively, you can use the N-Panel (so named because you can press the N key to toggle its display). The N-Panel allows name editing from the Item field at the top. (See Figure 1.24.)

**Figure 1.23**

Assign meshes meaningful names to simplify object selection and project management.

Source: The Blender Foundation.

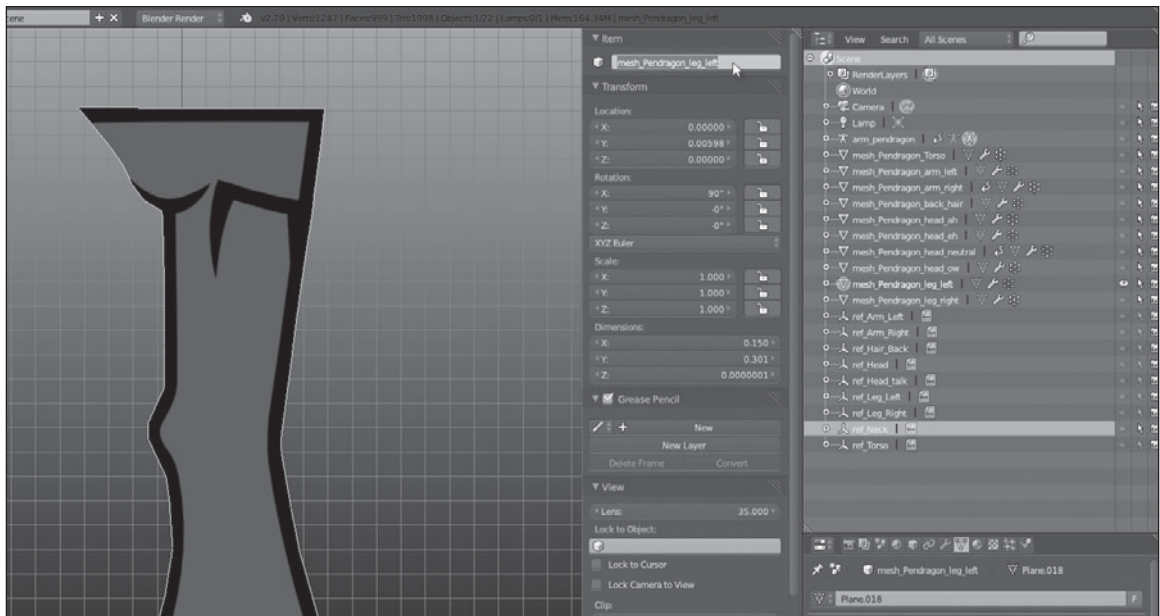


Figure 1.24
Renaming the selected object from the N-Panel.

Source: The Blender Foundation.

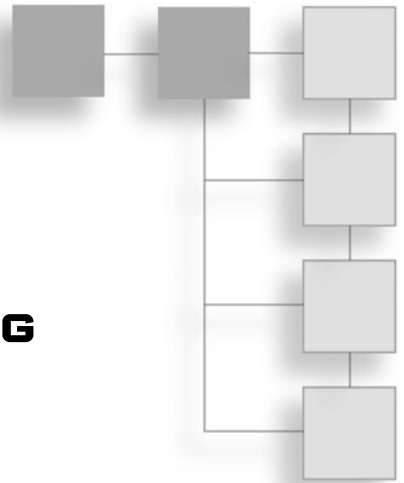
Note

In addition to a solid naming convention for objects, I also recommend using an incremental saving scheme for your Blender scene files. This is as simple as choosing Save As from the Blender main menu and appending `_01`, `_02`, `_03`, etc., to the end of each filename. This helps you build a backup history of files should you ever need to revert to them, and it always indicates the latest file, too (the highest number).

CONCLUSION

Modeling a 2D character from a reference is just the start of a longer process, but it's important nonetheless. The underlying mesh topology for a 2D character has significant consequences for how an object is textured, rigged, animated, and exported to a game. For this reason, it's critical to build your character mesh closely to your reference, shaping and spacing vertices in regular and patterned ways to achieve the best possible deformations when animating. Once modeling is completed, you'll end up with a gray-shaded mesh that has no material or inherent appearance. Consequently, the next step after modeling is to UV unwrap and texture the meshes, making them look like the character reference from which they were constructed. That subject is explored in the next chapter.

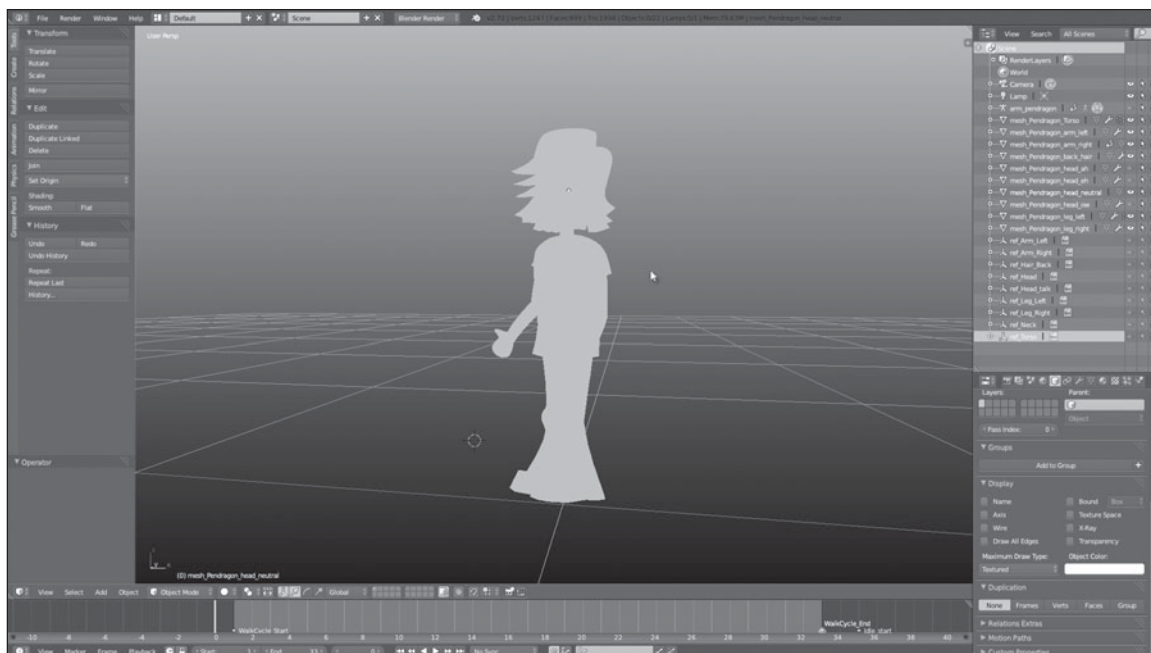
CHAPTER 2



UV MAPPING AND TEXTURING

The modeling stage, as defined in Chapter 1, “Preparation and Modeling,” is where the basic form and shape of your character mesh is created. As you saw there, you use reference images to construct separate meshes forming the limbs and moving parts of the character, like arms and legs. These meshes will ultimately be imported into a game engine like Unity, complete with texture and animation data.

By now, your completed Pendragon mesh should look like the one in Figure 2.1. This chapter focuses on assigning texture information to the mesh using Blender. It’s about drawing the character texture onto the mesh surface, effectively re-creating what is already viewable in the reference images.

**Figure 2.1**

The completed Pendragon mesh, ready for mapping and texturing.

Source: The Blender Foundation.

Before starting the texturing stage, it helps to be confident that your mesh topology is exactly as you need it. Be sure your vertices, edges, and faces are positioned and sized exactly where they should be in the mesh, free of n-gons. (For help removing n-gons from a mesh, see the section “Step 4: Optimizing Topology for Games and Animation” in Chapter 1.) Why must you be so strict about mesh structure? Because UV-mapping data is normally baked into the mesh vertices. That means changes in mesh vertices after mapping can potentially distort and corrupt your mapping data, requiring you to tweak the mapping or to remap from the beginning. Thankfully, as you’ll see, UV mapping a 2D character is far quicker and more straightforward than mapping regular 3D objects. Making mapping changes, if they’re even necessary, is not such a big deal.

Note

The completed Pendragon model (without texturing information) is included in this eBook's companion files in the Chapter02 folder. You can open up this file and follow with this chapter if you prefer.

STEP 6: REMOVING DOUBLES

No matter how carefully you model in Blender, doubled-up vertices (called *doubles*) can easily introduce themselves into your mesh. A double is where two or more vertices are stacked one on top of the other so that, to the eye, it appears that only one vertex is present. These are typically introduced into your mesh through Extrude and Bevel operations, especially if you've attempted to undo them. These operations, though initiated with one click of a button, often involve multiple steps under the hood in Blender, and they require multiple undos to be fully undone. That's why a single undo can sometimes leave you with doubles.

Thankfully, Blender offers an easy tool for removing doubles: Remove Doubles. It's important to use this before mapping to make sure your mesh is structured exactly as it appears. To remove all doubles from a mesh, follow these steps:

1. Enter Edit mode by pressing the Tab key (Maya style controls).
2. Press Ctrl+A to select all vertices.
3. From the 3D View menu, select Mesh > Vertices > Remove Doubles. (See Figure 2.2.) This removes all doubles by merging them into one vertex. That is, it merges together all separate vertices occupying the same location.

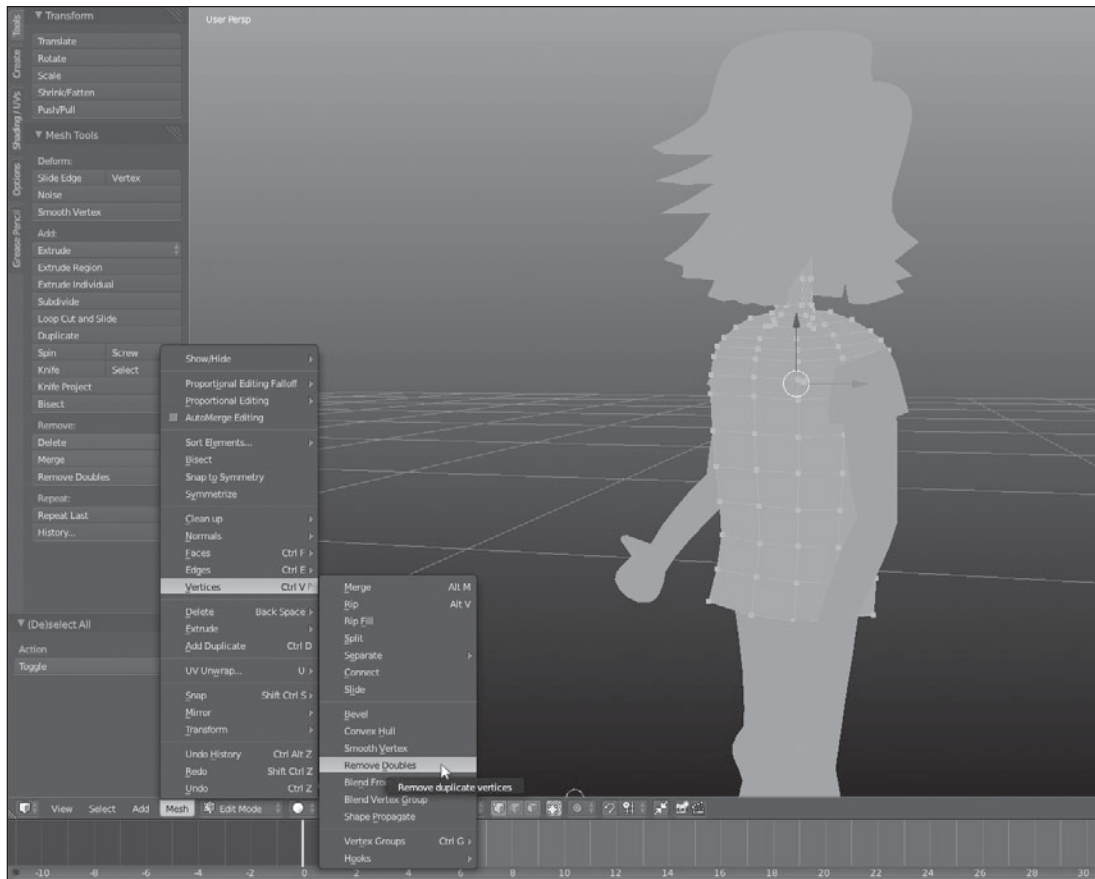


Figure 2.2
Removing all doubles in a mesh in preparation for mapping.

Source: The Blender Foundation.

STEP 7: CREATING THE CHARACTER ATLAS TEXTURE

Before UV mapping a character mesh, you must first think about the texture you'll be mapping. The reference images used so far for building the mesh contain all the texture data you need, but the different limbs and elements were stored in separate texture files. This is both inconvenient and problematic. For texturing an object, it's better to consolidate all the different elements onto one texture, known as an *atlas*. (See Figure 2.3.) This has a couple of benefits. First, you can map all the character parts within the same texture space, reusing and recycling the same texture. Second, atlas textures can offer enhanced run-time performance for many different engines, include Unity and the Unreal Engine.

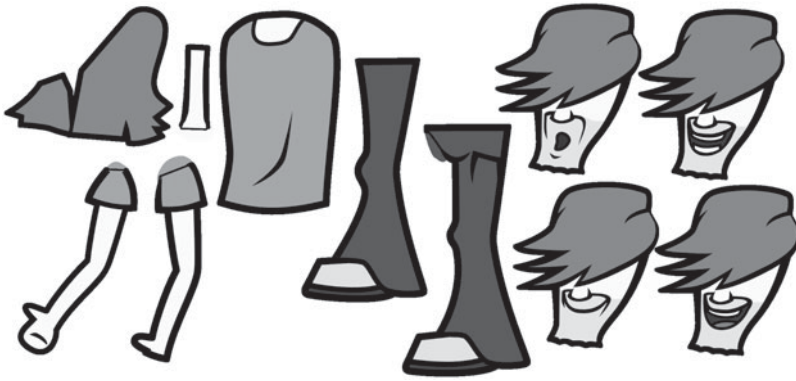


Figure 2.3

Copying and pasting the character parts onto a single texture sheet (in other words, an *atlas texture*).

Source: The Blender Foundation.

When creating an atlas texture, avoid positioning elements close to the image's edges. Leave some space (known as *pixel padding*) between the elements and the image's edges. This optimizes your texture for real-time game renderers. If you don't do this, you may observe artifacts and glitches around the edges of your texture when applied to the mesh in-game.

In addition, notice in Figure 2.3 how the outline pixels of character elements have been expanded. The exterior black borders are thicker and the blue color on the sleeve goes beyond the shoulder joints. These expanded areas were created by standard paint tools found in most paint programs. Their purpose is to create some pixel duplication for each element. This is known as *manual clamping*. Its aim is to compensate for the way real-time render systems draw textures. They sometimes render border pixels or pixels just slightly outside where they should go. Including expanded pixels around your elements in this way ensures that mismatching background pixels won't be drawn instead.

Note

Common paint programs used for creating game textures include Adobe Photoshop (<http://www.adobe.com/uk/products/photoshop.html>), Paint Shop Pro (<http://www.paintshoppro.com>), and GIMP (<http://www.gimp.org/>).

STEP 8: UV PREPARATION AND PROJECTION

Let's return to Blender to map the Pendragon mesh. UV mapping is a mathematical process in which a 3D object is structurally unraveled and flattened out onto a plane, allowing a two-dimensional texture to be mapped on it. The mapping information ultimately

defines how a texture should be projected onto a 3D surface, making it look like something believable as opposed to a dull gray object. (Refer to Figure 2.1.) When mapping 3D objects like cubes, cylinders, and spheres, you typically insert cuts or splices into the model at locations where it should come apart to unfold. In Blender, these cuts are known as *seams*. 2D objects and characters like Pendragon, however, don't need manually marked seams. That's because, being flat and 2D, they are by nature already unwrapped.

To start the UV-mapping process for Pendragon, follow these steps:

1. Switch to the UV Editing interface in Blender. To do so, click the Screen Layout drop-down list (found in the Info panel at the top of the interface) and choose UV Editing, as shown in Figure 2.4. This activates the UV Editing interface, shown on the left side of Figure 2.5. (The viewport is on the right.)

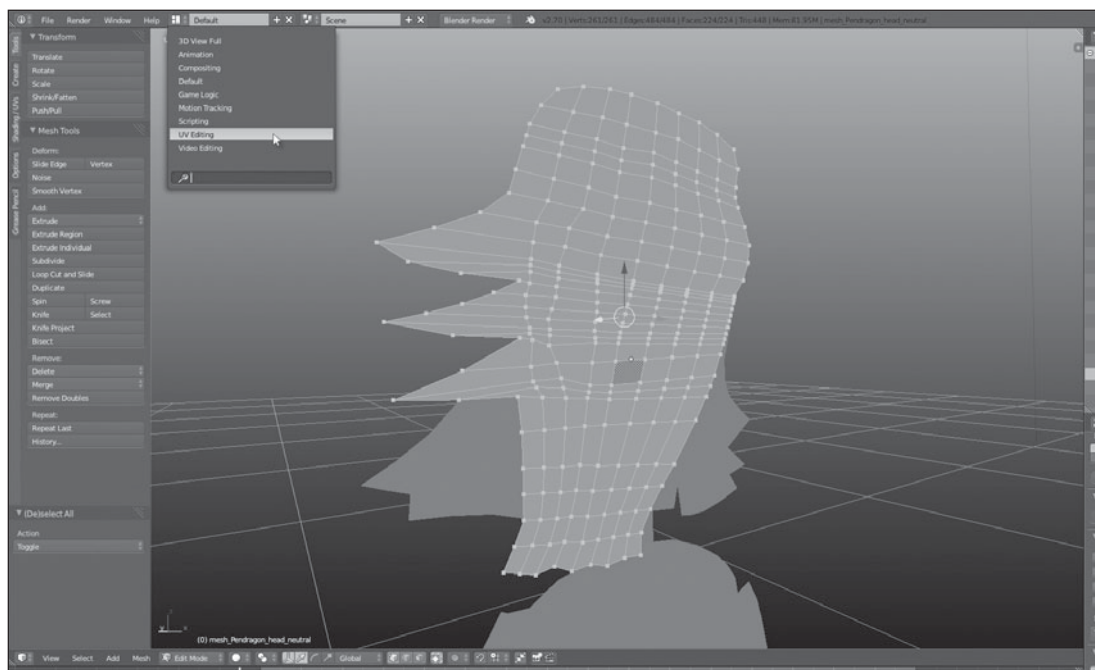
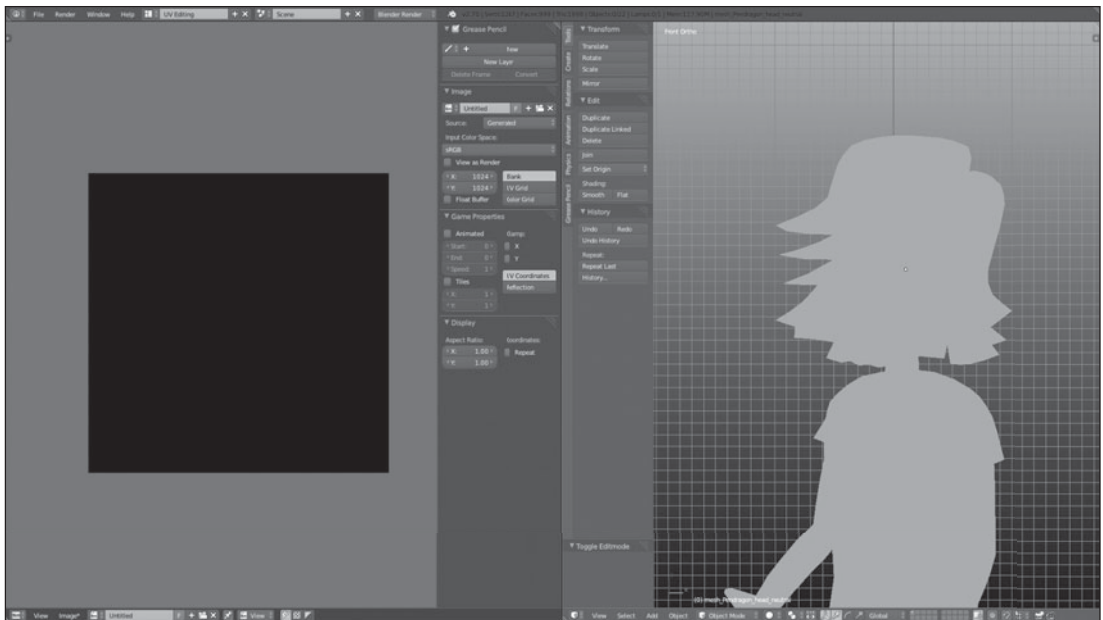


Figure 2.4
Getting ready for UV mapping.

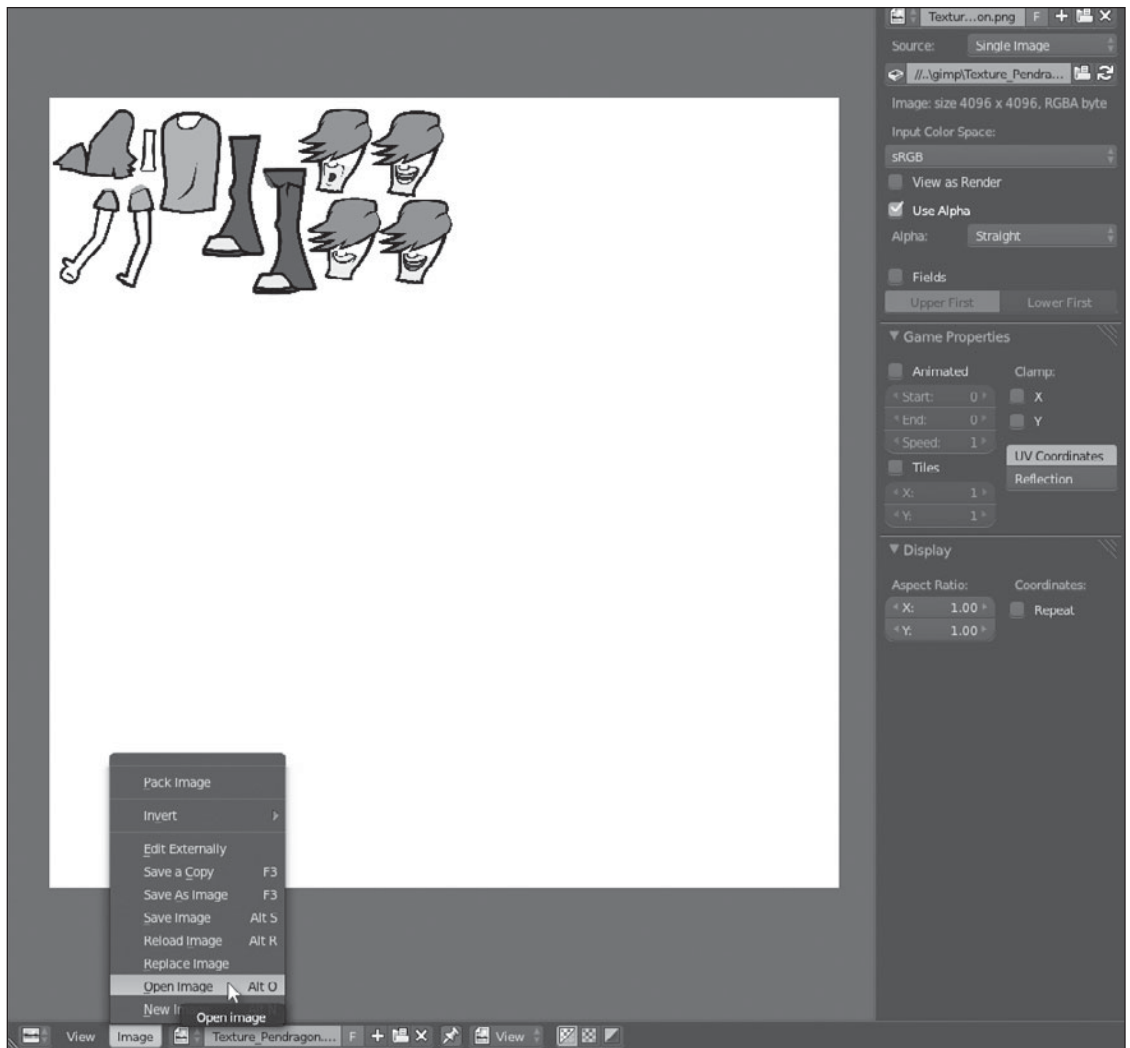
Source: The Blender Foundation.

**Figure 2.5**

The UV Editor is useful for mapping objects.

Source: The Blender Foundation.

2. In the Image Editor, choose Image > Open Image (see Figure 2.6) or press Ctrl+O. Then select your texture from the dialog box that appears. In this case, notice how large the texture is and how little space the Pendragon elements occupy. This is because I plan to add more characters and graphics to the same texture sheet. If you're texturing just one character, however, then you should size your texture according to your needs—no larger and no smaller. Smaller textures cannot accommodate your needs, and larger textures will have wasted space.

**Figure 2.6**

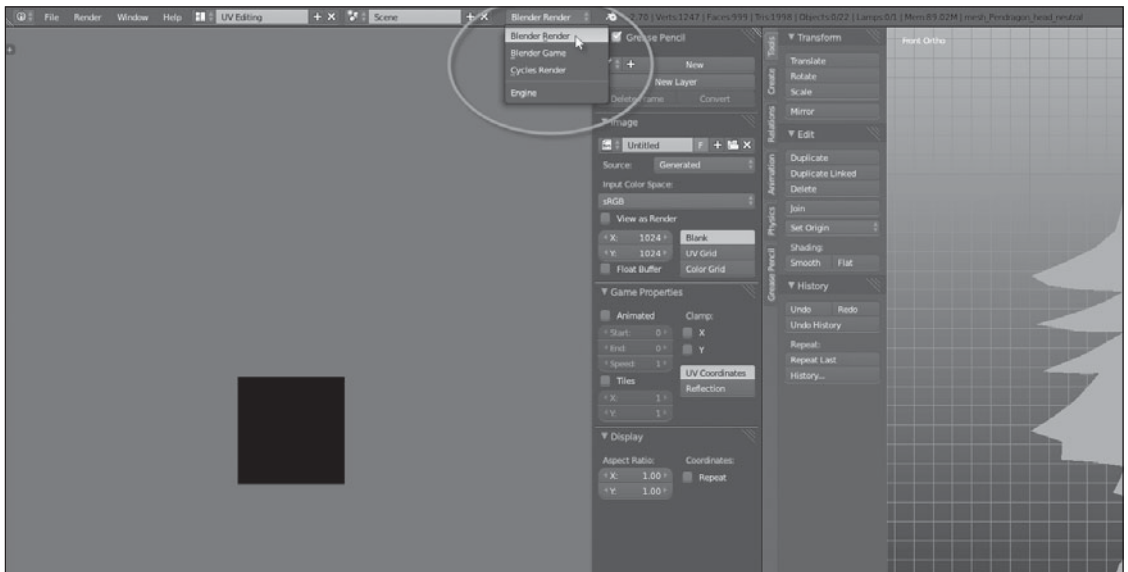
Loading the Pendragon atlas texture into the UV Editing interface.

Source: The Blender Foundation.

3. Make sure Blender Render (rather than Cycles Render) is selected as your active render system. To do so, open the Renderer drop-down list in the Info panel and choose Blender Render, as shown in Figure 2.7.

Note

This step is not essential, but it is recommended. This setting is important for getting reliable and clear real-time previews of your mesh and its texture in the viewport. When the texture is applied to your mesh, the Cycles renderer will typically shade the mesh black by default. In contrast, Blender Render will immediately give you a real-time preview of the mesh with the texture applied. No further settings or adjustments are required.

**Figure 2.7**

Setting the active renderer to Blender Render for easiest real-time texture previews.

Source: The Blender Foundation.

- Let's start mapping the Pendragon mesh. Each element should be mapped individually: head, arms, legs, torso, etc. Although the order doesn't really matter, I typically start with the head. To begin, select the head mesh in the viewport and press the Tab key to enter Edit mode.
- Press Ctrl+A to select all the faces in the mesh. You're selecting all the faces here because all of them should map together onto the texture. When the faces are selected, you may see a wireframe UV layout in the UV Editor, depending on how your mesh is configured. (Right now it doesn't matter either way because you'll be generating new mapping for the object soon.)

- With the faces selected, click the Browse Image button in the UV Editor and select the atlas texture from the list that appears. This assigns the atlas texture to the selected faces. Initially, no tangible or visible change may be apparent. (See Figure 2.8.)

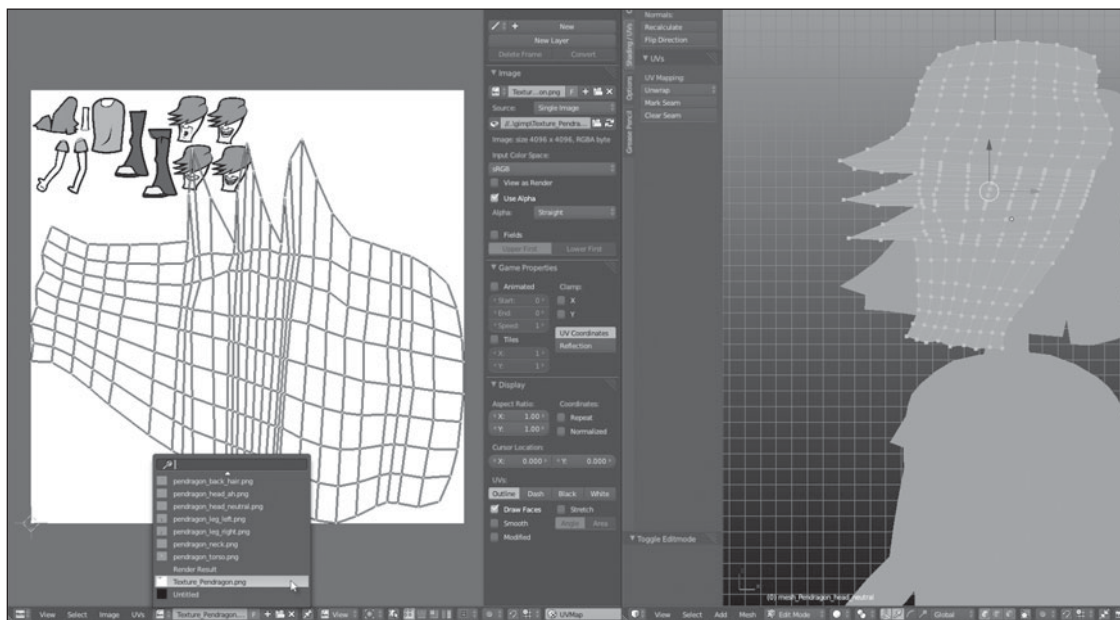


Figure 2.8

Assigning the atlas texture to the selected faces.

Source: The Blender Foundation.

- Apply a UV project to the Pendragon head mesh. This will unwrap, or auto-calculate, how the mesh should be unraveled into texture space. To achieve this, first switch your viewport to a front orthographic view to get a direct view of your mesh. Then select the head mesh, enter Edit mode, and select all faces. Finally, switch to the Shading/UVs tab in the toolbox, click the Unwrap drop-down list, and choose Project from View. (See Figure 2.9.) The result should look like Figure 2.10. As you can see, projecting a layout from an orthographic view produces a UV layout in the Image Editor that matches exactly with the head mesh in the viewport.

Caution

Don't project from a perspective view because you'll end up with distortion. If you accidentally project from the wrong view, repeat the Unwrap > Project from View operation to generate a new layout, overwriting the previous one.

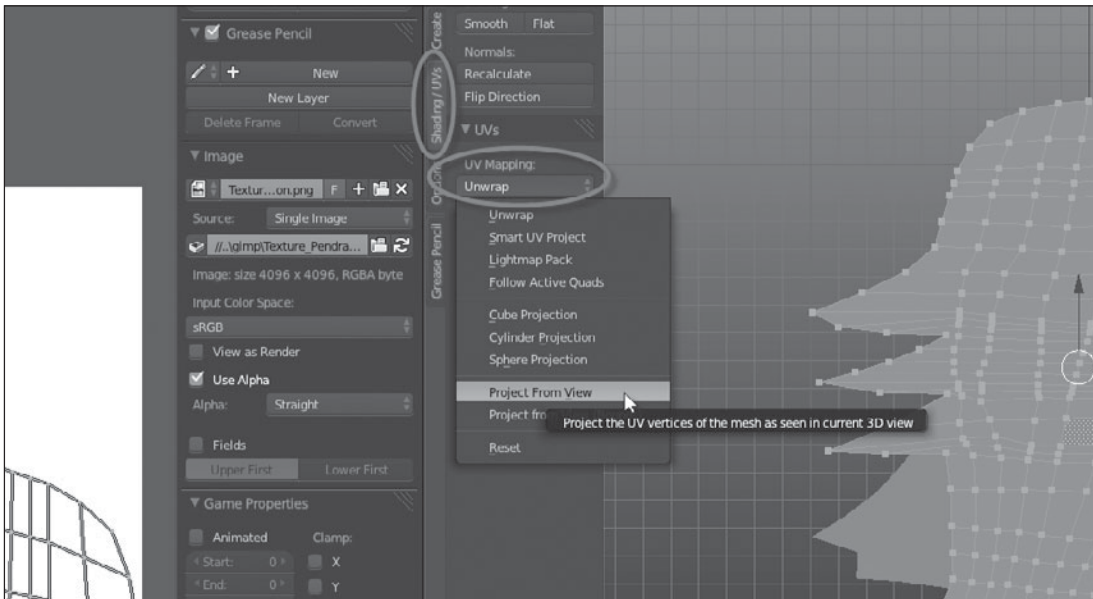


Figure 2.9
Generating a UV layout by projecting from an orthographic view.
Source: The Blender Foundation.

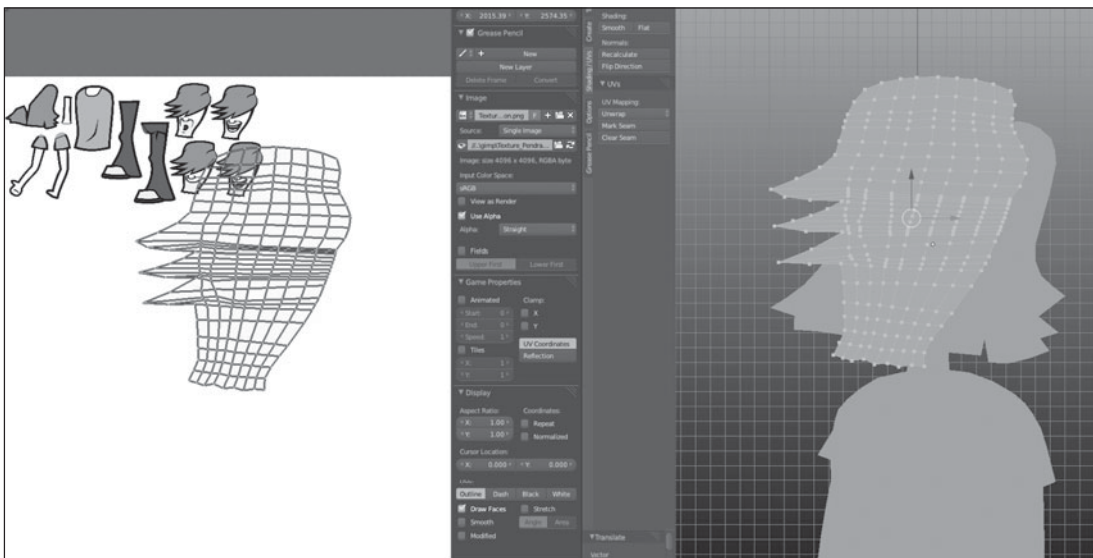


Figure 2.10
Project from View produces a UV layout that matches the mesh shape in the viewport.
Source: The Blender Foundation.

8. The UV layout is looking good in the Image Editor, but it's also useful to see the results of that mapping applied to the mesh surface as a texture in the 3D viewport. Your view might already be textured, depending on how Blender is configured. If it's not, you can enable texture previewing by switching the 3D Viewport Shading mode to Textured, as shown in Figure 2.11. Notice that the head's texture matches the layout in the Image Editor.

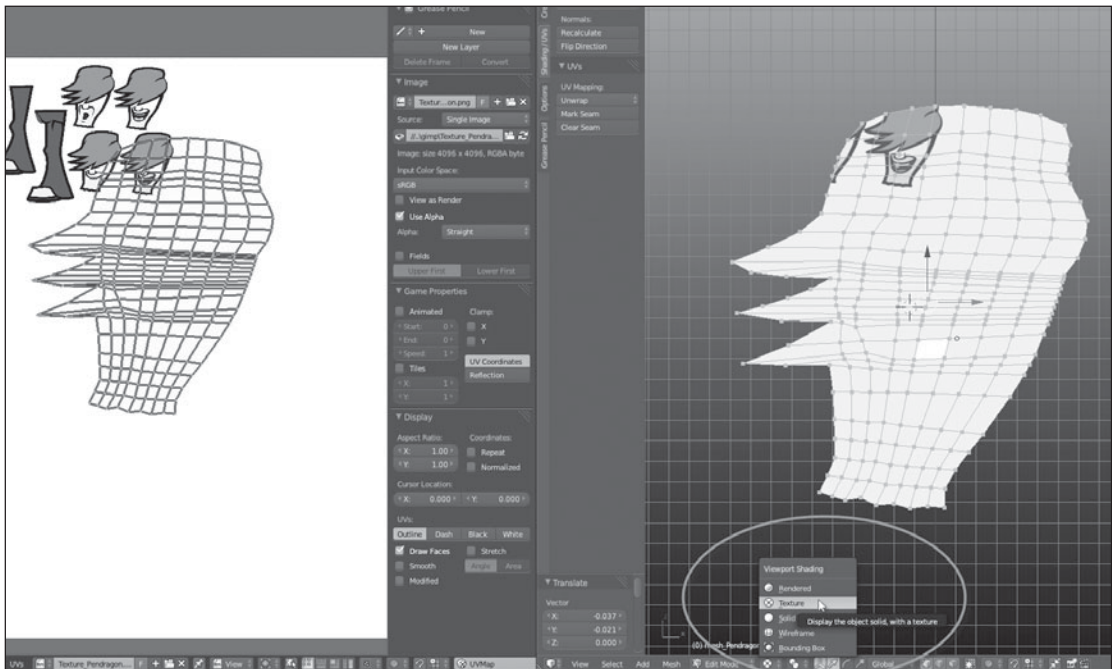


Figure 2.11

Displaying texture mapping in the 3D viewport.

Source: The Blender Foundation.

Note

If your mesh looks completely black in the viewport after you switch to the Textured view, try enabling Shadeless display. To do this, press N to open the N-Panel and select the Shadeless checkbox in the Shading group.

STEP 9: CREATING UV ISLANDS AND MAPPING

When you project a UV layout from the mesh, a UV island is generated in the UV Image Editor. An island represents a single body of connected vertices (UVs) in the Image Editor. Thus, the complete UV layout for the Pendragon head is a single island, and it controls the mapping for the entire mesh.

It's unlikely that the mapping will look exactly as you need it to after projection, even though the layout does reflect the mesh proportions and shape. The island will typically be too large or too small and positioned incorrectly within the texture, as shown in Figure 2.12. Before editing the mesh UV layout to correct these issues, first enter Island mode. This restricts your selection to complete islands as opposed to individual vertices (called UVs in the Image Editor). Simply right-click inside the Image Editor and choose Island from the context menu that appears. Alternatively, click the Island Selection button in the bottom toolbar of the Image Editor. (See Figure 2.13.)

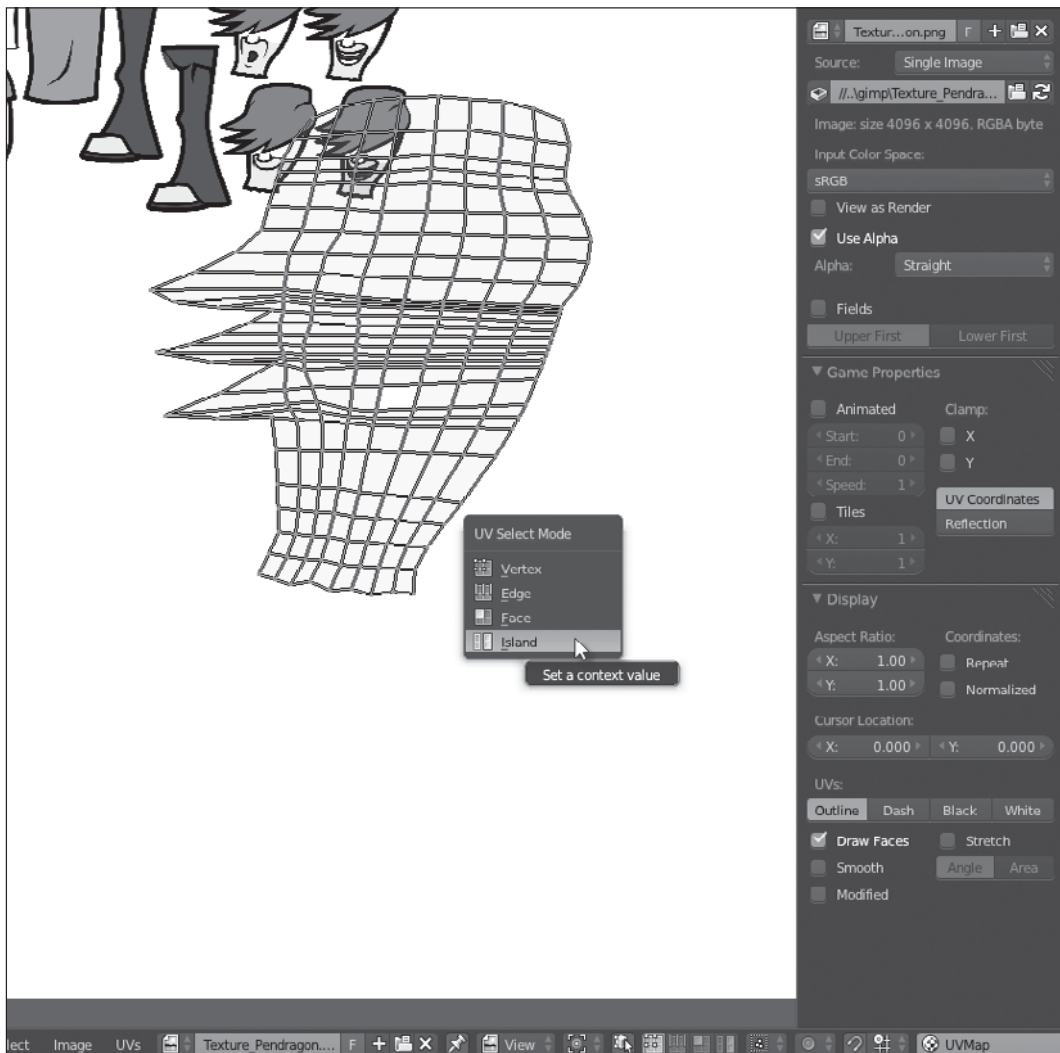
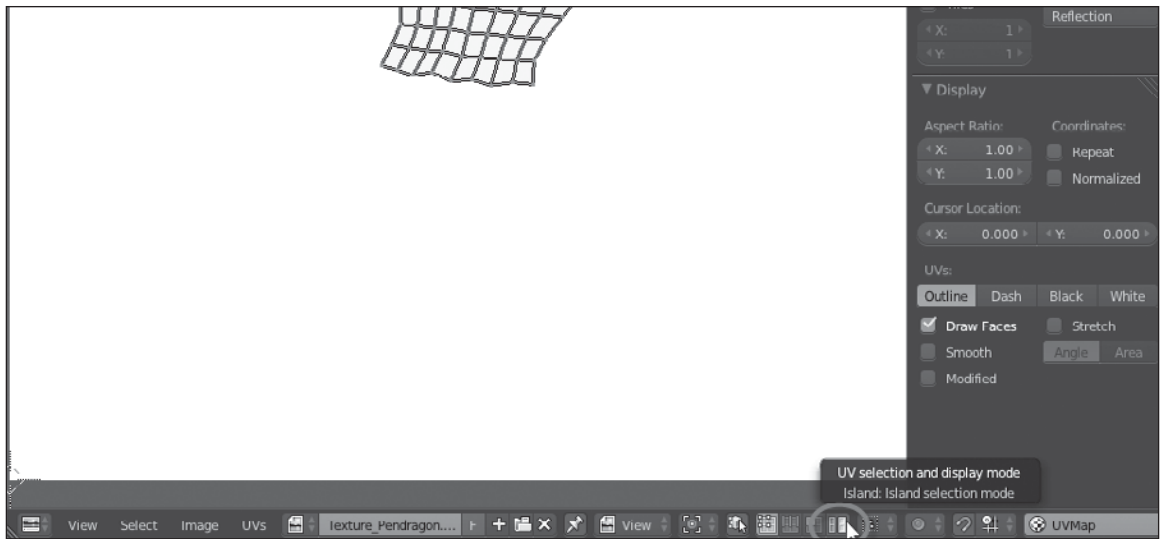


Figure 2.12

Editing mapping using UV islands.

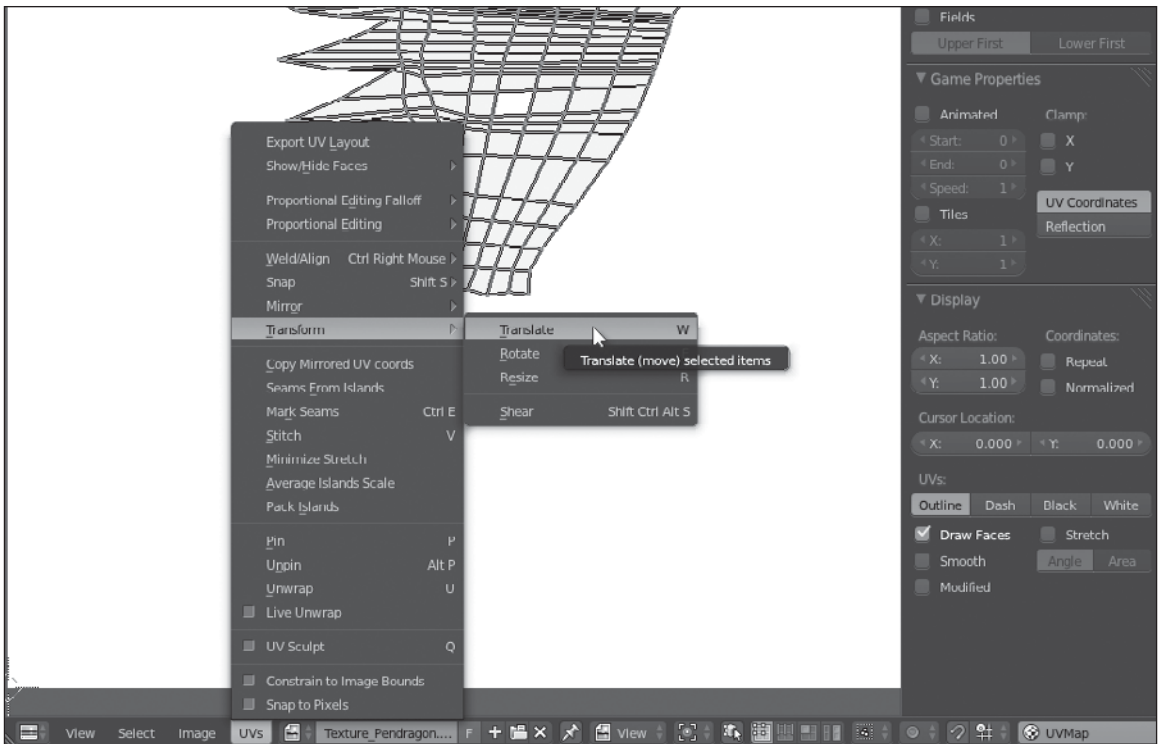
Source: The Blender Foundation.

**Figure 2.13**

Island Selection mode enables you to select and manipulate complete UV islands as opposed to single vertices (UVs).

Source: The Blender Foundation.

You can reposition and resize islands through scaling and translation. The UV Image Editor supports the transformation of islands, just as the 3D view supports the transformation of meshes. Simply click a UV island and press W for movement or R for Scaling (Maya). You can also access the Transform tools for islands (Translate, Rotate, and Scale) from the UV Image Editor by choosing UVs > Transform. (See Figure 2.14.)

**Figure 2.14**

Translating UV islands.

Source: The Blender Foundation.

Using the Transform tools for islands, position and scale the head island into its proper place in the texture in the UV Editor, matching the shape and structure of the island to the head in the texture. As you do, observe the effects your movements have in the 3D viewport, as the texture conforms to the mesh in a live update. Continue to scale and move the island until the texture looks correct on the mesh in the viewport. (See Figure 2.15.)

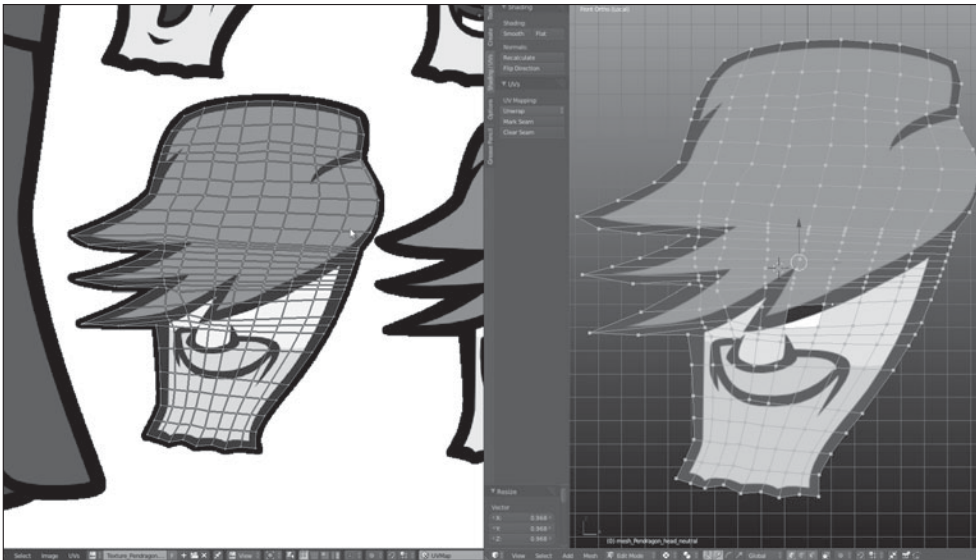


Figure 2.15
Transforming the Head UV island into position.

Source: The Blender Foundation.

Remember to slightly scale down the head UV island, leaving some extra padding around the outside where there's a black border. This is because in step 7, you created an atlas texture where each part of the body was expanded with extra edge pixels, like a bolder black border. This is a safety margin of pixels to compensate for renderer inaccuracies during gameplay and should not be intentionally mapped onto the mesh. (See Figure 2.16.)

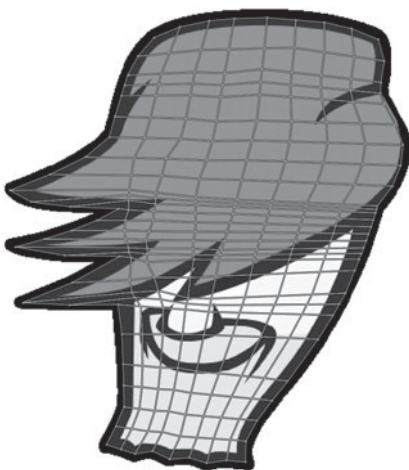


Figure 2.16
Leave some texture padding outside the UV island.

Source: The Blender Foundation.

Sometimes simply translating and scaling an island, as you've done here, won't be enough to get it looking right on your mesh. In these cases, you'll need to manually tweak specific UVs and to individually match the island with the texture more accurately. To achieve this, right-click in the UV Image Editor and select Vertex (see Figure 2.17). Alternatively, choose Vertex Mode from the UV Image Editor menu.

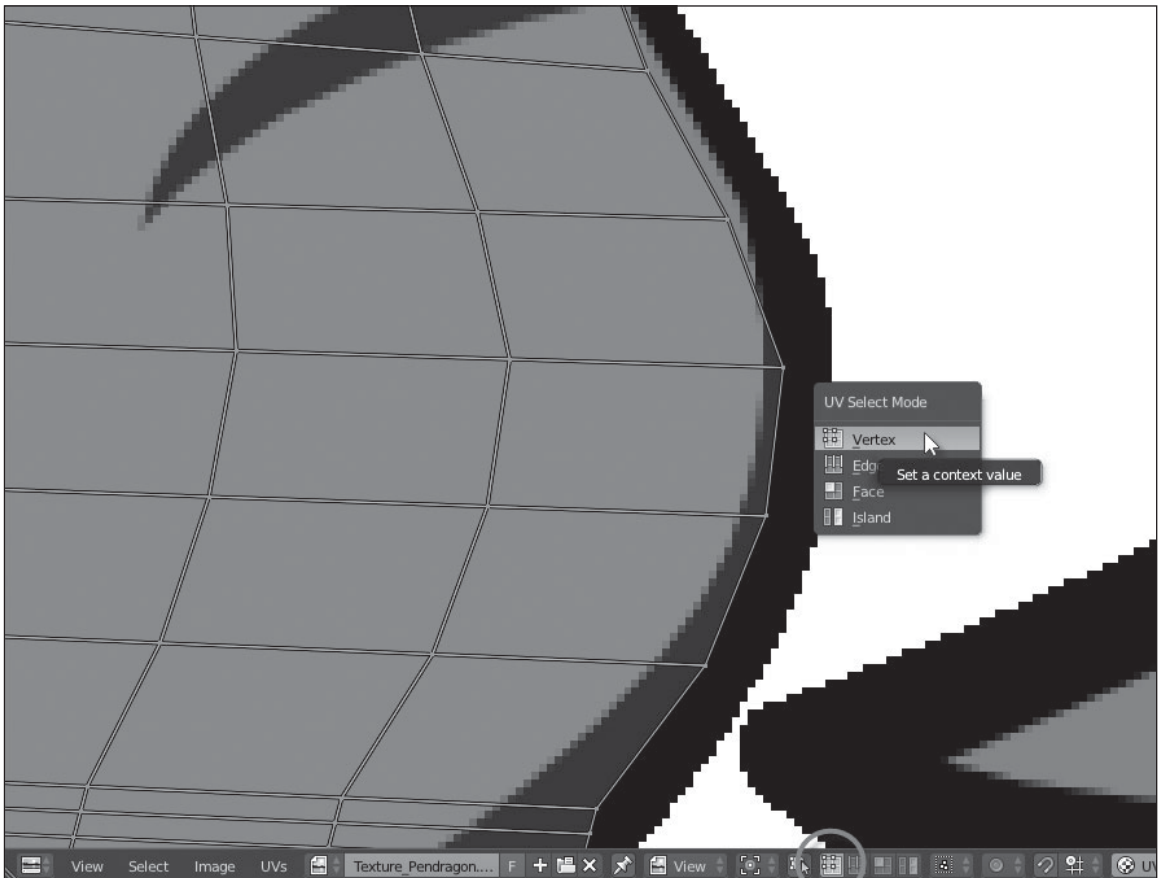


Figure 2.17

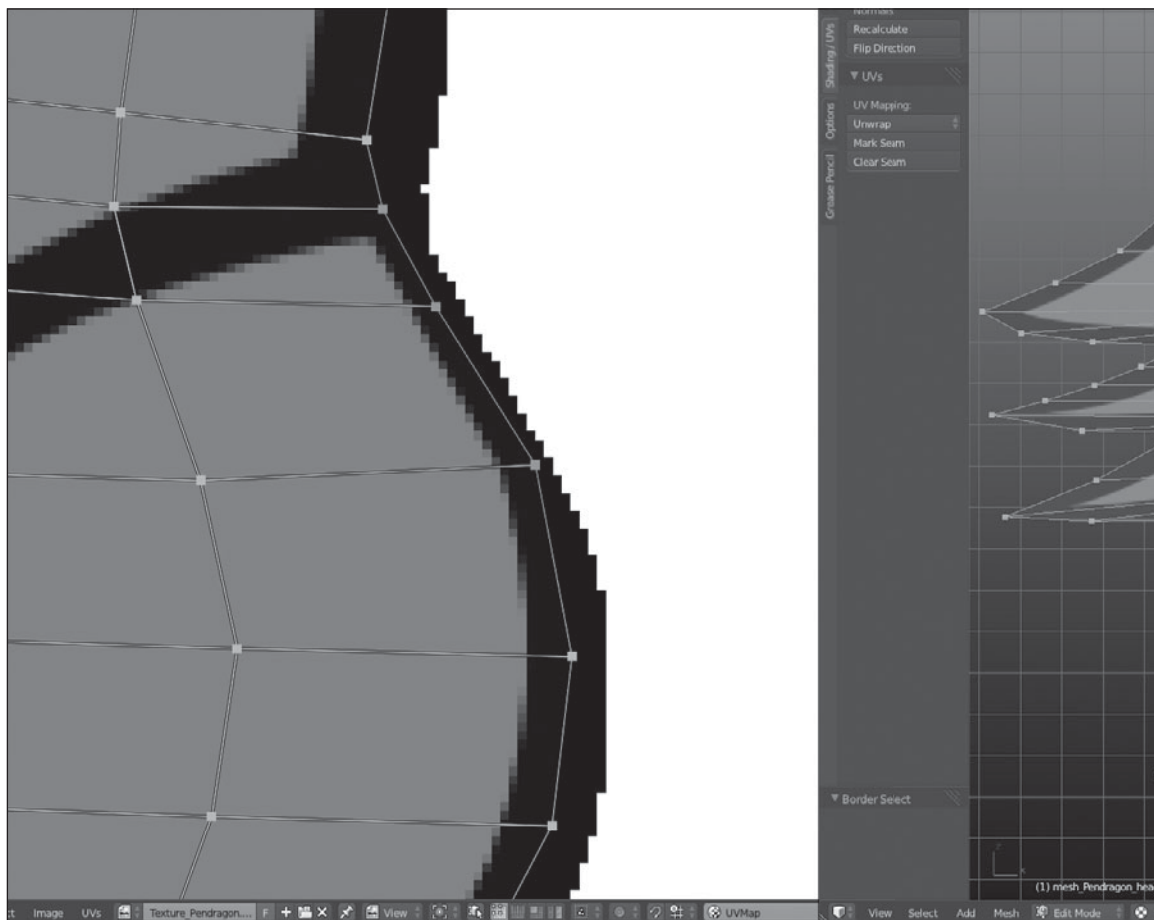
Vertex mode enables you to tweak specific UVs in an island, just like tweaking vertices in a mesh.

Source: The Blender Foundation.

With Vertex mode active in the UV Image Editor, you can select and transform UVs like regular vertices. This lets you select erroneous vertices and move them into place to bring the island into line with the texture. (See Figure 2.18.)

Note

Unfortunately, Blender doesn't currently support a transform gizmo (axes) for the UV Image Editor. I'd really like to see this feature. Right now, you must enter Translate mode and use the mouse to move the vertices to their new location. No gizmo or visual indicator appears in the interface when translation occurs.

**Figure 2.18**

Translate UVs into place to tweak and refine a UV island.

Source: The Blender Foundation.

STEP 10: COMPLETING THE MAPPING

You should now have completed the UV mapping for the character mesh's head element. Now repeat the procedure for all other parts of the mesh: arms, legs, torso, and back hair. (See Figure 2.19.) The same procedure should apply for almost every character. The most important thing is the result, however. If it looks good on the mesh in the viewport, then it should look good in-game.

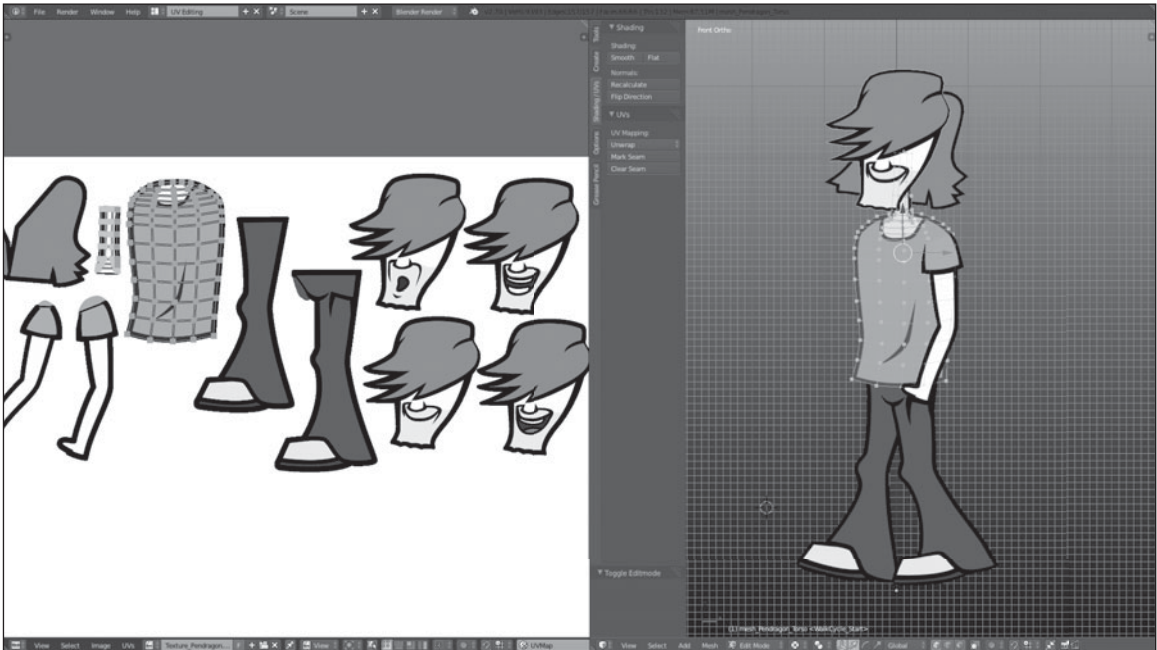


Figure 2.19

Completing the mapping for Pendragon.

Source: The Blender Foundation.

Use the final result as a measure of your progress and quality. Should you ever need to export the UV layout for a mesh as a renderable image (for reference or texture refinement), you can do so directly from the Image Editor. Select all vertices in the UV island (press Ctrl+A) and then choose UVs > Export UV Layout. (See Figure 2.20.)

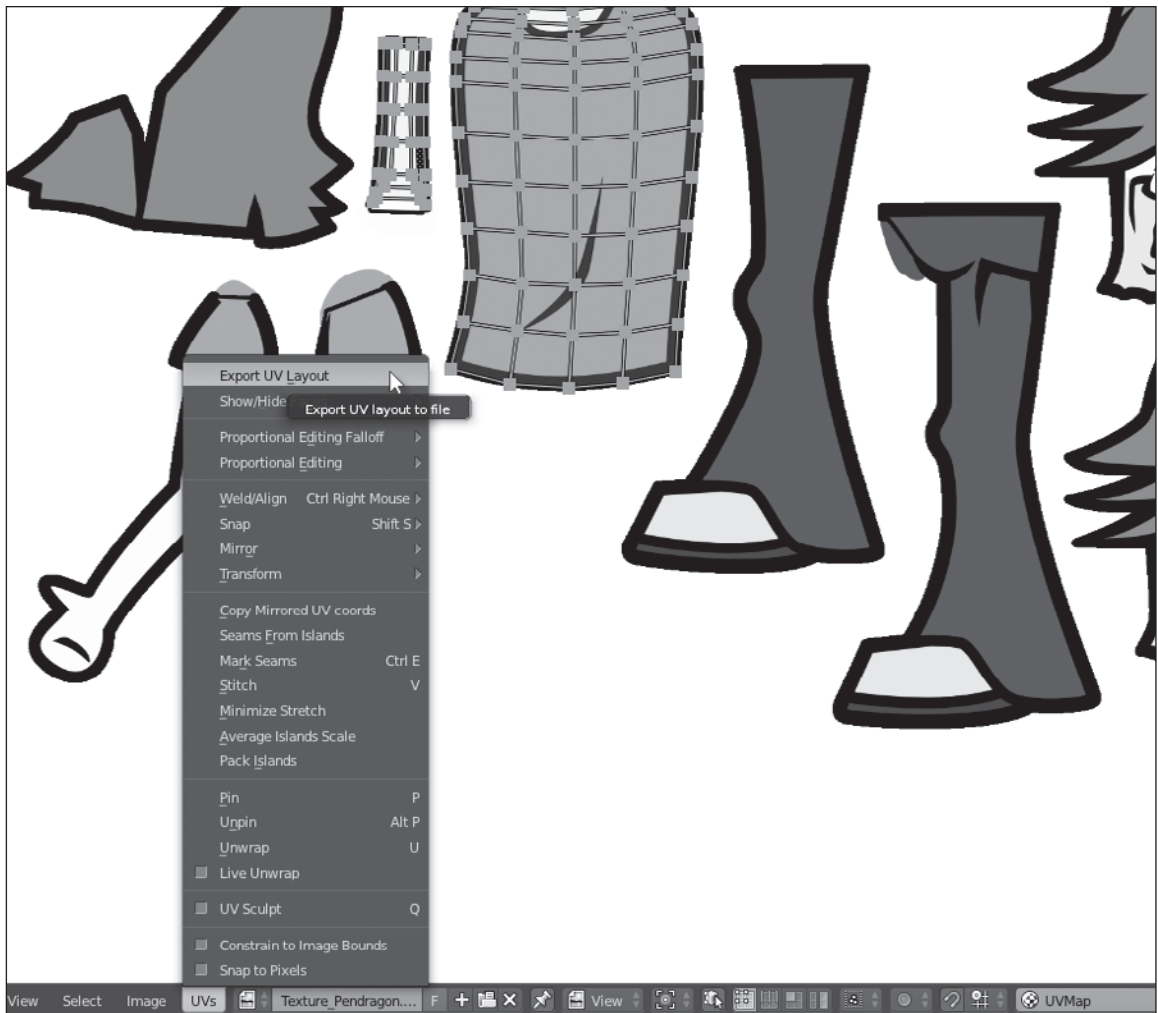


Figure 2.20
Exporting a UV layout from the Image Editor.

Source: The Blender Foundation.

The exported UV layout for the Pendragon torso looks like Figure 2.21. These kinds of UV layout renders can be useful for planning textures and understanding where elements are placed so as not to overlap them. They can also be used for creating wireframe effects. For Pendragon, however, no such UV layout images are required.

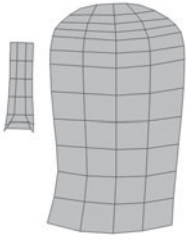


Figure 2.21

The exported UV layout for the Pendragon torso.

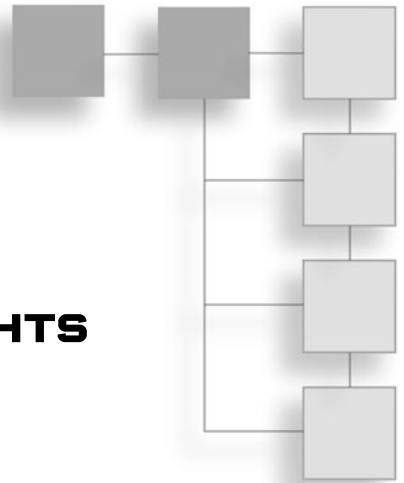
Source: The Blender Foundation.

CONCLUSION

You should now have a completely modeled and textured Pendragon character that looks just like the character reference image. It's an important step, and represents an end point of sorts. Specifically, the character is now modeled and textured, and is created in full as a game-ready entity. What remains is the animation part—where you configure the character to walk in a complete, looping cycle. Even at this early stage, you could import the character into a game engine, but he would be imported as a static and motionless entity—looking more like a cardboard cutout than an animated and living thing. Now it's time to breathe that kind of life into the model. In the next chapter, you'll explore rigging, a preparatory stage to animation.

This page intentionally left blank

CHAPTER 3



RIGGING, BONES, AND WEIGHTS

When you’ve fully modeled and textured your character, as you have Pendragon, it’s easy to feel excited about animation. It’s considered a “best practice” to think about animation early in this way, getting as clear and precise an idea of the range of animations needed. Indeed, you shouldn’t approach animation without such ideas! But even beyond these, it’s important to prepare technically and logistically for animation, making sure your mesh is the kind of object that Blender can animate well, with all the power you need. This kind of software-specific preparation is known as rigging.

With rigging, you generate a skeleton or bone structure underpinning the mesh, ensuring every joint or bend point has a corresponding set of bones in the skeleton. A link is then generated between the skeleton and the mesh so that when the skeleton moves, the mesh will move along with it, conforming and bending. (See Figure 3.1.) The linkage between a skeleton and a mesh is known as skinning information or weighting. Creating a skeleton object that is separate from the mesh is useful because in theory, the skeleton can be reused on other bipedal meshes. This enables you to run a single animation on multiple objects.

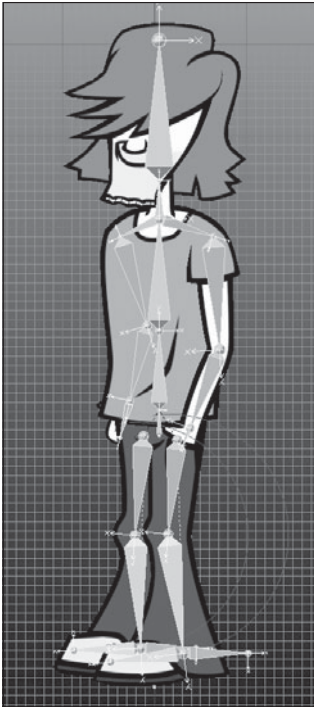


Figure 3.1
Rigging defines the underlying mechanical structure of a mesh.

Source: The Blender Foundation.

In this chapter, you'll explore the complete process of rigging Pendragon using bones and weights, resulting in a character that deforms well during animation. You will use Blender's animation tools to create the actual walking animation in Chapter 4, "Animation."

Note

The completed Pendragon model, ready for rigging, is included in the eBook companion files in the Chapter03 folder. You can open this file and follow with this chapter if you prefer.

STEP 1 1: CREATING ARMATURES

The Blender term for a skeleton is *armature*. An armature is a special control object that defines the mechanical structure of a model—the position of its limbs and joints. It doesn't matter if you're modeling a human, a monster, a mechanical digger, or a length of rope—an armature specifies the underlying pieces of the model in a general and abstract way. This makes armatures very important for animation.

The Pendragon character needs an armature to define how its legs, arms, and head are connected to the torso, and how the whole bone system works together when in motion. To create the armature, follow these steps:

1. Center your mesh. If your character mesh is not centered at the scene origin, then now is an ideal time to make it so. It'll make working with and exporting armatures much easier. The character's pivot, or center, is usually at the base of the feet—in the horizontal center, where they make contact with the ground. I recommend moving your meshes such that the foot is positioned at the origin. Then reset, or clear, the object transformation data by first selecting Object > Apply > Location and then selecting Object > Apply > Rotation & Scale. (See Figure 3.2.)

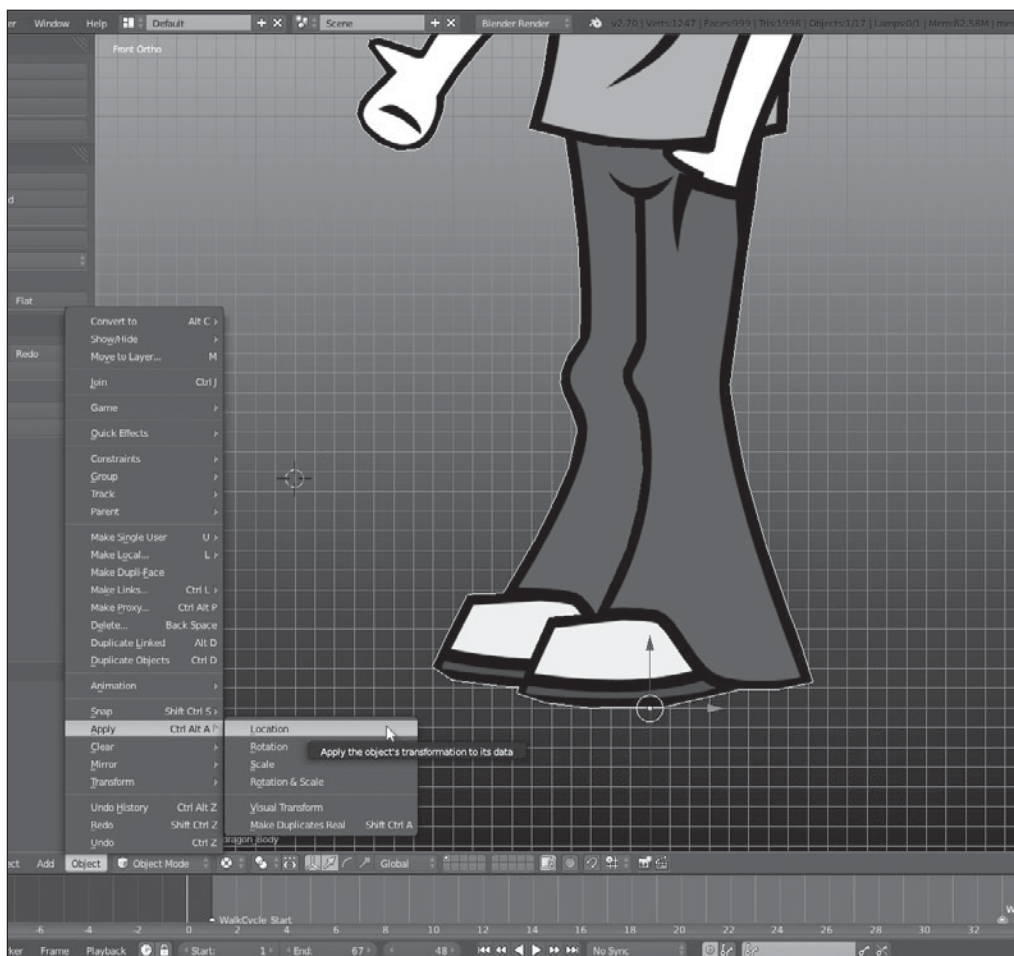


Figure 3.2
Positioning the character mesh with its feet at the world origin.

Source: The Blender Foundation.

2. Position the different parts together using scene depth to offset some of them, like the arms and legs, either closer to or farther from the camera. That is, move some parts deeper into the scene than others. Each limb must be drawn at the right depth setting when viewed in the front orthographic viewport. For example, the left arm must appear in front of the torso and the right arm behind. Don't move bones too far apart, though, as this will affect how Blender skins the mesh later! (See Figure 3.3.)

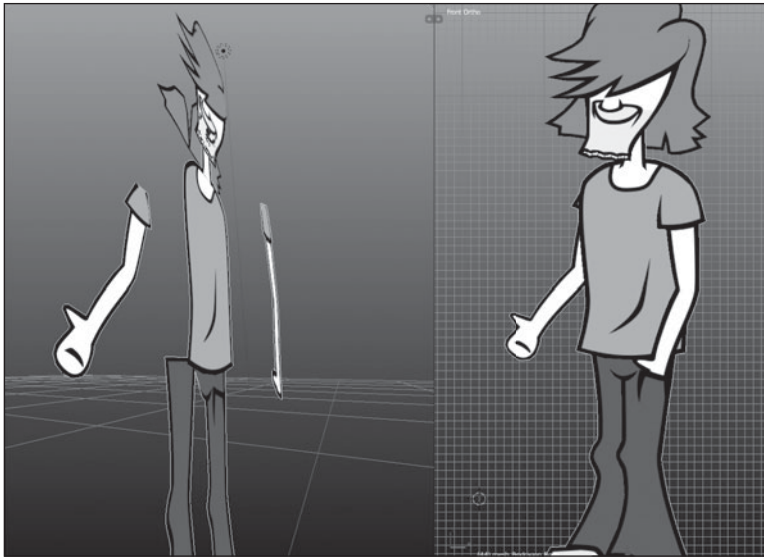


Figure 3.3
Offsetting character parts to simulate depth.
Source: The Blender Foundation.

3. To create an armature that's easy to tweak, disable object selection. That is, select the Pendragon pieces in the scene (arm, head, torso, etc.) and use the Outliner window to disable the Selection icon for each piece. That way, you won't be able to select them by accident. This will make it easier to see the Pendragon as a reference, without selecting it in the viewport. (See Figure 3.4.)

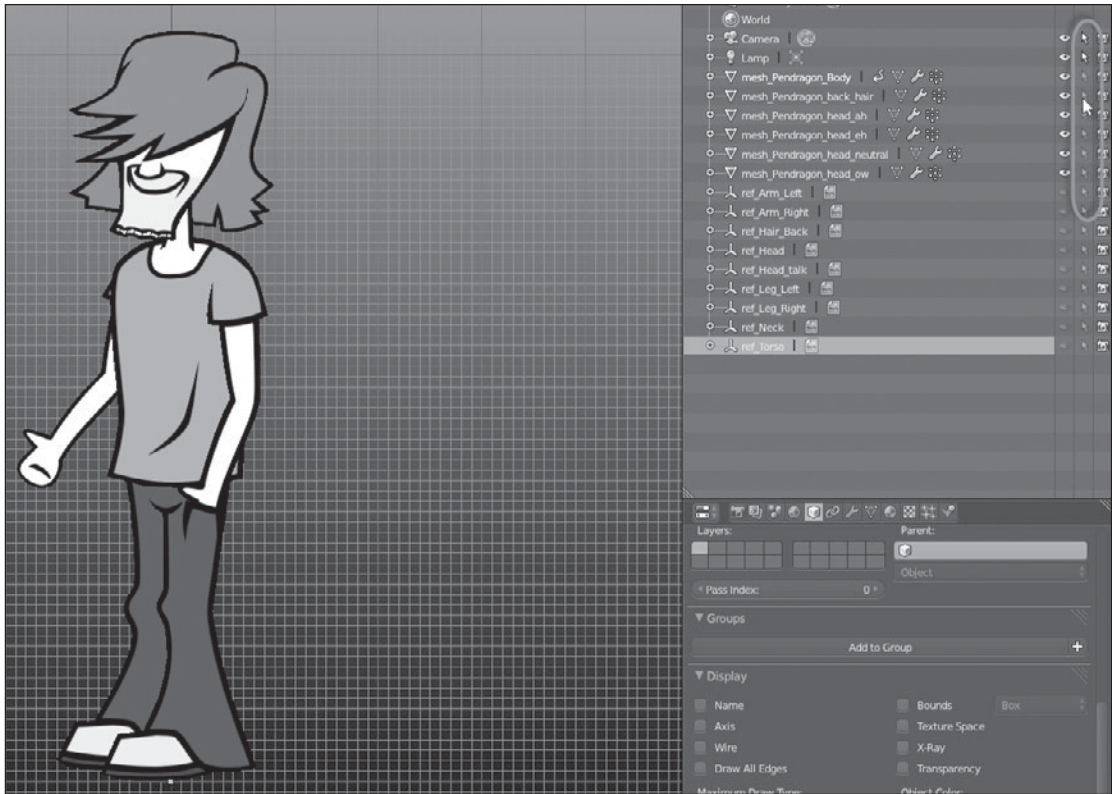
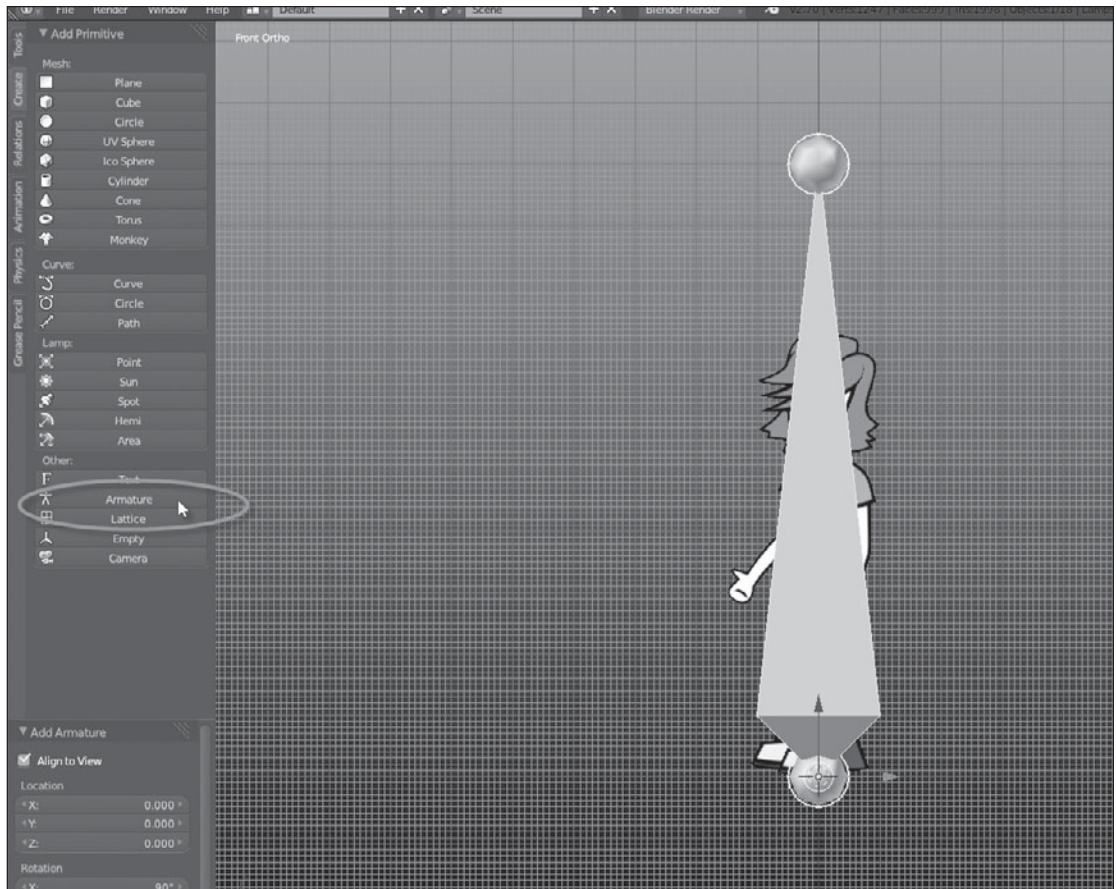


Figure 3.4

Disabling mesh selection in preparation for rigging.

Source: The Blender Foundation.

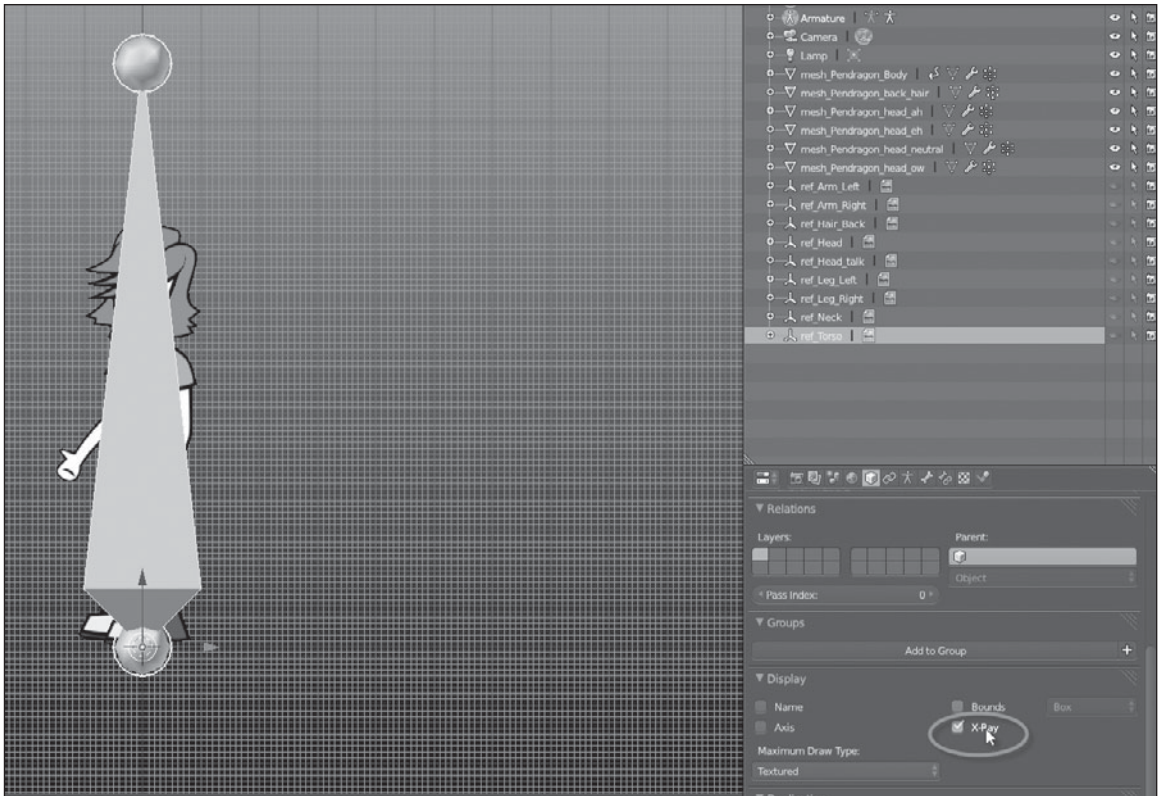
4. Switch to a front orthographic view and activate the Create panel in the toolbox.
5. Click the Armature button to create a new armature object in the scene at the position of the 3D cursor. (This should be the world origin.) This creates an aligned armature with a single, large bone, as shown in Figure 3.5. Note that the bone may initially appear larger than the mesh. That's not a problem, however. You'll correct it shortly.

**Figure 3.5**

Creating a new armature in the scene. Armatures begin with a single, large bone.

Source: The Blender Foundation.

6. Depending on the position of your mesh and armature, your mesh may appear in front of or intersecting the armature. But when creating and posing armatures, it's important to see the bones clearly, so you'll usually want them to appear in front of the mesh (even if your mesh is actually closer to the viewport camera). To achieve this, use X-Ray mode. Simply switch to the Object Data tab in the Properties panel and select the X-Ray checkbox in the Display group. (See Figure 3.6.) This forces Blender to render the armature in front of all meshes.

**Figure 3.6**

Viewing armatures in front of other objects, including meshes.

Source: The Blender Foundation.

STEP 12: CREATING BONES

In Blender, armatures can be in one of three modes:

- **Object mode:** The default, Object mode lets you to select and transform a complete armature.
- **Edit mode:** This mode lets you dig deeper into an armature, creating a connected bone layout.
- **Pose mode:** Pose mode lets you animate the armature bones.

After creating an armature, the next phase is to create a bone layout that maps onto your character mesh. Follow these steps:

1. Enter Edit mode by pressing the Tab key or by choosing Edit Mode from the Mode drop-down list on the toolbar, as shown in Figure 3.7.

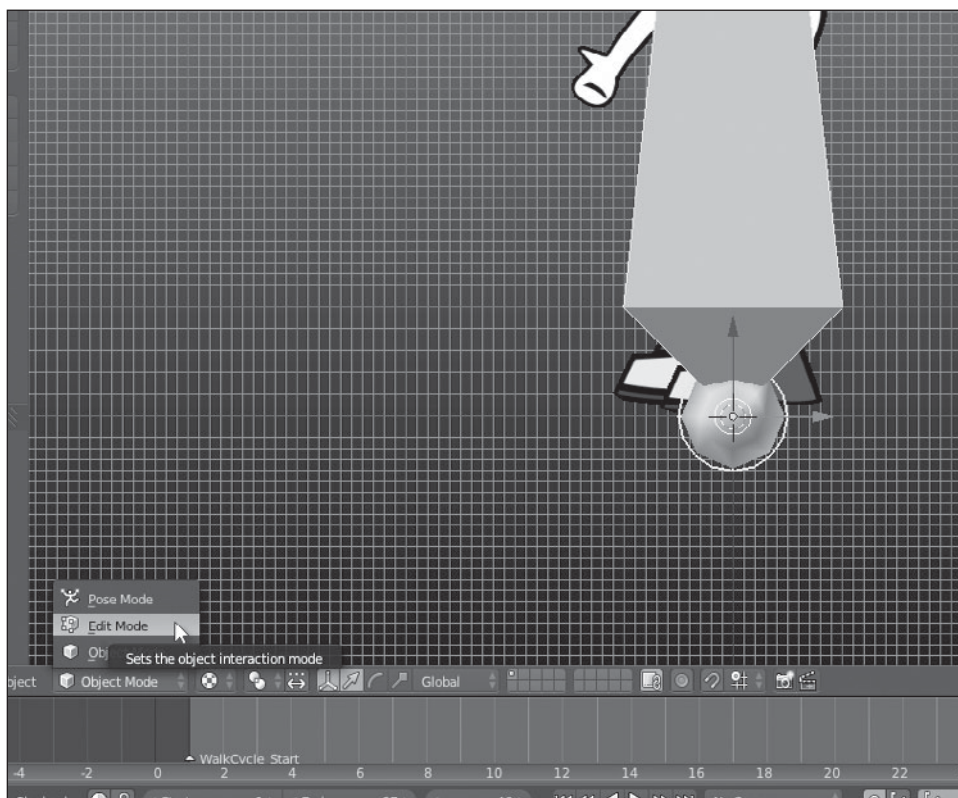


Figure 3.7

In Edit mode, an armature bone layout is defined.

Source: The Blender Foundation.

2. Having created an armature object, the first bone in the skeleton will be created automatically for you. This will be the hip bone, which will serve as the parent for the whole hierarchy of bones. Select the main part of the bone, not the spherical sections on the end (the head and tail), and use the Transform and Scale tools to position and scale the bone into place at the character's hip. (Note that selection, translation, and scaling work the same way for bones as they do for meshes.) If you need to change the angle and orientation of the bone, you can transform the head and tail sections. See Figure 3.8.

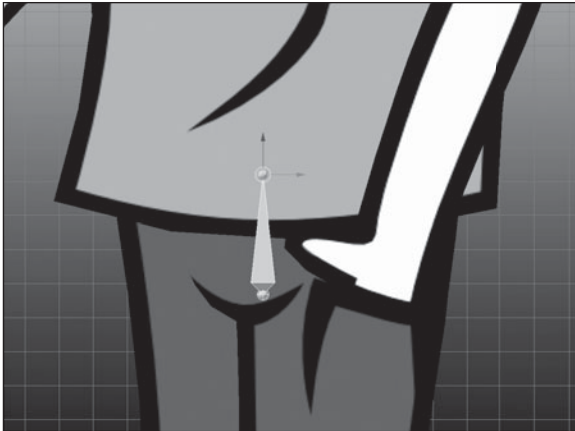


Figure 3.8

Creating the character's hip bone.

Source: The Blender Foundation.

3. It's important to give each bone a unique and meaningful name, to clearly distinguish one bone from another. This will be especially important later, when you skin and animate the mesh, because you will need to select and reference each bone by name. To name a bone, select it in Edit mode and rename it, just as you'd rename any other object. I've named the first bone "Hip." (See Figure 3.9.)

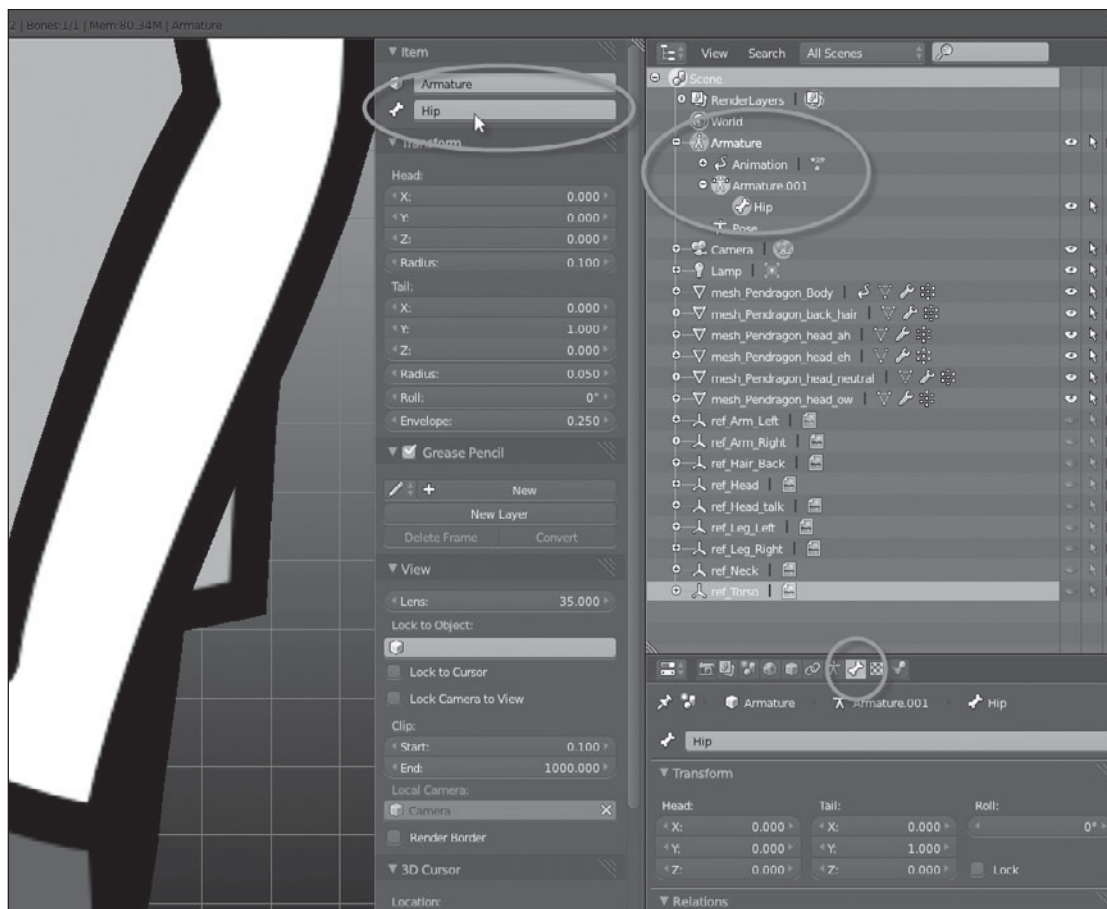
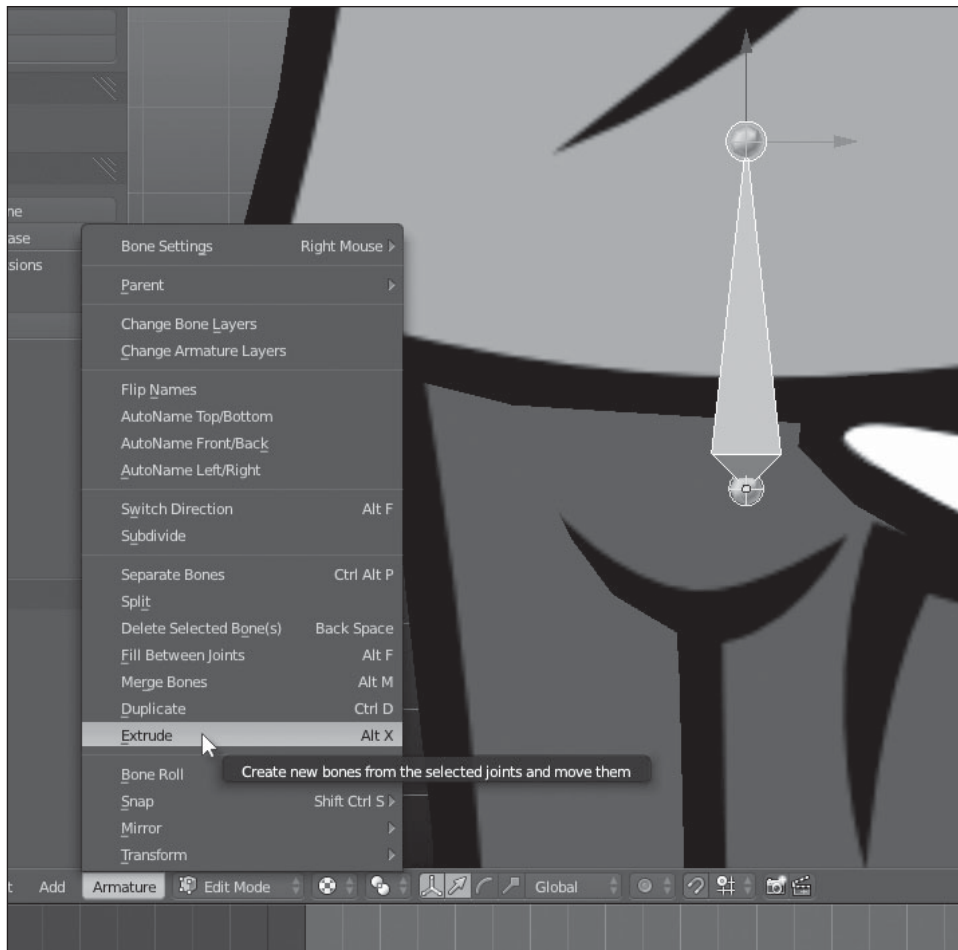


Figure 3.9
Give each bone a meaningful name.
Source: The Blender Foundation.

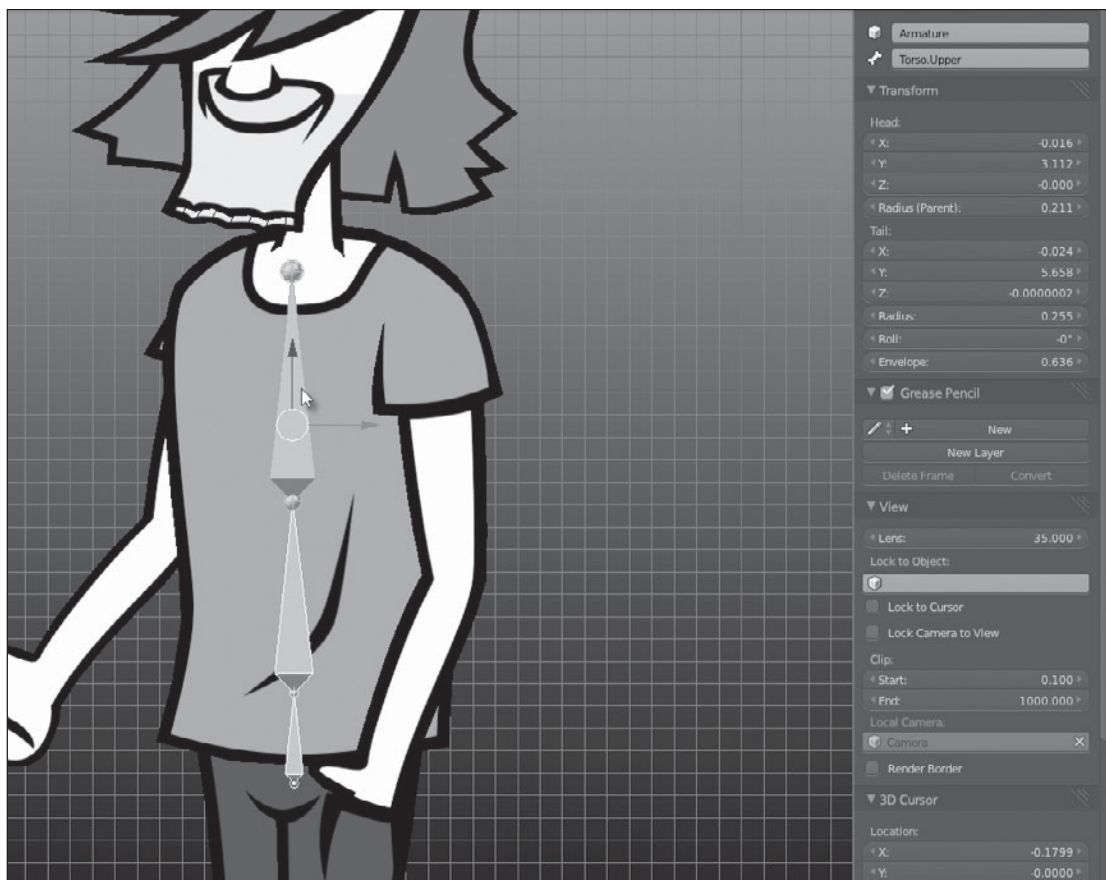
4. Use the Extrude tool to create a new bone connected to the existing one. To do so, select the head of the hip bone and choose Armature > Extrude from the viewport menu (see Figure 3.10) or press Alt+X. Use the transform axes to constrain the extrusion upward into the torso. This newly created bone represents Pendragon's lower torso.

**Figure 3.10**

Extrude a new bone using the Extrude tool.

Source: The Blender Foundation.

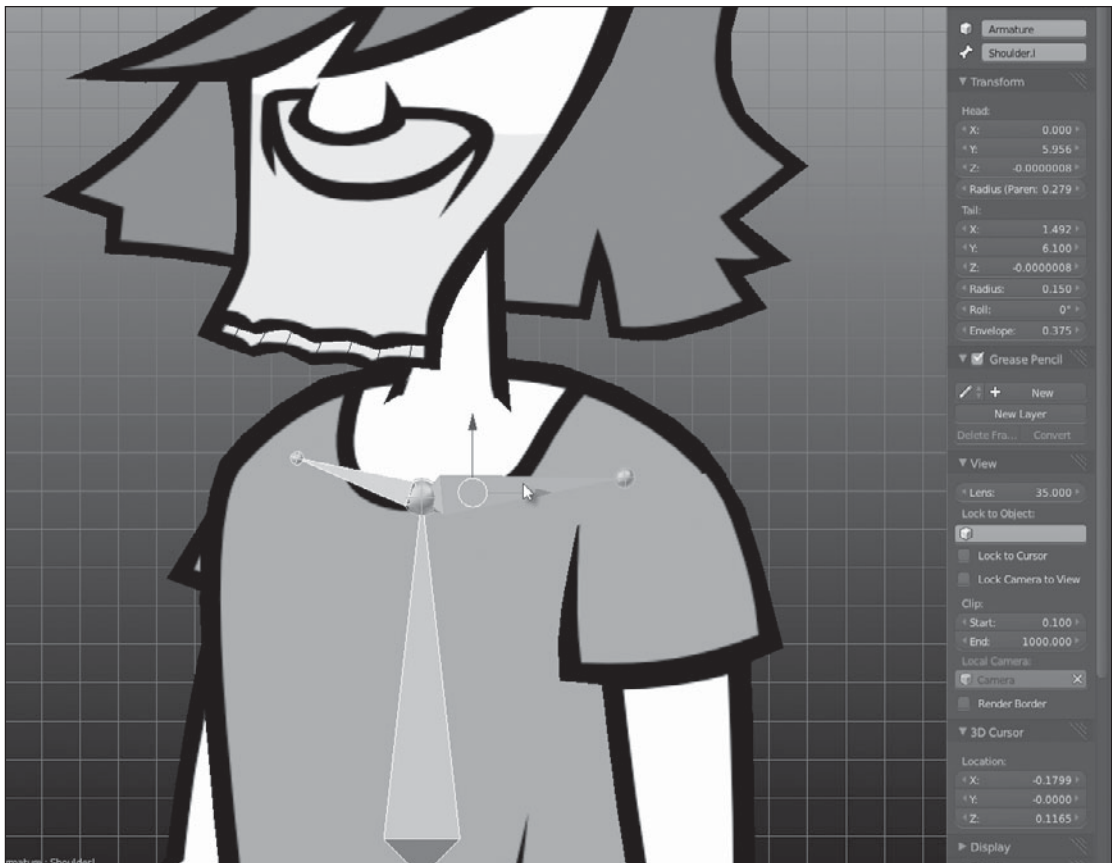
5. Name the bone that represents Pendragon's lower torso `Torso.Lower`.
6. You want to split the torso into two main, vertically aligned bones, allowing at least one bend, as shown in Figure 3.11. This lets you create animations in which Pendragon crouches over, dances, or sways. Extrude a new bone from the lower torso for the upper torso and name it `Torso.Upper`. If required, transform the bone heads and tails to adjust the angle and incline, conforming to the mesh. Remember to keep all bones in the same plane (at the same depth in the scene), however. The character is a flat, 2D mesh, after all!

**Figure 3.11**

Creating bones for the torso: lower and upper.

Source: The Blender Foundation.

7. Select the head of the upper torso bone, where it comes close to the character's neck. Then extrude a bone outward to the right to meet the top of the character's left arm, positioning it where the upper arm socket would be.
8. Select the head of the upper torso bone a second time and extrude another bone, but in the opposite direction. This bone should match up with the right arm, which appears behind the character's torso. Remember to name each bone appropriately—for example, *Shoulder.l* and *Shoulder.r*. (See Figure 3.12.)

**Figure 3.12**

Creating shoulder bones.

Source: The Blender Foundation.

9. Complete the upper body, head, and arms, as shown in Figure 3.13. As you can see, the torso has two bones, there are two shoulder bones, and there is one neck bone. The head consists of one bone, while each arm consists of two bones (upper and lower arm). In addition, I've created one additional bone for each hand. Pendragon will not move individual fingers, so you don't need to rig those. (For more realistic, non-cartoon characters, you may need to rig the hand using multiple bones, depending on your needs.) Notice that the bone joints—the heads and tails—correspond to the real joints in the model, such as the elbows and shoulder joints. This is important for producing clean and reliable deformations during animation.



Figure 3.13

Completing the upper body.

Source: The Blender Foundation.

10. Next, you'll complete the character's legs and feet. Note, however, that these require special consideration. Thus far, all the bones you've created are directly connected to another bone. A bone tail touches the head of a neighboring bone, and so on in a bone chain until it reaches an end point, such as the head or hands. The legs, however, work slightly differently: They won't be directly connected to the main body. Rather, they'll be offset. This is because the legs will work as a separate, self-contained system from the rest of the body. Doing this is not essential, but it makes animating the legs simpler, enabling you to edit them in isolation without affecting the rest of the body. Start by creating the leg left. To create this bone, click the Add button in the Bones group in the Armature Tools panel, which is accessible only when the armature is in Edit mode. (See Figure 3.14.) This creates a new, disconnected bone in the armature.

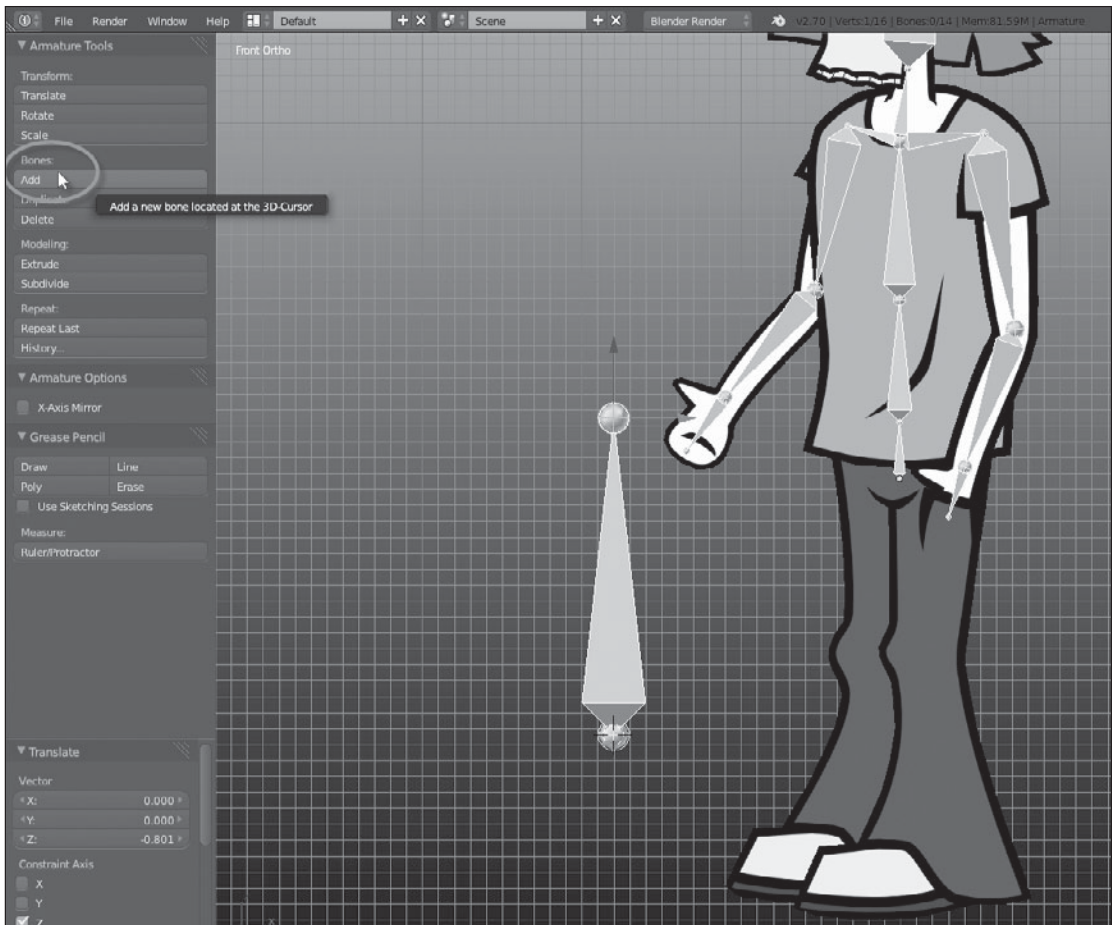
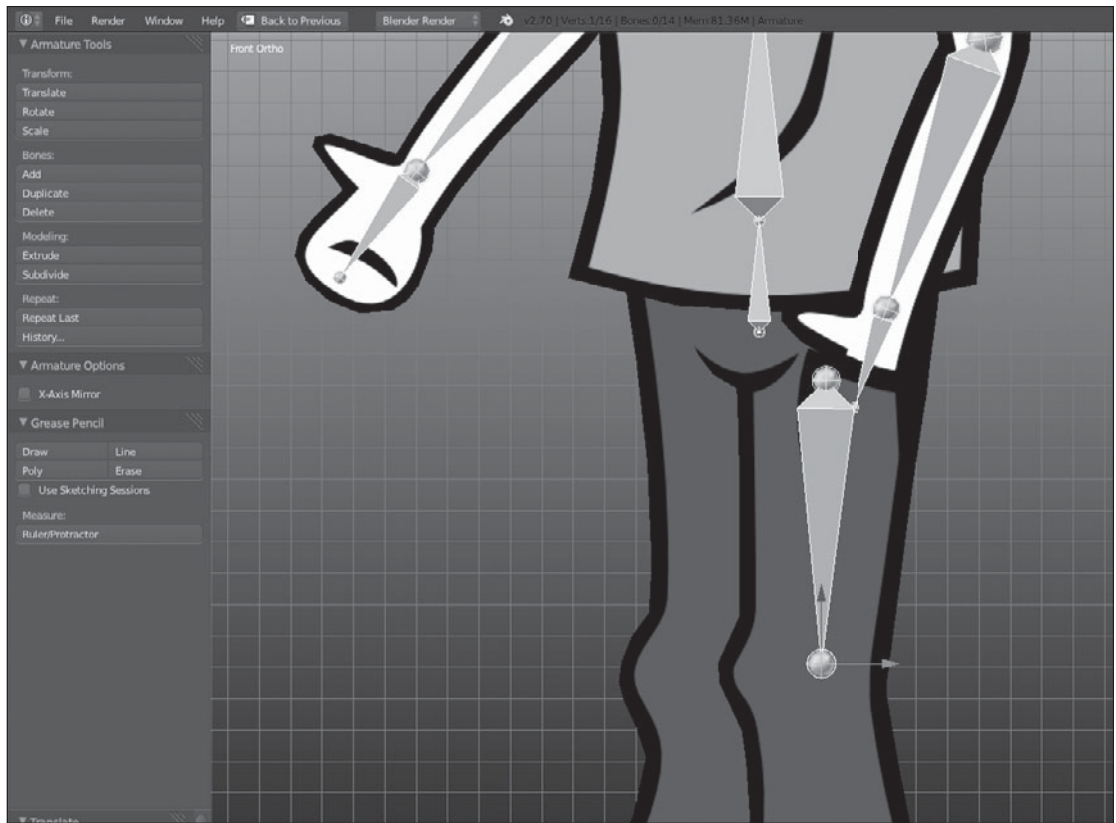


Figure 3.14

Adding a new and disconnected bone to the armature in preparation for rigging the legs.

Source: The Blender Foundation.

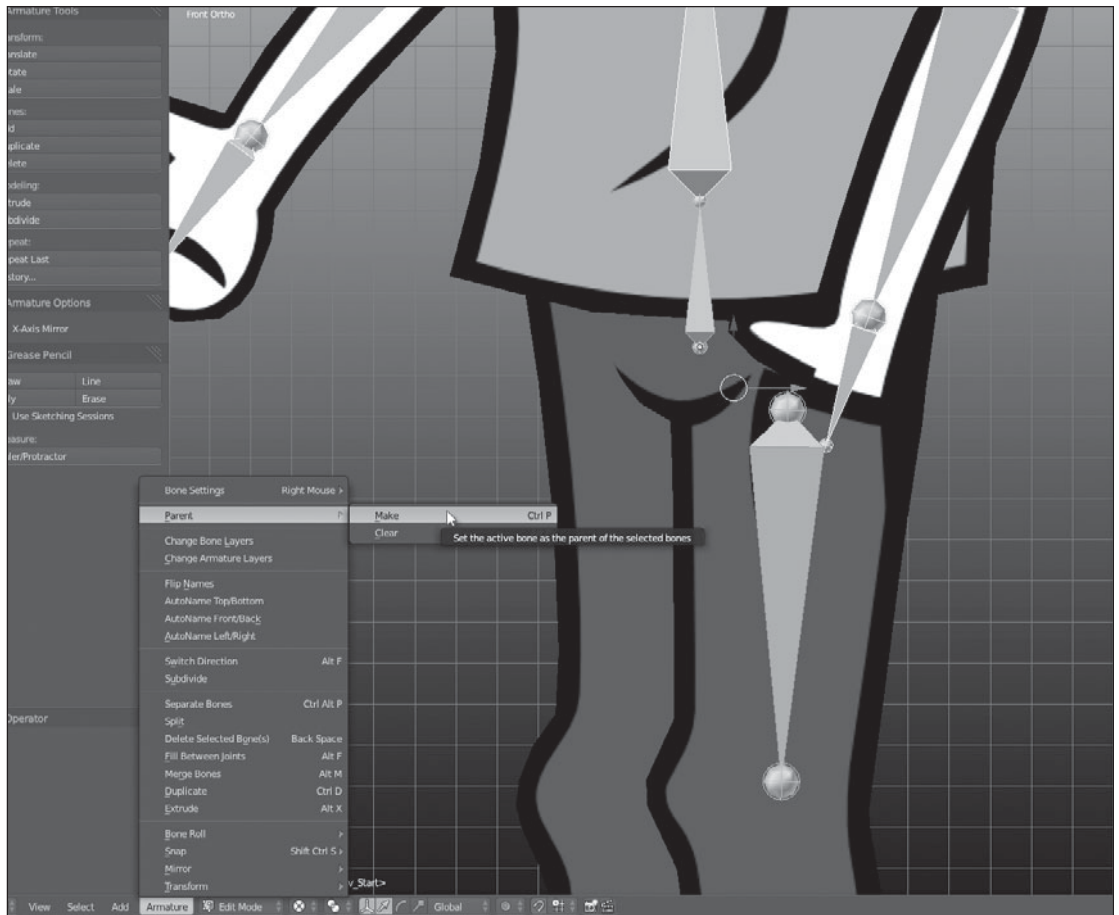
11. Position the bone head and tail along the upper leg, or the thigh (thigh.l and thigh.r). The bone tail should be where the upper leg connects to the hip and the bone head should be at the knee joint. (See Figure 3.15.)

**Figure 3.15**

Creating the thigh bone.

Source: The Blender Foundation.

12. Connect the thigh bone to the hip, making it a child bone. To do this, click the thigh bone to select it, and then Shift-click the hip bone, selecting both bones. Then press Ctrl+P or choose Armature > Parent > Make from the viewport menu, as shown in Figure 3.16. A context menu appears; choose Keep Offset. This parents the thigh to the hip while maintaining the bone's position. This parent-child relationship between the hip and thigh is useful; whenever the hip moves, the leg will follow along.

**Figure 3.16**

Connecting the thigh to the hip bone.

Source: The Blender Foundation.

13. Repeat steps 10–12 to create the right thigh.
14. From each thigh bone, extrude downward to create a lower leg bone that reaches to the ankle area (shin.l and shin.r).
15. Pendragon will have only one bone for each foot. To create this bone, extrude it out from the shin bone on each leg.

16. To ensure independent control of the feet, enabling them to touch the floor even when the leg bones rotate, disconnect each foot bone from the shin. To do so, select each foot bone and press **Alt+P > Clear Parent** or choose **Armature > Parent > Clear**, as shown in Figure 3.17.

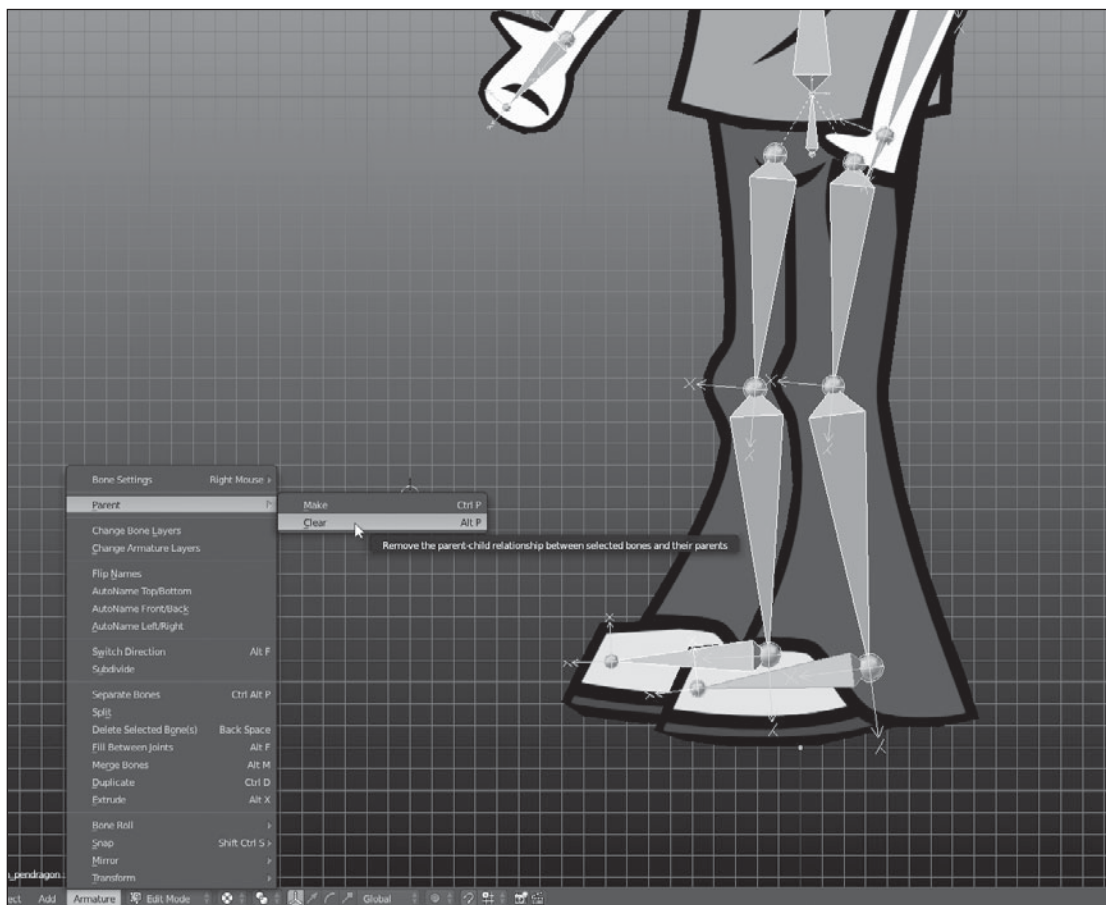


Figure 3.17
Creating disconnected feet bones.

Source: The Blender Foundation.

STEP 13: SKINNING

As shown in Figure 3.18, the Pendragon armature is almost complete, mapping out the main areas of the character mesh. Right now, however, the armature exists in isolation from the model—a separate and independent object. Yes, you can enter Pose mode and

move the bones around using the Translate and Rotate tools, but the Pendragon model itself won't conform to the bone transformations. For example, moving the leg bones won't move the leg areas of the Pendragon mesh. Now it's time to establish that connection—a process called *skinning*.

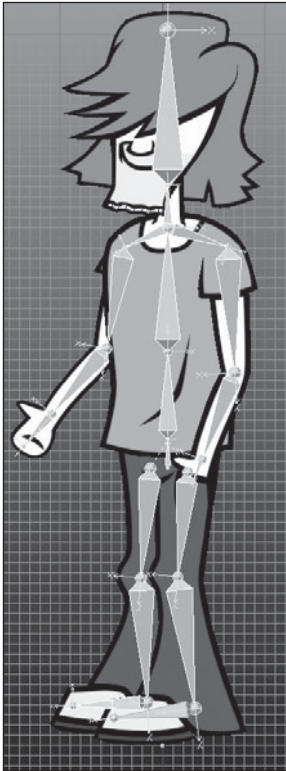


Figure 3.18
Completing the Pendragon skeleton.

Source: The Blender Foundation.

1. Combine all the Pendragon mesh elements into one object. To do so, select all the mesh parts—the head, arms, legs, etc.—and click the Join button (available from the toolbox when the mesh is in Object mode). Don't worry, you can separate the parts again later, if required. See Figure 3.19.

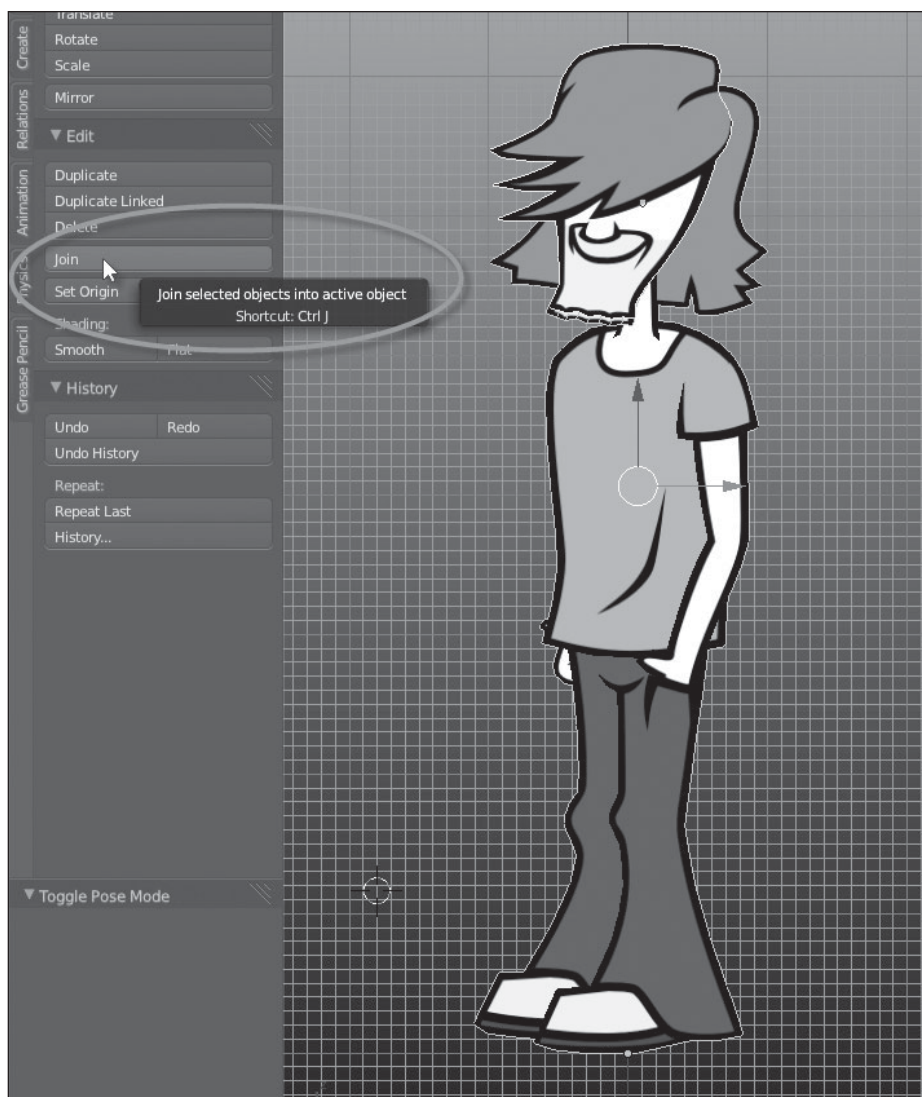


Figure 3.19

Joining the Pendragon parts together in preparation for skinning.

Source: The Blender Foundation.

2. Position the Pendragon mesh in the 3D viewport so it overlaps and intersects with the bones created in the armature (if they don't already). That is, the leg bones should intersect the leg region of the Pendragon mesh, and so on. This will be important later, when Blender calculates how the bones in the armature should affect

the character mesh. Note that the positioning doesn't have to be exact. A general, by-eye judgment is usually good enough. (See Figure 3.20.)

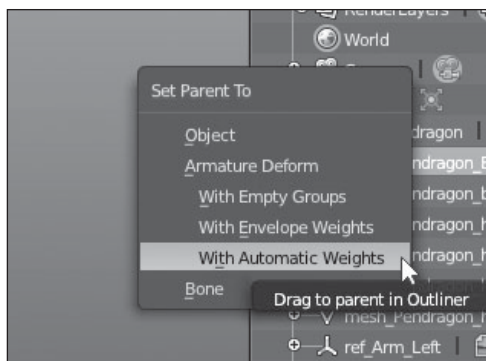


Figure 3.20

Intersecting bones with the Pendragon mesh.

Source: The Blender Foundation.

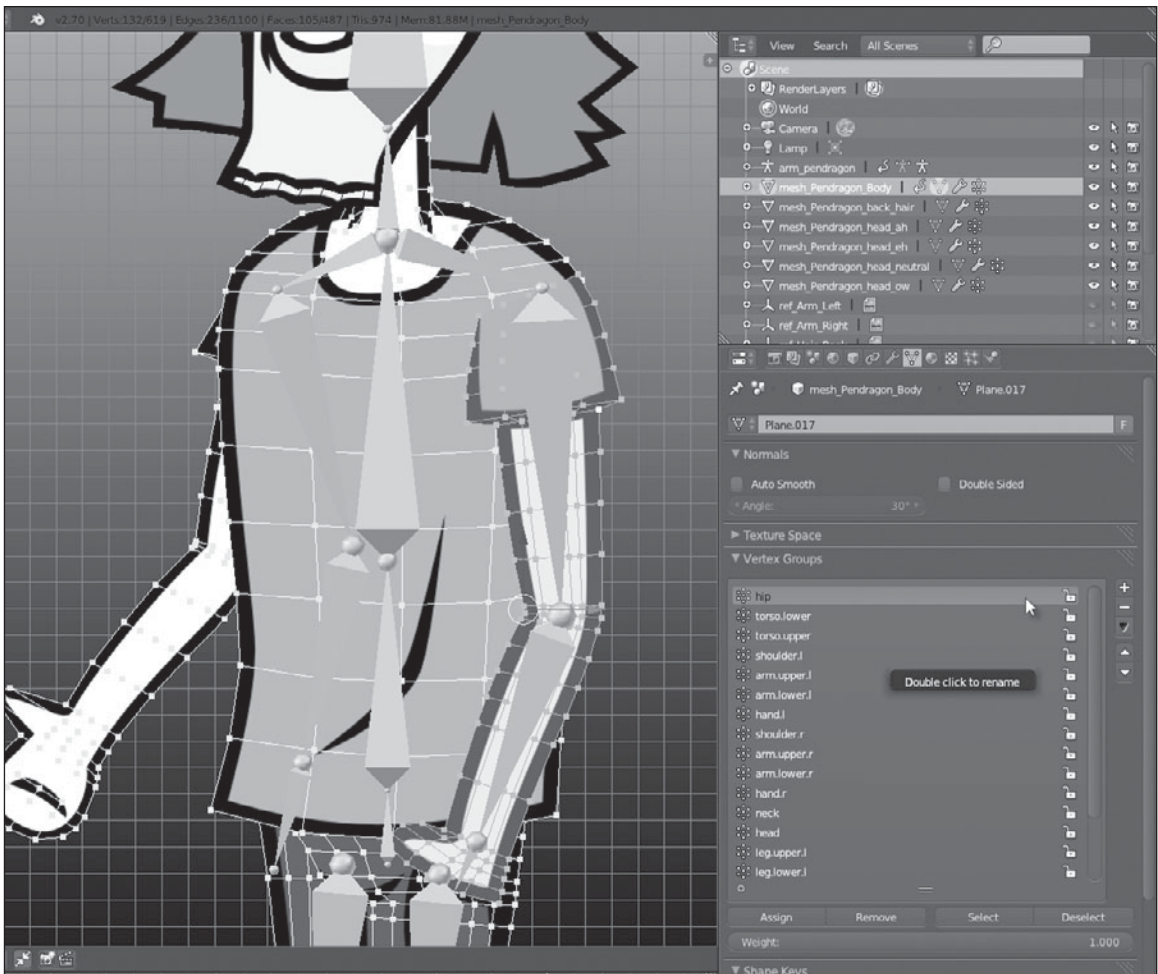
3. In the Outliner, drag and drop the Pendragon mesh object on top of the armature to parent it to the armature. This makes the mesh a child of the armature. When you parent the mesh to the armature, Blender establishes a connection between the two objects.
4. A context menu appears. Choose **Armature Deform with Automatic Weights**, as shown in Figure 3.21. Blender automatically generates skinning information for the mesh based on its best guess as to how you expect the bones in the armature to relate to the intersecting mesh. To test the effect, move the armature bones in Pose mode. The Pendragon mesh will deform in response.

**Figure 3.21**

Generating automatic weights for the Pendragon character.

Source: The Blender Foundation.

The aim of skinning is, ultimately, to get the balance of weighting correct across all vertices in the mesh so that each vertex is affected exactly as intended by each bone in the armature. Typically, the automatic weighting will be correct, but there may be times when it generates errors that must be corrected. To correct these errors, you must first understand how Blender encodes the skinning information. Blender encodes this information into *vertex groups*, which are baked into the mesh object itself. Each bone in an armature has one vertex group, which is named after the bone. A bone's vertex group contains the complete set of vertices in the mesh to be affected by that bone. Each vertex in the set has a weight value between 0 and 1, which determines how strongly that vertex will be affected by the bone. A value of 0 indicates there will be no effect, a value of 1 indicates a full effect, a value of 0.5 indicates something in the middle, and so on. To view all the vertex groups for a mesh, see the Vertex Groups section of the Object Data tab in the Properties panel, shown in Figure 3.22.

**Figure 3.22**

Viewing skinning information for a mesh.

Source: The Blender Foundation.

To display the weighting for any specific vertices as they relate to bones in the armature, select them in the mesh. To select whole groups of vertices for each limb in the mesh (such as all vertices in the arm, leg, or head), select one vertex in the limb and then press Ctrl+L or choose **Select > Linked** from the viewport menu (see Figure 3.23). This selects all connected vertices in the chosen limb. You can then inspect the weighting for the vertices from the **Vertex Groups** section of the **Object Data** tab. If you need to adjust the weighting, enter a new value for the **Weight** field and click the **Assign** button. This assigns the value to the selected vertex group. For example, if you were to assign a value of 0 to

the selected vertices in the Left_Arm group, then those vertices would no longer be affected by the left arm bone in the armature.

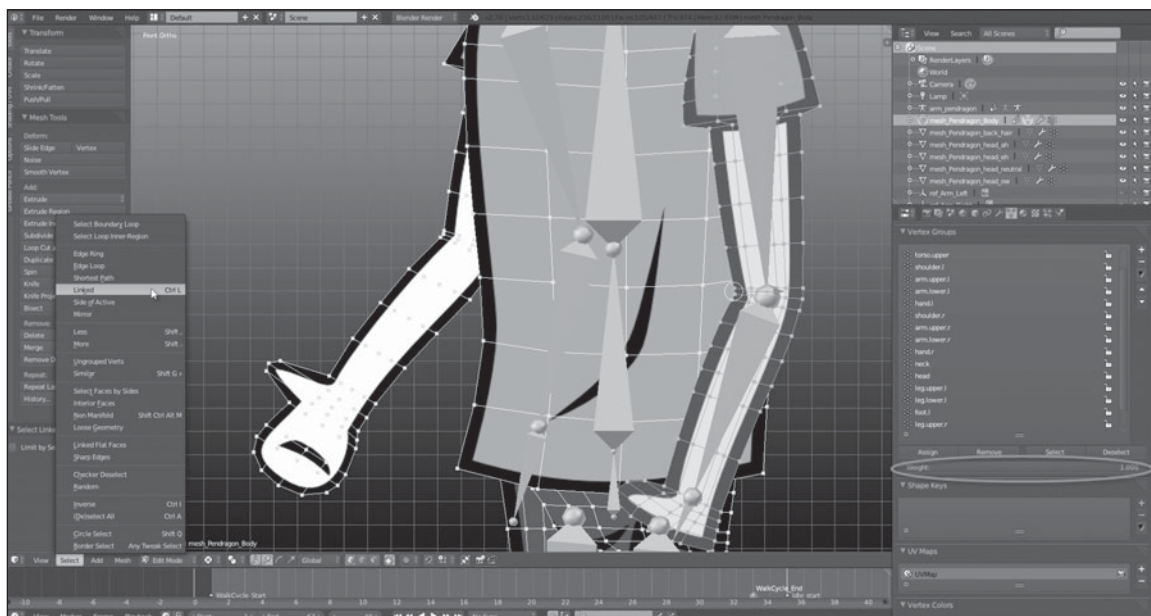


Figure 3.23
Inspecting weight data for the arm.

Source: The Blender Foundation.

Although Blender's automatic weighting is usually very accurate and requires only a few tweaks, 2D meshes like the Pendragon mesh probably require more tweaking. Before proceeding, however, I prefer to decouple the mesh from the armature, returning it to an independent object. Then I use a modifier to connect the armature to the mesh rather than using a parent-child relationship. That way, I can easily switch the armature-mesh connection on and off from the Properties panel. (I only use the drag-and-drop parenting method to generate the initial automatic weight data.) Follow these steps:

1. In the Outliner, drag and drop the mesh back to its original location. Alternatively, select **Object > Parent > Clear Parent** from the viewport menu. This removes the connection between the armature and the mesh, but the vertex group data and skinning information remain intact.

2. Select the Pendragon mesh in the viewport, open the Modifiers tab, click the Add Modifier drop-down list, and choose Armature from the Deform group. (See Figure 3.24.) Then select the armature in the scene using the Object field in the Properties panel. This re-establishes the link between the mesh and armature.

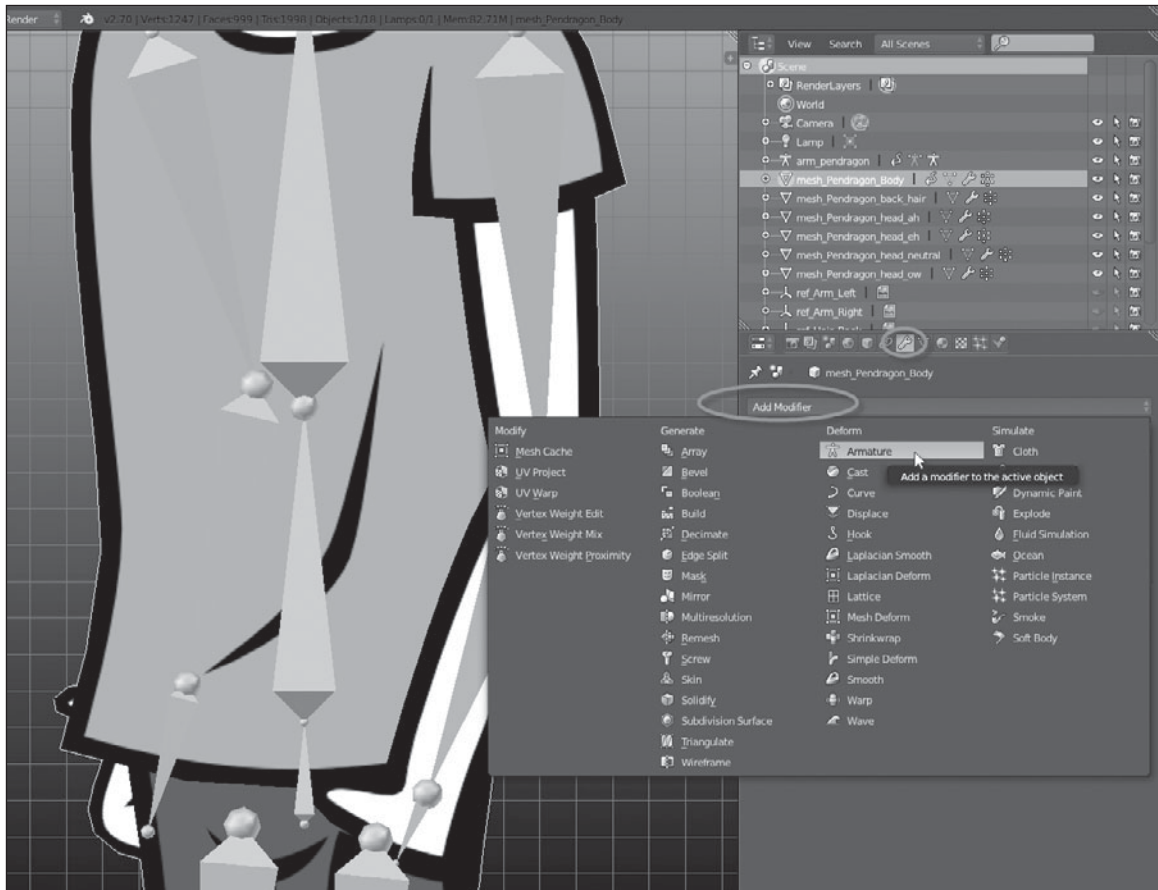


Figure 3.24

Connecting the mesh to an armature with the Armature modifier.

Source: The Blender Foundation.

STEP 14: ADJUSTING THE WEIGHTING

The automatic weighting applied to the Pendragon mesh will be inaccurate because Blender doesn't grasp that the model should be 2D. To see the extent of the inaccuracy, pose the armature in Pose mode and ascertain just how badly the bones deform the character mesh. As shown in Figure 3.25, bones in the arm affect areas of the torso, and bones

in the torso affect areas of the arms. Blender simply doesn't recognize the depth and separateness of these limbs. You must adjust the bone weighting to account for the 2D nature of the mesh.

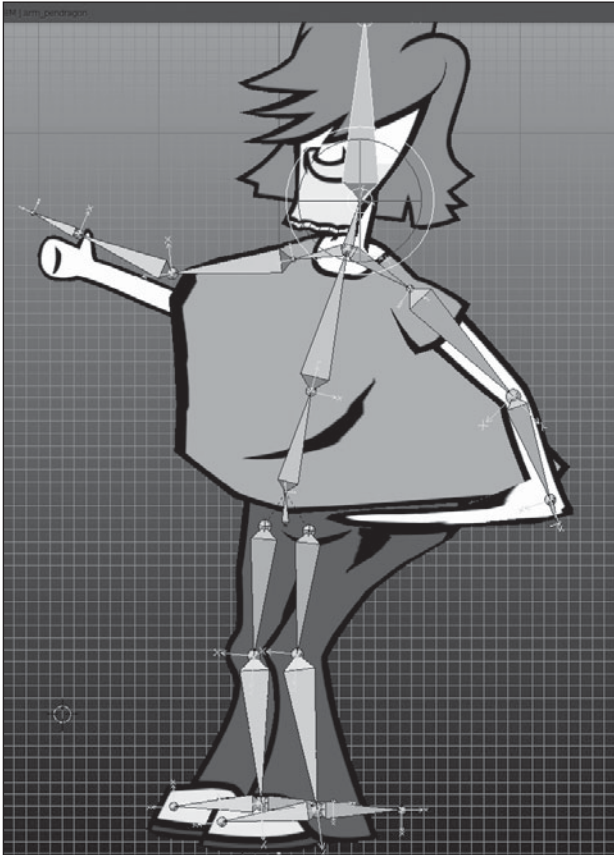


Figure 3.25

Automatic weighting is a start, but needs refinement.

Source: The Blender Foundation.

As discussed in the previous section, you can adjust bone weight by editing the Weight value for vertices in the Vertex Group section of the Object Data tab. Another method is to use Weight Paint mode, which enables you to use your mouse or tablet to interactively “paint” weights onto vertices inside the viewport.

To fix the Pendragon mesh, start by addressing problems in the legs. If you try moving bones in the left leg, you will see that vertices in the right leg are also deformed. This is because the left leg bones are close enough to the right leg vertices for Blender to think

there's association between them. There is no such association, however. The left leg bones should move separately from the right leg vertices. To fix this, follow these steps:

1. Select a vertex in the right leg.
2. Choose Select > Linked to select all other vertices in the leg. This will select all vertices in the right leg.
3. In the Vertex Group section of the Object Data tab, select the Left Leg Upper group.
4. Type 0 in the Weight field and click the Assign button. This assigns a weight value of 0 to the selected vertices of the selected bone group. As a result, the right leg vertices will now be unaffected by the left leg bones. (See Figure 3.26.)

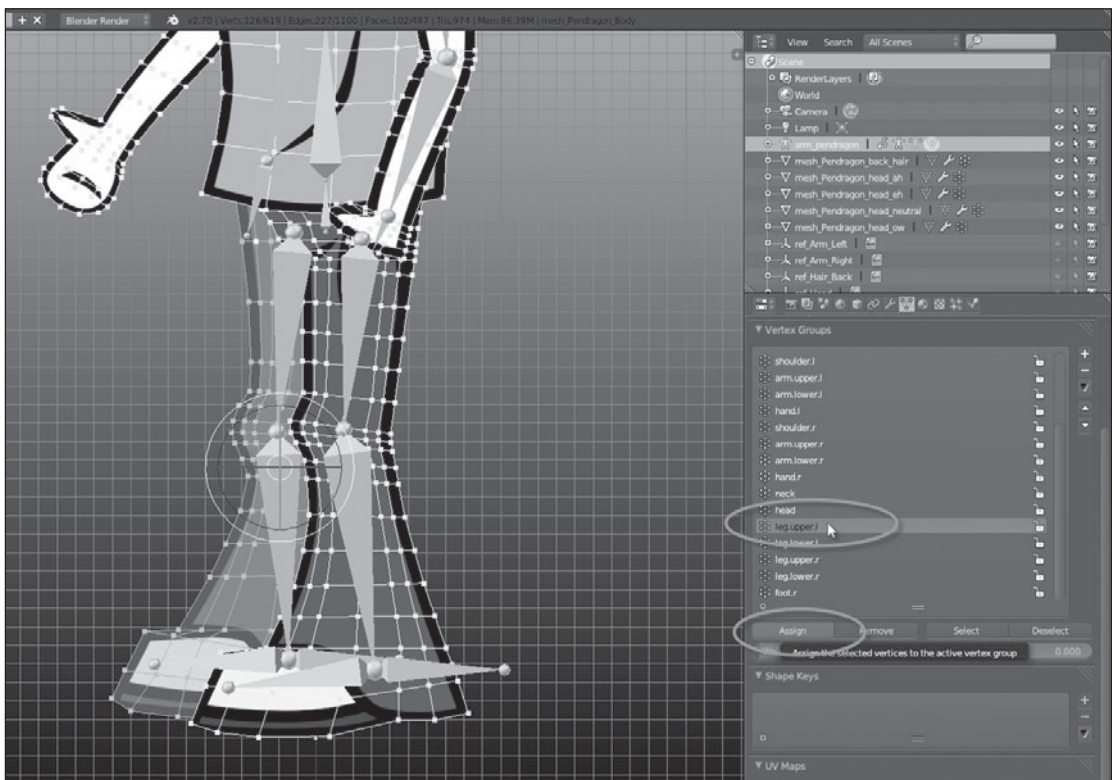


Figure 3.26

Correcting the leg weighting.

Source: The Blender Foundation.

5. Repeat steps 1–4 for the lower leg.
6. Repeat steps 1–5 for the other leg, breaking the connection between the left leg vertices and the right leg bones.

By correcting the vertex weighting using the Vertex Group settings, you can completely separate the leg weighting, enabling specific bones to affect only the appropriate portions of the mesh. That is, the left leg bones will affect only the left leg vertices, and the right leg bones will affect only the right leg vertices. (See Figure 3.27.) You can repeat this process elsewhere in the mesh, wherever the influence of a bone must be completely removed from a selection of vertices.

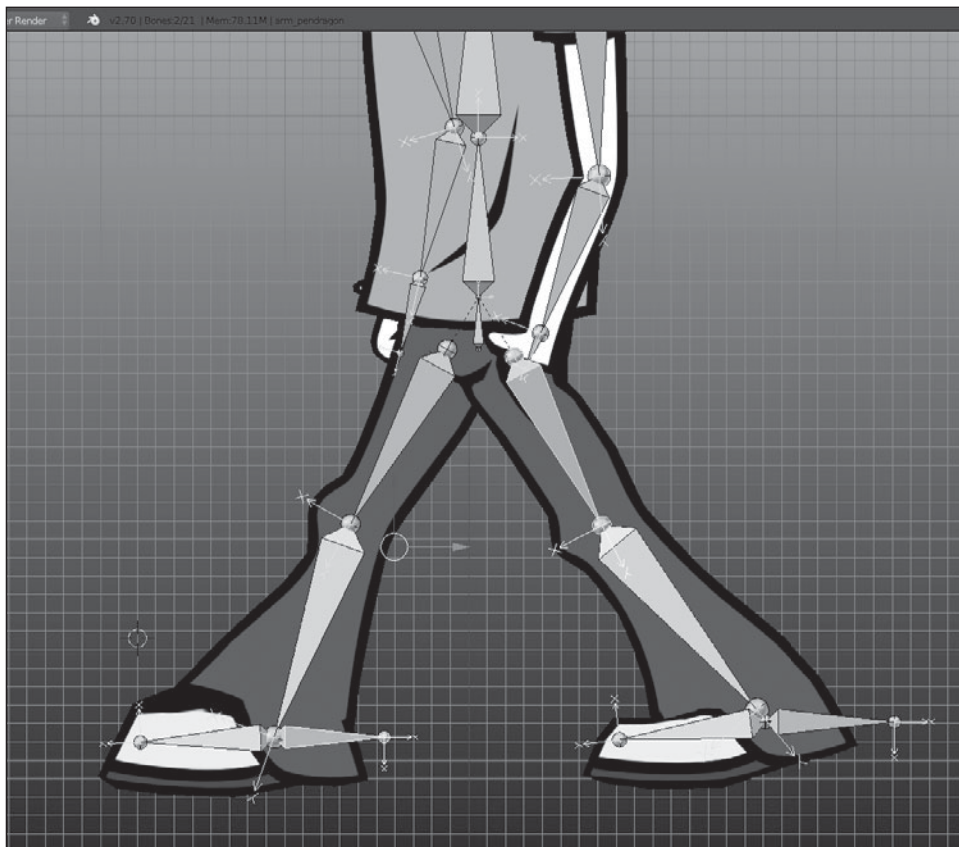


Figure 3.27
Legs moving independently.
Source: The Blender Foundation.

The disadvantage of using the Vertex Group settings to assign bone weights is that it's difficult to visualize and assign weights between 0 and 1. Sometimes—for example, where the torso meets the arms, or where the pelvis meets the legs—a vertex requires some weight,

but its influence should be moderated. In these cases, it's easier to use Weight Paint mode to define bone strength. To experiment with this mode, follow these steps:

1. In Pose mode, select the armature's upper arm bone. (I'll start with the left arm, though other bones may also need tweaking in this way.) You'll now control its weighting with the torso region. (See Figure 3.28.)

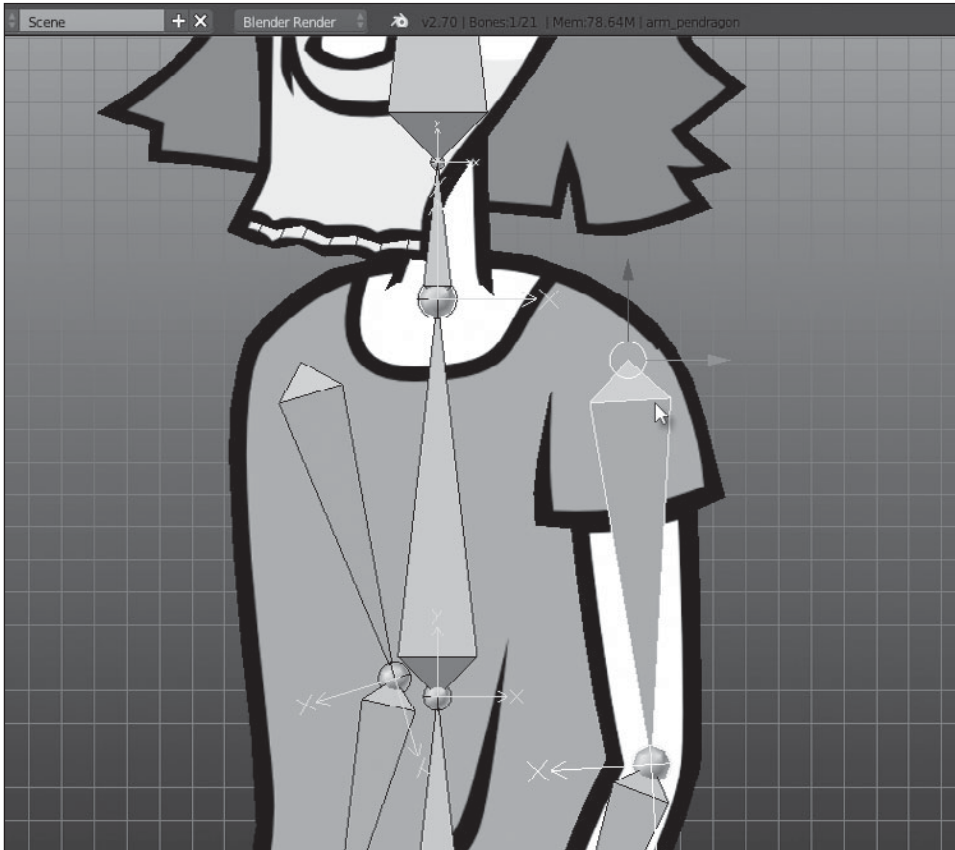
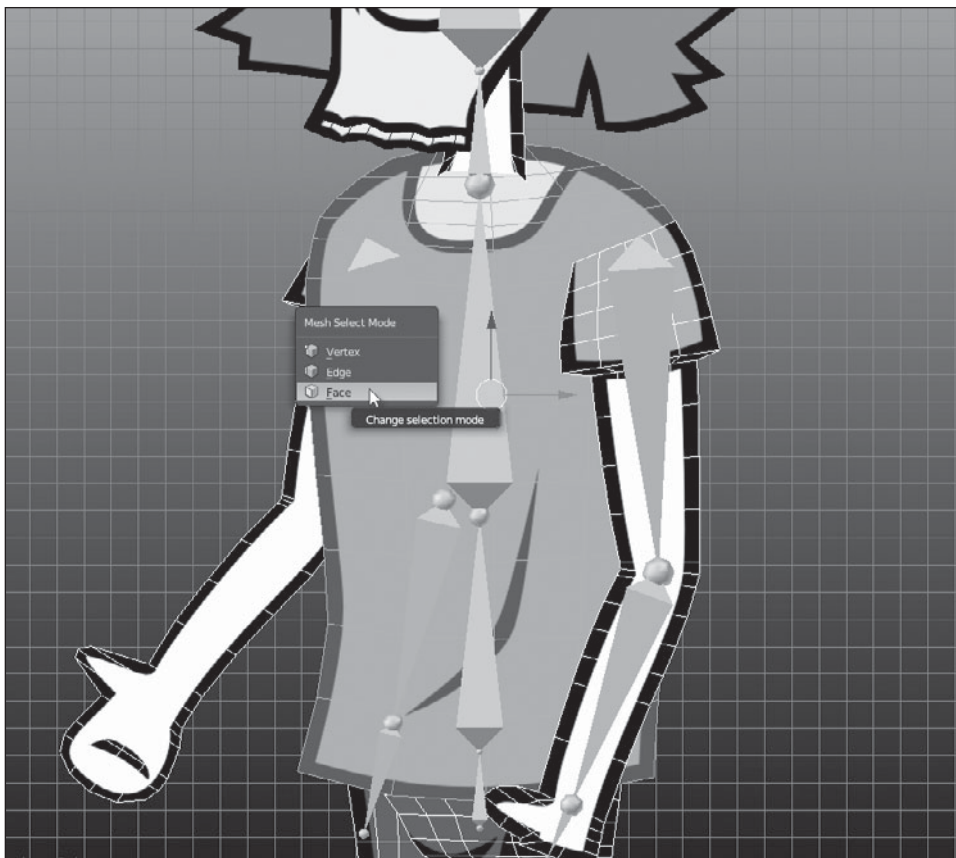


Figure 3.28

Select a bone in the armature.

Source: The Blender Foundation.

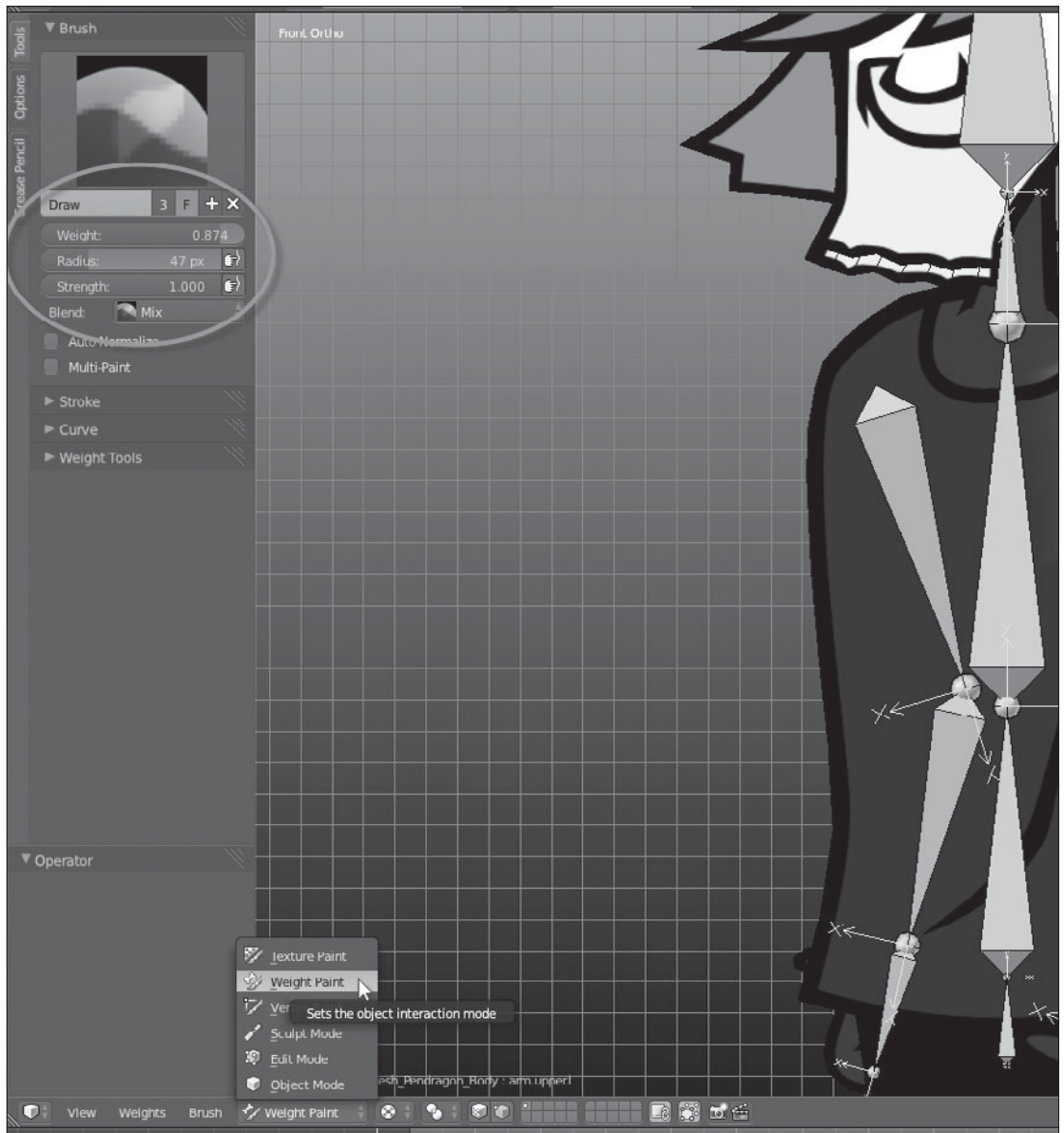
2. Select the part of the mesh whose weights must be edited for the selected bone—in this case, the torso area.
3. In Edit mode, select all the faces in the torso. To do so, pick one vertex in the torso and press Ctrl+L or choose Select > Linked from the viewport menu. Then switch to Face mode to select all the torso faces. (See Figure 3.29.)

**Figure 3.29**

Select all faces whose weighting must be edited.

Source: The Blender Foundation.

4. Open the Mode drop-down list at the bottom of the viewport and choose Weight Paint. In Weight Paint mode, the shading of the mesh in the viewport changes to a color scheme of blue, green, orange, and red. Mesh areas marked in blue are completely uninfluenced by the selected bone in the armature (Weight: 0). In contrast, mesh areas marked in red are fully affected by the selected bone (Weight: 1). (You can think of the blue areas as being “cold,” the red areas as being “hot,” and the green and orange areas as being somewhere in between.)
5. When Weight Paint mode is active, brush tools become available in the toolbox on the left. (See Figure 3.30.) These enable you to “paint” using the mouse or a tablet. The Radius setting defines the size of the brush, and the Weight value determines the weight to be painted onto the vertices when that brush is applied. To paint, simply click the desired area in the mesh.

**Figure 3.30**

Weight Paint mode.

Source: The Blender Foundation.

6. By default, painting affects all vertices in the mesh, not just the selected ones. This can be frustrating if you want to restrict the effect to specific faces, such as those in the torso or arm. Fortunately, you can use Blender's Face Selection masking feature to limit weight painting to the selected faces. To toggle this option on, click the Face

Selection Masking button in the viewport toolbar, as shown in Figure 3.31. With this enabled, weight painting will apply only to the selected torso faces.

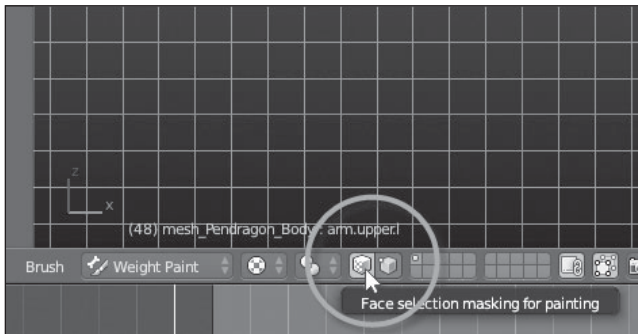


Figure 3.31

Restricting weight painting to the selected faces.

Source: The Blender Foundation.

7. Apply the brush to the torso, varying and blending the weights so that the arm lightly influences the torso. As the arm rotates and twists, the torso should deform to some extent, but the arm's influence should not pull the torso too strongly from its default state. Figure 3.32 shows the weight painting I've applied to define the left arm strength.

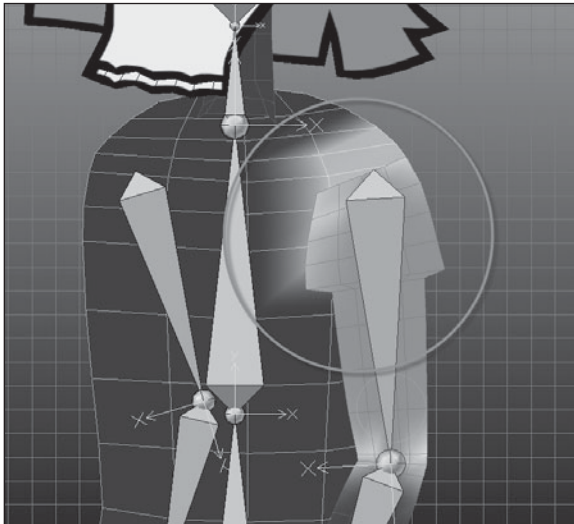


Figure 3.32

Tweaking the weight painting.

Source: The Blender Foundation.

8. Continue weight painting and manually setting the weights of vertex groups until all bones in the armature affect the Pendragon mesh as required. Each limb should move believably with the rest of the body, without pulling and deforming unintended regions. This process can be long and laborious, especially for more intricate meshes with props, costumes, and unconventional anatomy. But it's necessary and worth it!

STEP 15: APPLYING INVERSE KINEMATICS

The final rigging issue to consider is Inverse Kinematics (IK). In the case of Pendragon, IK relates specifically to the legs, but it can also apply to arms or any other long chain of bones.

As things stand right now, to animate the character's legs for a walking motion, you'd have to start by rotating the thigh bone, then the shin bone, and then the foot bone, working from the top to the bottom of the bone chain until the foot lands exactly where you need it to for every frame in the animation. Although this approach would work, and over time could create an entirely believable animation, it's tedious to transform so many different bones. It'd be better if you could simply grab the feet bones, position them wherever you need to, and then have the armature automatically calculate how the intervening shin and thigh bones should rotate and bend (while respecting the hip and feet bone positions). This, in essence, is what IK allows you to do. In this section, you'll use IK to complete the rig for the feet.

1. Select each foot bone, then press Alt+X to extrude one bone out from the back of each foot bone. Then press Alt+P to disconnect this new bone from the foot, leaving it in place as an isolated bone. (See Figure 3.33.) These bones will be used internally by Blender's IK system to mark the end points of a bone chain representing the leg. Name the bones leg.ik.l and leg.ik.r.

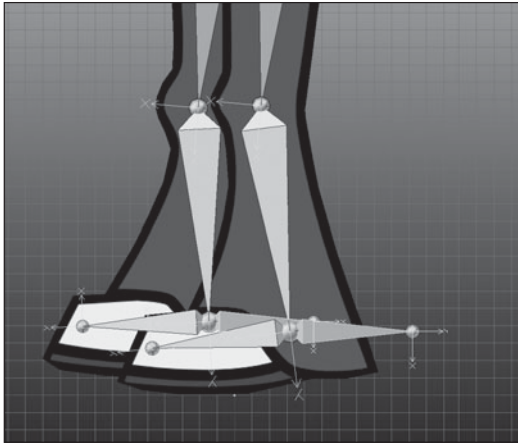


Figure 3.33
Creating IK handle bones marking the end of a bone chain.
Source: The Blender Foundation.

2. In Pose mode, select the shin bone for each leg (the lowest bone in the leg chain). Then, in the Bone Constraints tab in the Properties panel, open the Add Bone Constraint drop-down list and choose Inverse Kinematics, as shown in Figure 3.34.

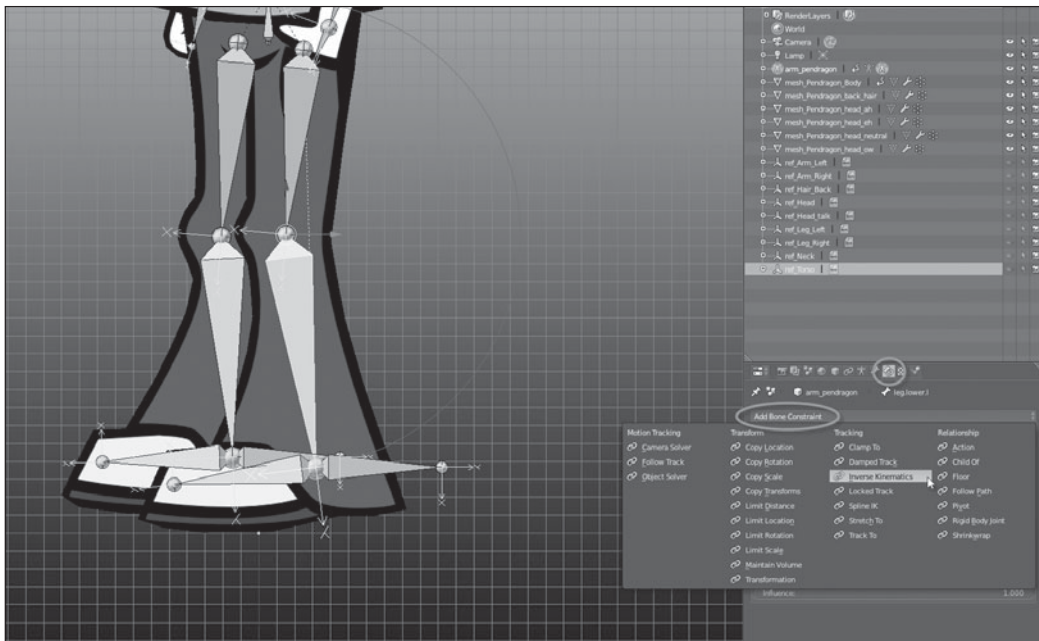


Figure 3.34
Adding an Inverse Kinematics constraint to the shin bone.
Source: The Blender Foundation.

3. In the Inverse Kinematics settings, open the Target list and select the armature, open the Bone list and select the appropriate IK bone for each leg (either leg.ik.l or leg.ik.r), and specify a Chain Length of 2, as shown in Figure 3.35. (This is because there are a total of two bones in the bone chain: the shin bone and the thigh bone.)

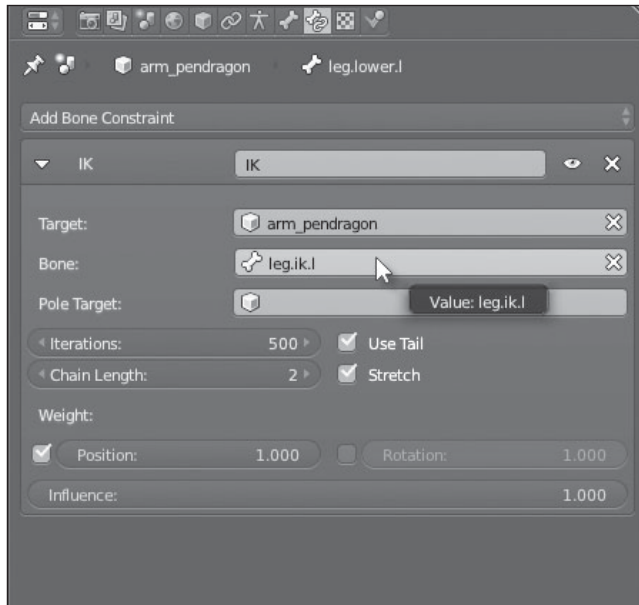


Figure 3.35
Configuring the IK constraint.

Source: The Blender Foundation.

Note

In Figure 3.35, the IK target is listed as "arm_pendragon." Remember, the prefix "arm" is an abbreviation for "armature" (skeleton), and does not refer to one of the character's arms.

4. It's a good practice to impose specified limits on bone rotation for IK. For example, if a leg bends at the knee, there are two ways it can rotate: inward and outward. But only the outward rotation is acceptable, since legs cannot bend in both directions. To prevent Blender from getting confused, you can apply rotation constraints to the shin bone. To do so, select the shin bone, switch to the Bone Data tab in the Properties panel, and scroll down to the Inverse Kinematics group. There, lock rotation entirely on the X and Y axes and limit rotation on the Z axis between 0 and 180 degrees. As you do this, a rotation arc appears in the viewport, offering a preview of the acceptable limits for rotation. (See Figure 3.36.)

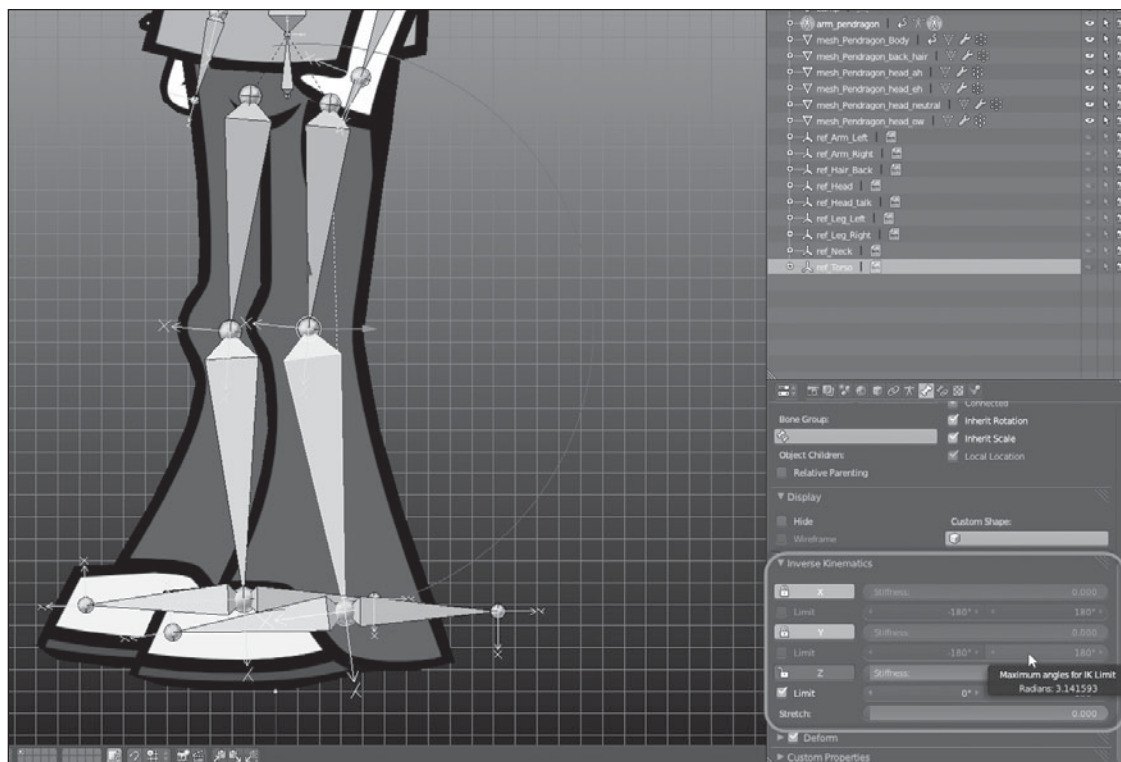


Figure 3.36
Constraining leg rotation.

Source: The Blender Foundation.

5. IK is now configured for Pendragon. To see it in action, grab either one of the IK bones, move it to position the foot, and see the leg bend to accommodate the new position. (See Figure 3.37.)

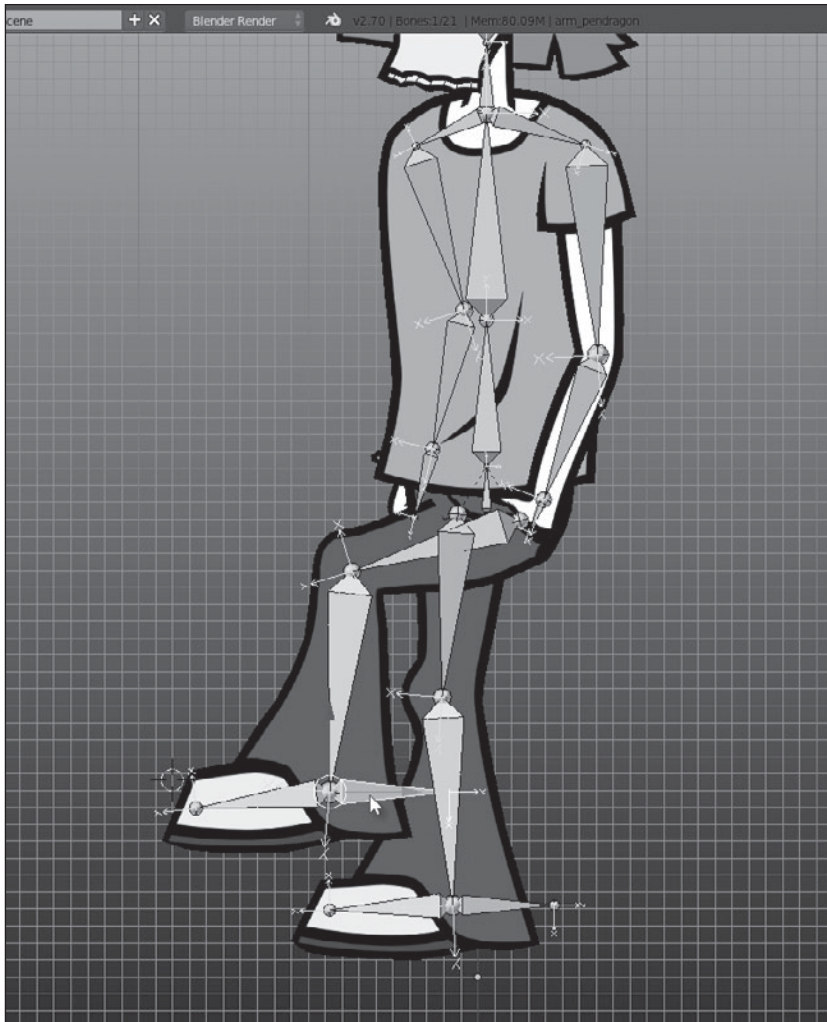


Figure 3.37

Positioning the feet with IK.

Source: The Blender Foundation.

6. IK even works if you position the hip/pelvis bone instead of the feet, keeping the feet on the floor. This time, the character bends automatically at the knees, as though crouching. (See Figure 3.38.)

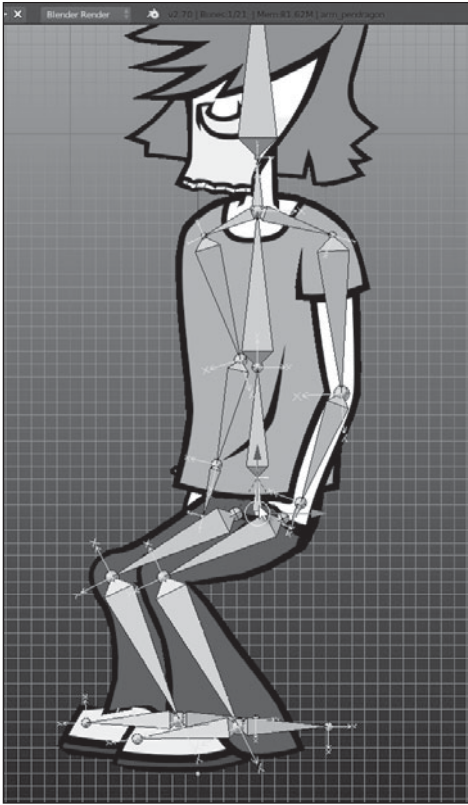


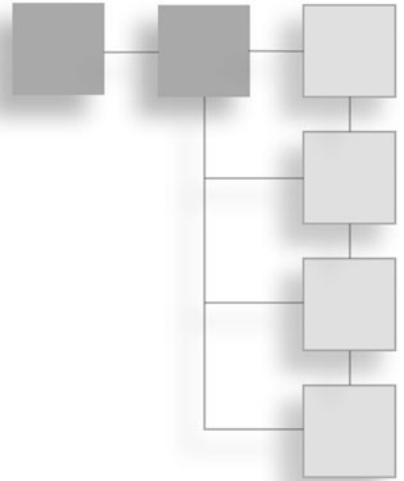
Figure 3.38
Moving the hips with IK.

Source: The Blender Foundation.

CONCLUSION

Splendid! The Pendragon character is now modeled, textured, and fully rigged for animation. You accomplished the rigging using a combination of armatures, skinning, and Inverse Kinematics. The general process is not short or obvious, but it's rewarding because the feature set gives you enormously flexible and far-reaching control over the pose for a character. With a rig in place designed to accommodate your animation needs, it's now time to move on to the process of animation itself. In that phase, you'll animate the pose of the armature over time and let it work its magic on your mesh to create a believable walk sequence that you can reuse for a real-time video game.

CHAPTER 4



ANIMATION

In this chapter, you'll take your fully modeled, textured, and rigged character and animate him through a complete walk cycle using the Blender animation tools. (See Figure 4.1.)

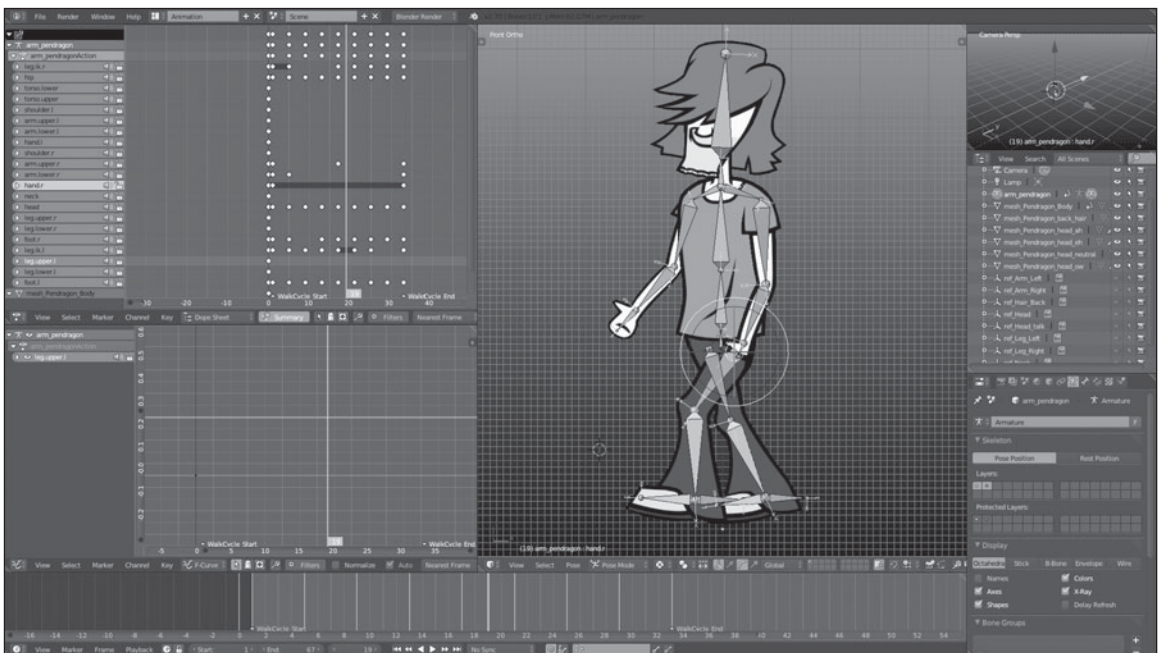


Figure 4.1
The rigged and animated Pendragon character.
Source: The Blender Foundation.

The term *walk cycle* means two things for animation and video games specifically:

- Your character will walk in a loopable and seamless way. That means you'll be able to play back the animation on a loop to make him walk continuously for as long as you need.
- The character will walk in-place. That means he'll walk on the spot, moving his legs, arms, and head and exhibiting all walking motions, but without actually moving forward in space. That's very important for games. It lets you move a pre-animated and walking character wherever you need during gameplay. This kind of walk cycle will play in-game without overriding or disrupting the general character motion, which often changes and is controlled by the player in real time.

To achieve these two features for walking, you'll use all three Blender animation toolsets: the Timeline, the Dope Sheet, and the Graph Editor. Let's get started!

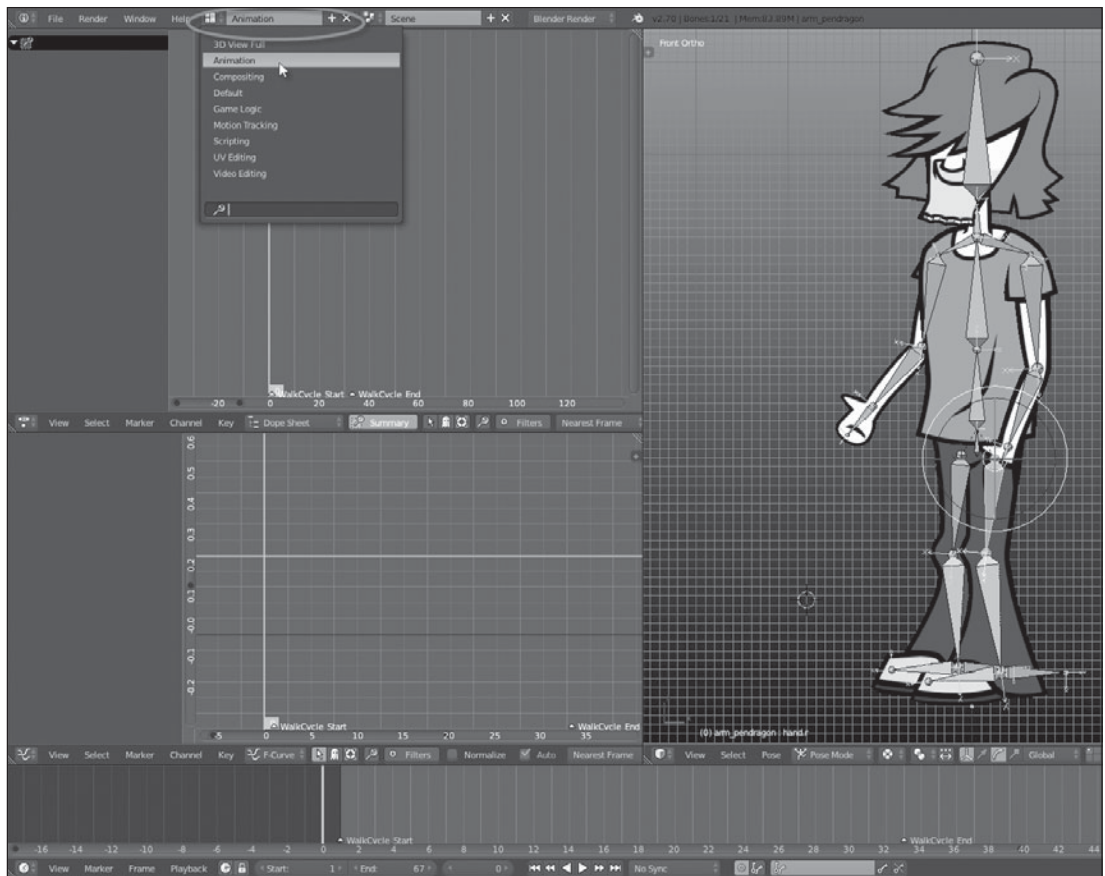
Note

The completed Pendragon model, including rigging information, is included in the eBook companion files in the Chapter04 folder. You can open the file and follow with this chapter, if you prefer.

STEP 16: PREPARING FOR ANIMATION

Before animating the Pendragon character, let's tweak and configure Blender to enhance your workflow. Follow these steps:

1. Change the default interface configuration to the Animation preset, which displays all three animation windows alongside the 3D viewport and Properties panel. This makes it easier to edit the animation and preview the results in real time. (See Figure 4.2.)

**Figure 4.2**

Enabling the Animation layout for Blender.

Source: The Blender Foundation.

2. The total frame duration for the walk animation should be 32 frames, so set the frame range in the Timeline window. To achieve this, type 1 in the Start setting and type 33 in the End setting. (Note that the final frame is 33 and not 32 because it will effectively be a duplicate of the first frame to create a looping sequence.)
3. To make things clearer, I like to add markers (text handles) in the Timeline window to show human-readable messages about where the start and end of animation is. To set a marker for frame 1, right-click the first frame in the Timeline window to set the playhead at that frame. Then press the M key to insert the marker or choose

Marker > Add Marker from Timeline menu. (See Figure 4.3.) To rename the marker, click it to select it and press Ctrl+M or Choose Marker > Rename Marker from the Timeline menu.

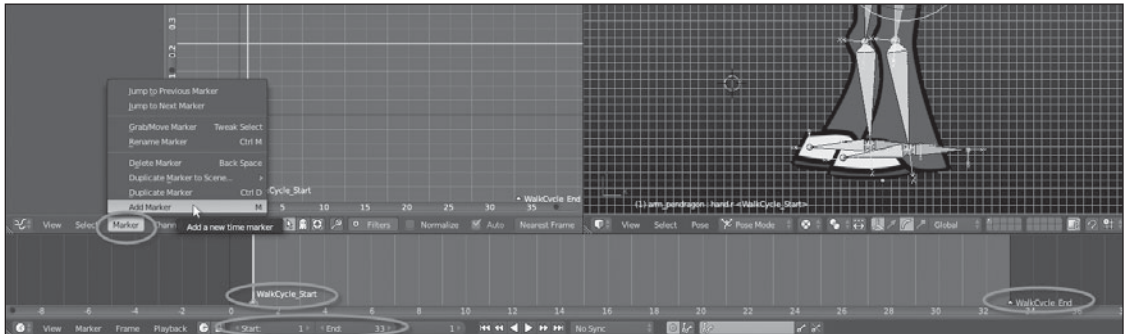


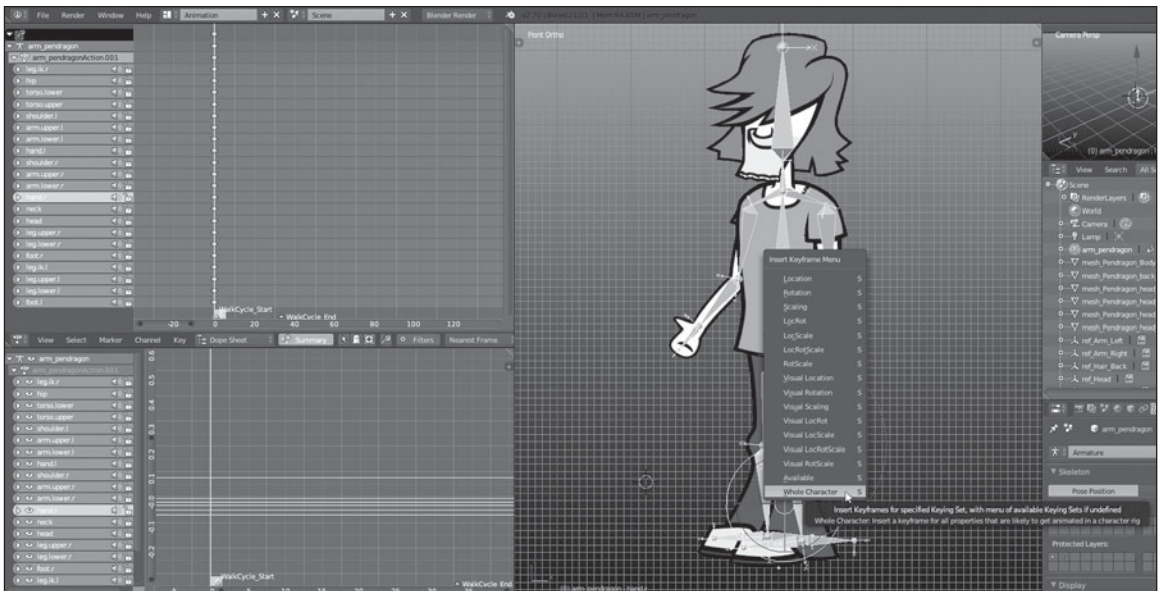
Figure 4.3
Creating a frame range and markers.

Source: The Blender Foundation.

Note

To position or slide the animation playhead in the Timeline or Dope Sheet, just right-click using the Maya Controls. For the Graph Editor, however, you must hold down the K key while left-clicking. You can change the controls, if preferred, through the User Preferences dialog box, accessible via File > User Preferences > Input.

- Record or “keyframe” the character’s default pose to frame 0 (before the first frame for the walk animation). This helps you easily revert to the default pose and even copy the pose to other frames if required. To do this, move the animation playhead to frame 0 in the Timeline window. Then select the armature object in the scene, enter Pose mode, select all bones, and press S or choose Pose > Animation > Insert Keyframe from the 3D viewport menu. This inserts a new keyframe for all bones. Finally, from the context menu that appears, choose Whole Character to set a keyframe recording the state for the complete armature. (See Figure 4.4.) A keyframe is added, represented by a dot.

**Figure 4.4**

Keyframe the character starting pose at frame 0.

Source: The Blender Foundation.

STEP 17: BLOCKING IN THE WALK CYCLE

Walking is often described as a series of “controlled falls.” That’s because walking is primarily about transferring weight in the body to maintain balance while moving.

The walk sequence for a biped, like a human, can be broken into four distinct stages, known as poses, each referring to the state the legs are in:

- The contact pose
- The down pose
- The passing pose
- The up pose

These four poses don’t constitute a complete walk cycle in themselves, but a walk cycle is made up of a repeated sequence of these basic poses linked together. These four poses simply define how one foot is moved in front of another. A complete walk cycle, therefore, requires eight main poses: the four poses played back twice. This allows one foot (X) to move in front of the other foot (Y), and then for Y to move back in front of X, completing the cycle. See Figure 4.5 for the four main poses.

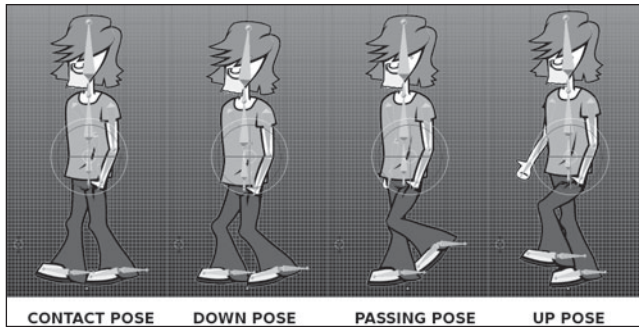


Figure 4.5
Walk cycle poses.

Source: The Blender Foundation.

The Contact Pose

Many walk cycles loop at the contact pose. This is where both feet are firmly planted on the ground and are spread apart at their limits. This pose is significant because it's an extreme pose—the character's legs will never be further apart than at the contact pose. It therefore marks the stride length of the character. To create this pose, follow these steps:

1. Move to the first frame (frame 1) and enable Auto-Key (see Figure 4.6). This enables Blender to detect the changes you make to the armature and to set keyframes automatically in the Timeline.

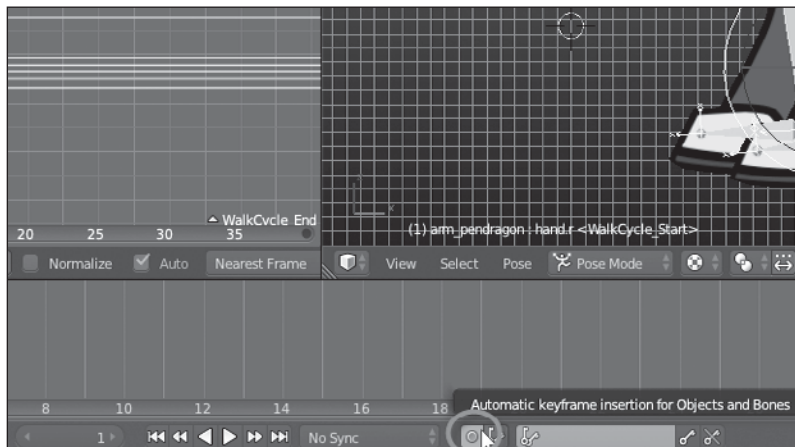


Figure 4.6
Enable Auto-Key.

Source: The Blender Foundation.

2. In Pose mode, pose the character's legs and arms in the contact pose. Remember, the character will move along at a constant and steady pace between frames 1 and 33. So the first contact pose is at frame 1, with the right foot forward and the left foot back.
3. For a complete walk cycle, you will need two more contact poses. The second is at frame 17, with the left leg forward and the right leg back—a mirror of the first contact pose. We'll discuss this pose in a moment. The third is at frame 33, which is identical to the first pose. To create the third pose, use the Dope Sheet to draw a box selection around all keys for the contact pose at frame 1 and then duplicate the selected keys by pressing Ctrl+D or selecting Key > Duplicate from the menu. (See Figure 4.7.) Then drag the duplicated keys to frame 33 and click the mouse or press Enter to insert them. This creates a duplicate set of keys for the final frame, so the poses at frame 1 and 33 are identical. (See Figure 4.8.)

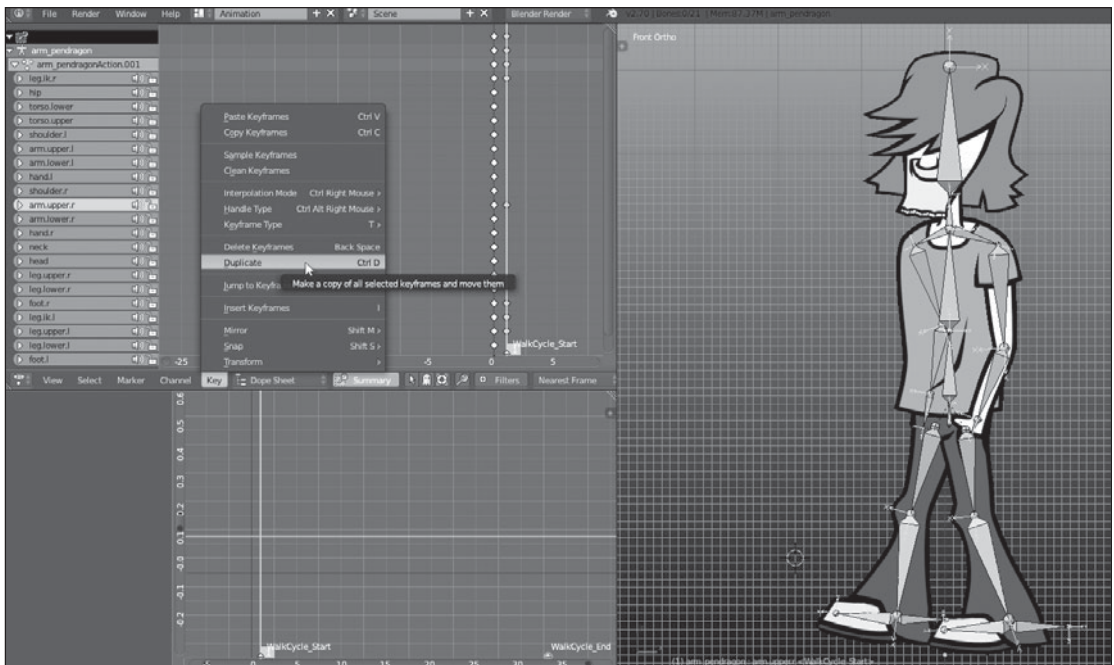
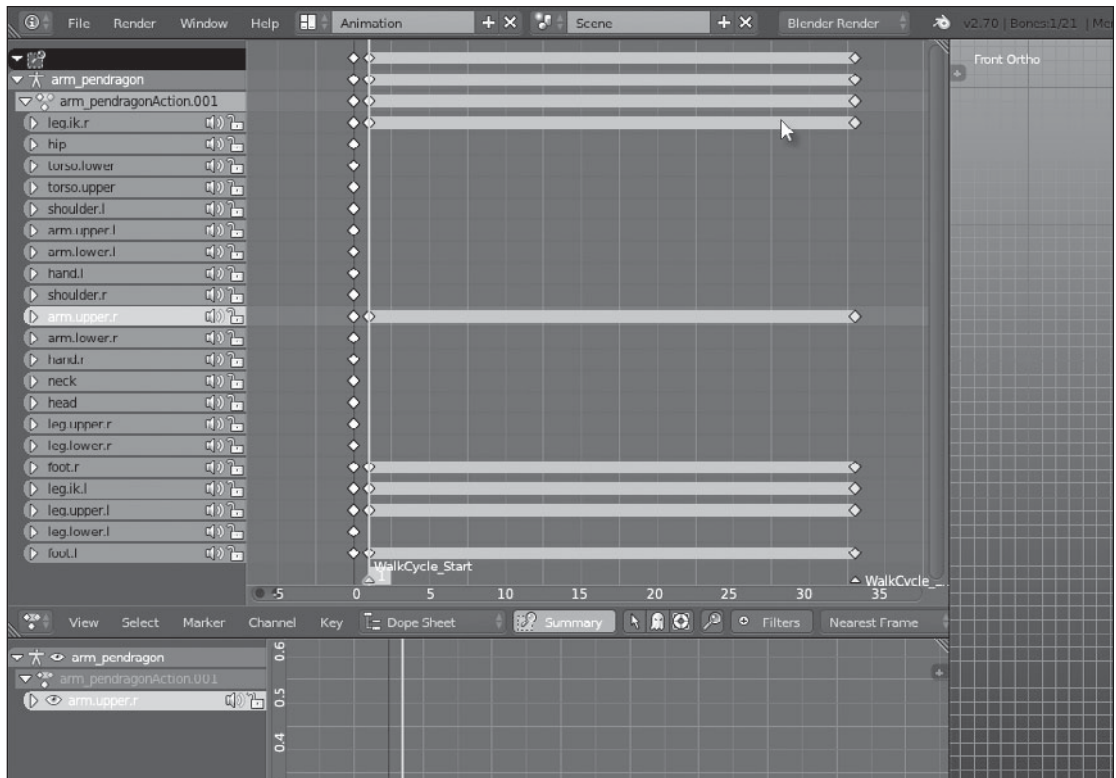


Figure 4.7
Duplicating keys.

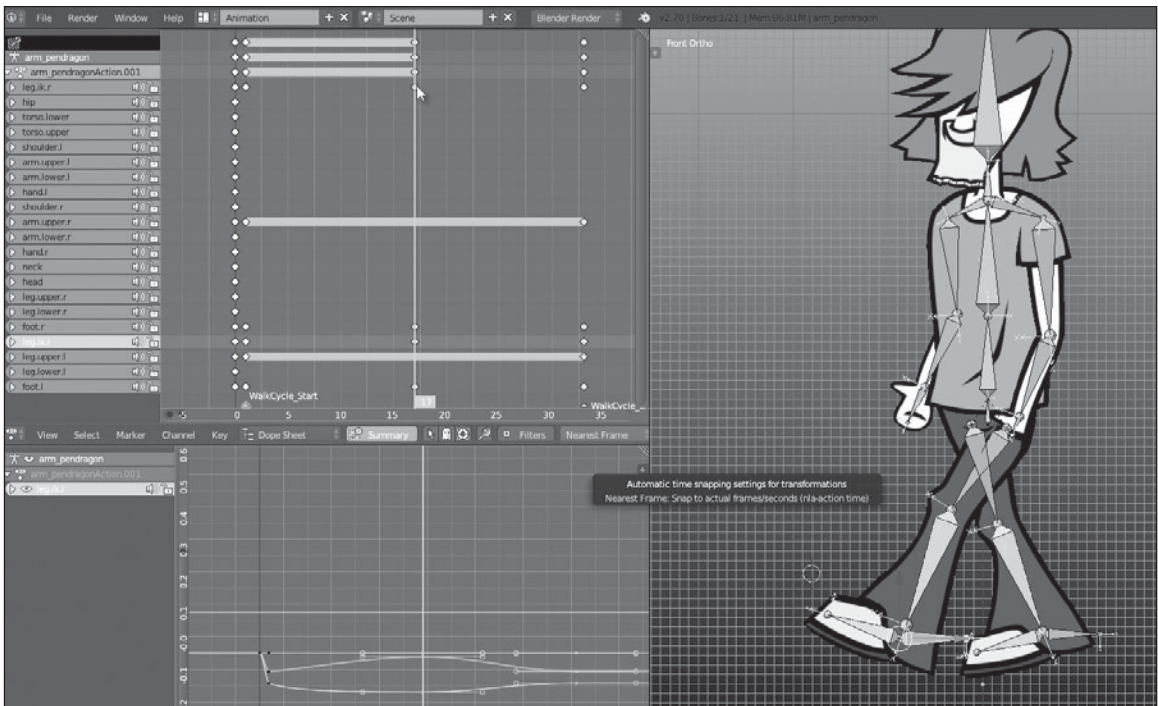
Source: The Blender Foundation.

**Figure 4.8**

Using key duplication to create the start and end contact poses.

Source: The Blender Foundation.

4. The remaining contact pose occurs at frame 17. To create this, duplicate frame 1 again and move the duplicate to frame 17. But as mentioned, this should be a mirror of the pose in frame 1, with the left leg forward and the right leg back. To achieve this, select the right leg and move it back, and select the left leg and move it forward, allowing Inverse Kinematics to handle leg rotation and bending. Be sure to keep the same distance between the legs as with the first and last contact poses to maintain the stride length. Assuming Auto-Key is enabled, Blender will update your keyframes to record the new pose. (See Figure 4.9.)

**Figure 4.9**

Creating the second contact pose at frame 17.

Source: The Blender Foundation.

This completes the three contact poses for the animation. You can play the animation in the Timeline to see how it is coming together, even at this early stage. The legs will slide back and forth between the three contact poses. This is a solid beginning, but the remaining poses must be added.

The Passing Pose

The passing pose occurs roughly midway between the two opposite contact poses. It's the point where the body rests its weight fully on one straightened leg to bring the other leg over and forward. In this phase, the back leg prepares to pass the front one, and the torso moves forward.

To create this pose between frame 1 and frame 17, follow these steps:

1. With Auto-Key enabled, move the playhead to frame 9 and position the left leg to pass the right, as shown in Figure 4.10. This pose should match the passing pose shown in Figure 4.5.

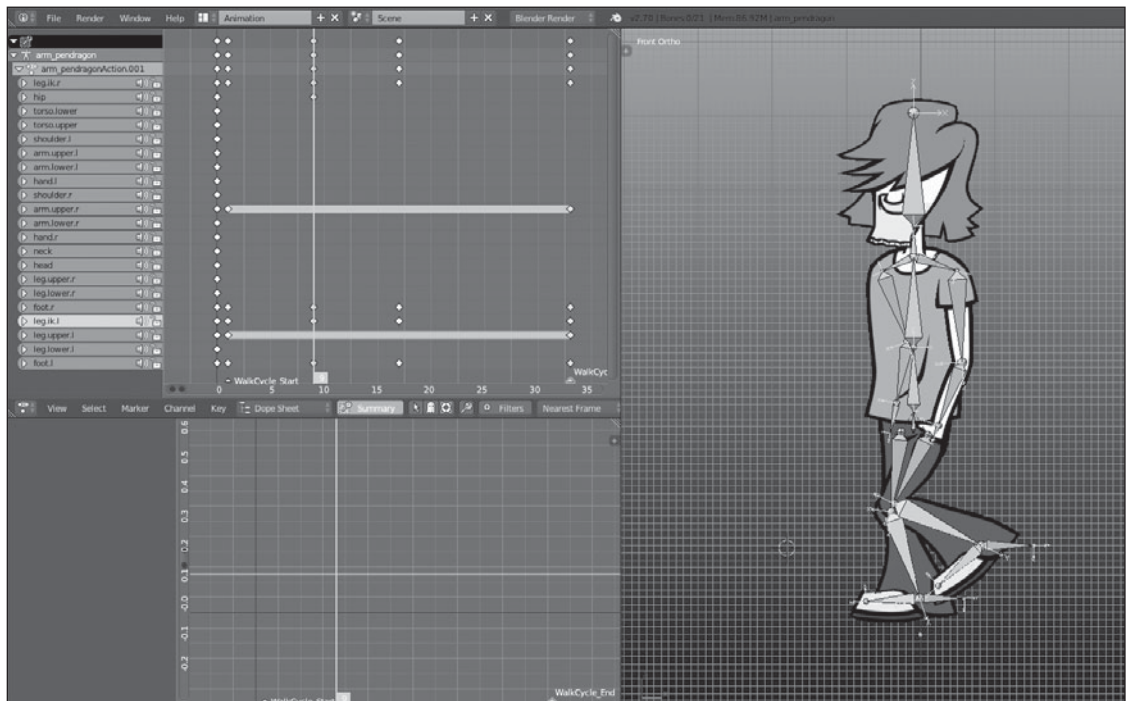
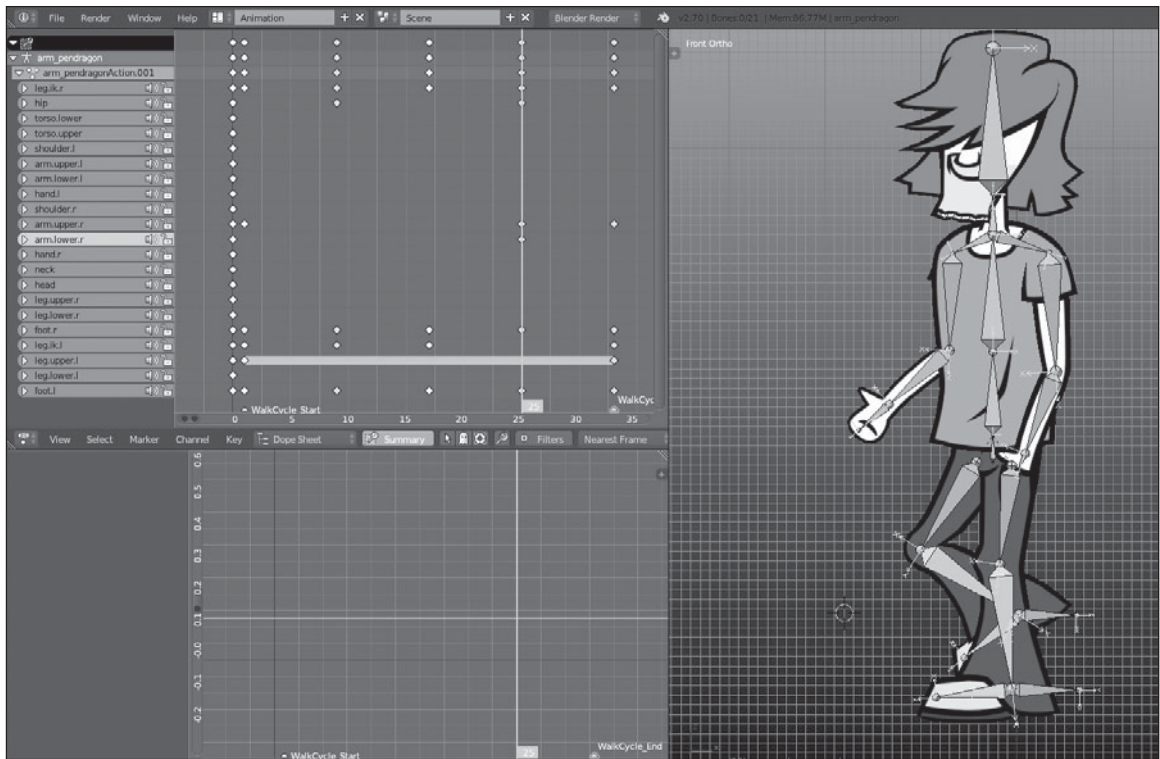


Figure 4.10
Creating the passing pose.

Source: The Blender Foundation.

2. Duplicate the passing pose keyframes at frame 9 and then move the duplicates to frame 25 to create the second passing pose between the contact poses at frames 17 and 33. Then reverse the positions of the legs. As shown in Figure 4.11, the right leg should be passing at frame 17 instead of the left leg.

**Figure 4.11**

Creating the second passing pose.

Source: The Blender Foundation.

The Down Pose

The down pose occurs roughly midway between the contact pose at frame 1 and the passing pose at frame 9. In the down pose, the body is preparing to transfer weight. Both of the character's knees bend slightly before he stands straight on one leg and lifts the other from the ground.

The down pose, along with the up pose (discussed next), is important for conveying the weight of your character. Exaggerated down poses make the character seem bulkier, heavier, sleepier, or lazier as he moves. Because Pendragon is a lazy and carefree character, his down pose should be fairly pronounced. To set the down pose, follow these steps:

1. Move the playhead to frame 5 and transform the armature bones to re-create the down pose shown in Figure 4.5. For this pose, both feet should be planted on the ground, and the hip bone lowered and brought forward. It should look something like Figure 4.12.

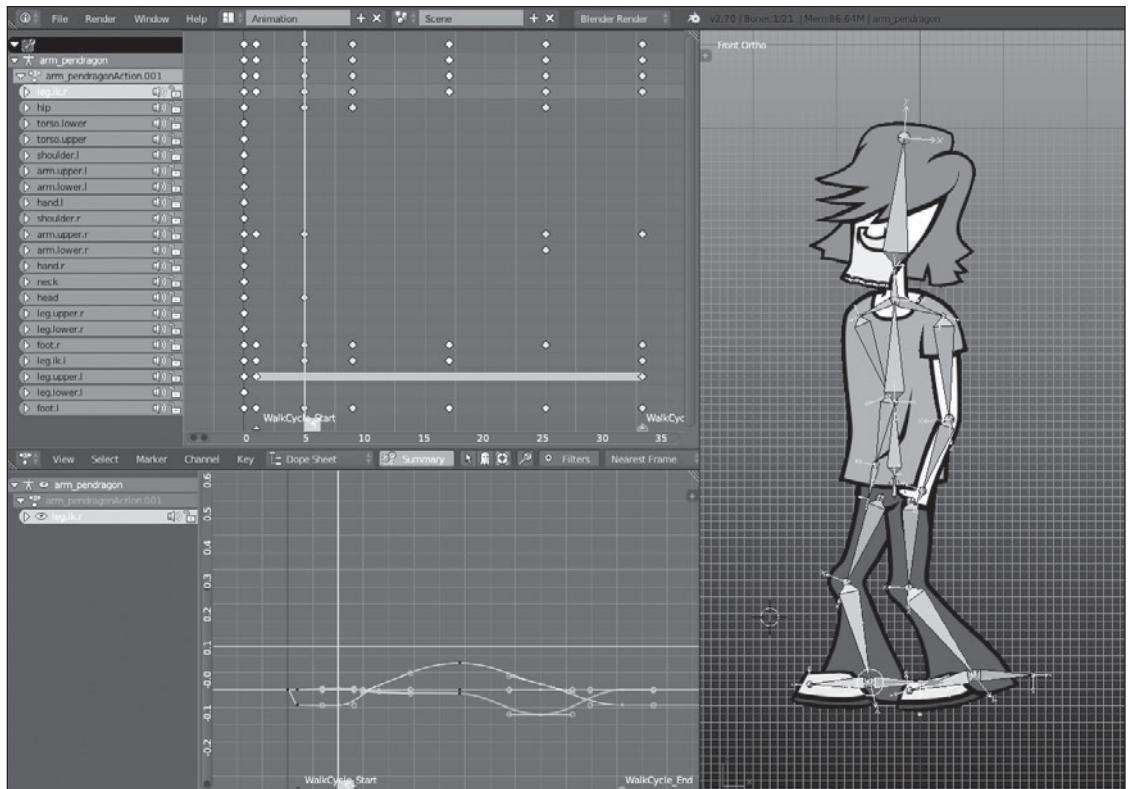
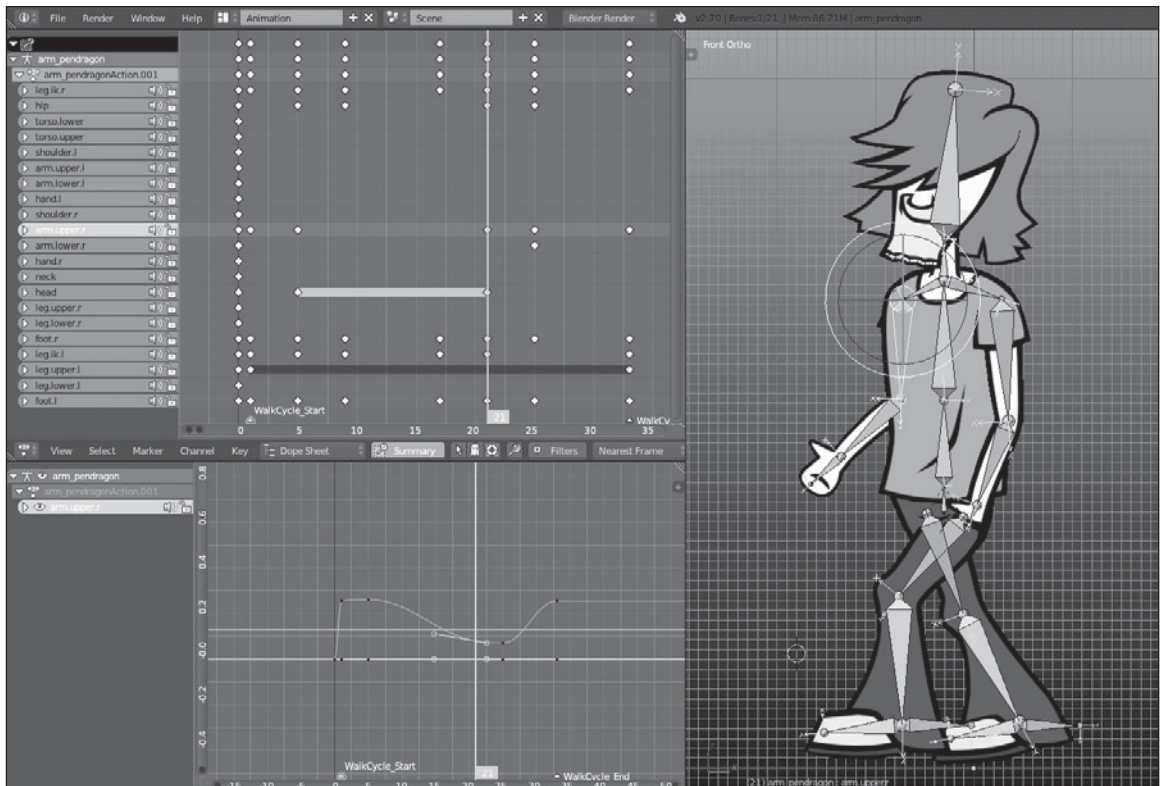


Figure 4.12
Creating the first down pose.

Source: The Blender Foundation.

2. As with the contact and passing poses, you must duplicate the down pose and adapt it to work for the second half of the cycle, where the right leg passes the left. The second down pose should occur at frame 21, between the second contact pose at frame 17 and the second passing pose at frame 25. See Figure 4.13.

**Figure 4.13**

Creating the second down pose, reversing the leg positioning from the first down pose.

Source: The Blender Foundation.

The Up Pose

The up pose first happens at frame 13, between the first passing pose at frame 9 and the second contact pose at frame 17. The up pose is the point where the trailing leg has passed and is coming down into contact with the ground. As this happens, the body raises slightly to transfer the weight from one side to the other.

Like the down pose, the up pose is significant in conveying the weight or mood of the character. Whereas the down pose can be exaggerated to express heaviness, slowness, and laziness, the up pose can be exaggerated to convey lightness, speed, and happiness.

For Pendragon, the up pose should be slightly subdued, as shown in Figure 4.14. As with the other poses, you should duplicate the up pose—it should occur at both frame 13 and frame 29—to complete the cycle. (See Figure 4.15.)

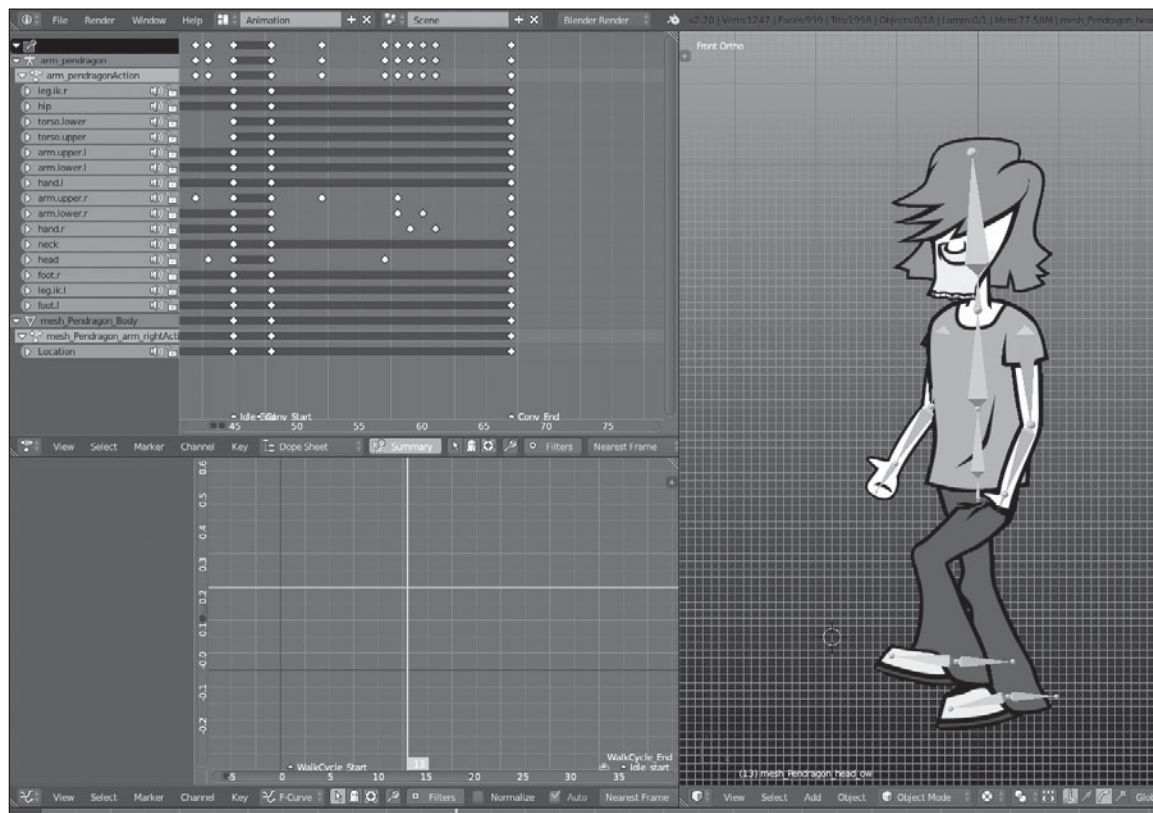


Figure 4.14

Adding the first up pose.

Source: The Blender Foundation.

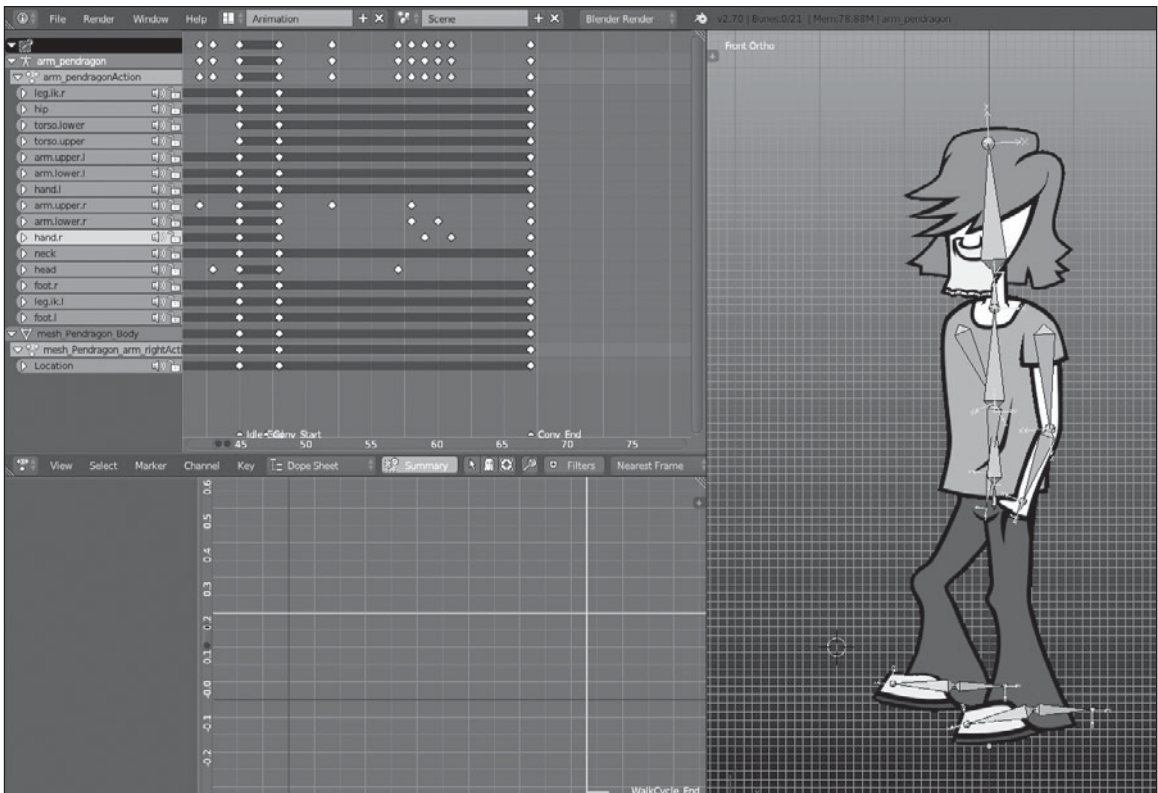


Figure 4.15
Adding the second up pose.

Source: The Blender Foundation.

STEP 18: MOVING THE ARMS

When walking, most people move their arms for balance. Typically, as the left leg moves forward, the right arm moves forward and the left arm moves back. Similarly, as the right leg moves forward, the left arm moves forward and the right arm back.

Pendragon is a somewhat exceptional case because his left arm always remains firmly inside his pocket. That means only the right arm needs to move. To create this motion, rotate the arm at each of the main poses to reflect the leg movement. (For guidance, refer to the figures for each main pose.) The rotation of the arm should be smooth and produce a curve or wave inside the Graph Editor. Otherwise, the motion will look sharp and unnatural. If you're experiencing this, select the animated arm bone, switch to the Graph Editor, and examine the keyframes in the graph as well as the curves generated

between the frames. If the graph looks sharp and jagged, consider changing the interpolation mode to Bezier. (To do this, select all arm keyframes. Then Ctrl-right click the frames and choose Bezier from the menu that appears, as shown in Figure 4.16.)

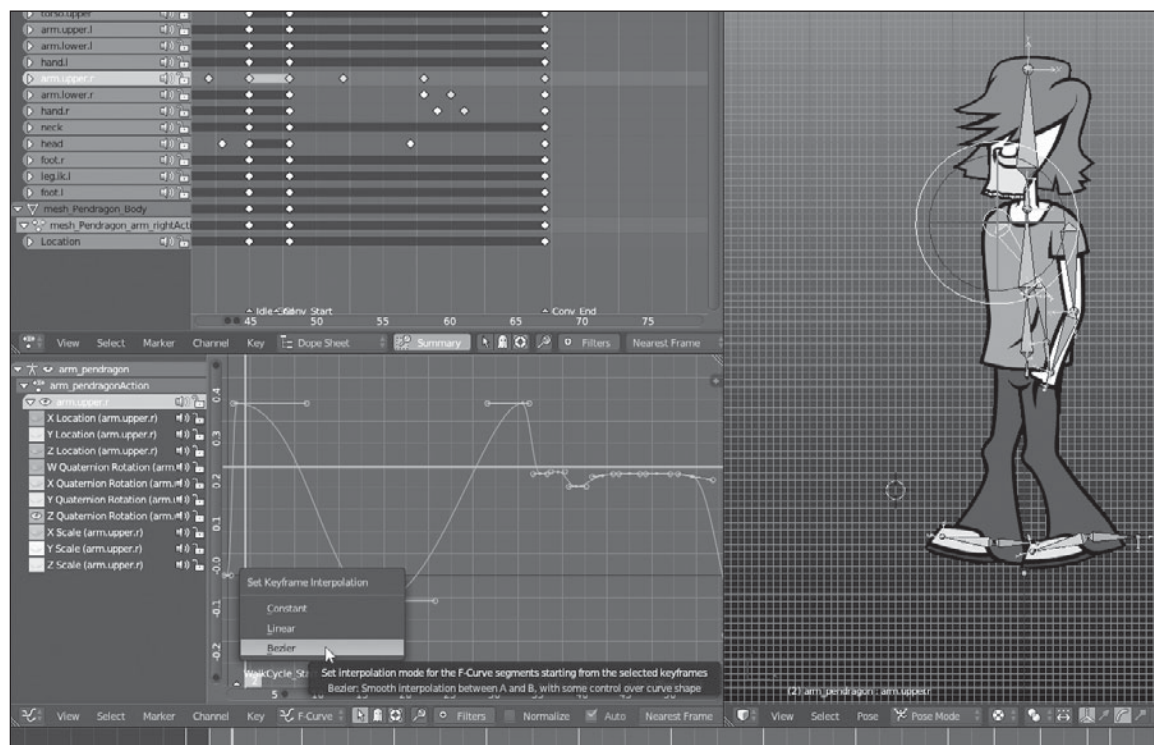


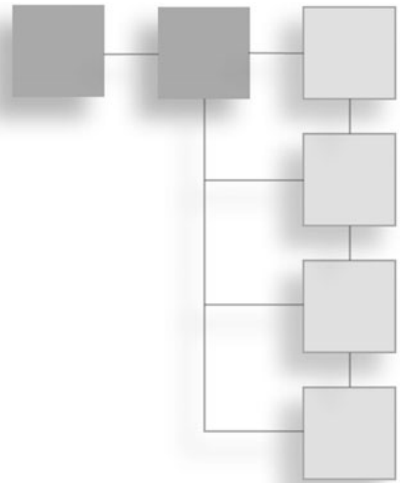
Figure 4.16
Tweaking keyframe interpolation for smoother motion.

Source: The Blender Foundation.

CONCLUSION

Pendragon's looping walk cycle is now complete. Excellent work! In reaching this far you've seen a comprehensive workflow for creating animated 2D characters using Blender. The main limitation of this method, as you may have noticed, is that it doesn't work so well for animations in perspective. That is, you can easily create characters from a side or three-quarters view who must walk from side to side (left and right), but you can't so successfully implement a character facing the camera who walks toward you or away. To create the latter, you'll typically need to draw out each frame or else be especially creative about how you apply the method given here. But for most 2D cutout animation, the method detailed here can serve you well—particularly for walks, runs, jumps, sits, idles, conversations, and other kinds of actions, too.

CHAPTER 5



EXPORTING AND TESTING

The Pendragon character is now completely modeled, textured, rigged, and animated. Now it's time to export him, complete with all appropriate data, from Blender into a game engine ready for inclusion in a real-time game. At this stage, there are many game engines a developer could choose, including the Unreal Engine, Leadwerks, Shiva, Godot, and plenty more. For this eBook, I'll choose the Unity engine, demonstrating how to optimally export Pendragon from Blender to Unity, including animation data. (See Figure 5.1.) So let's go!

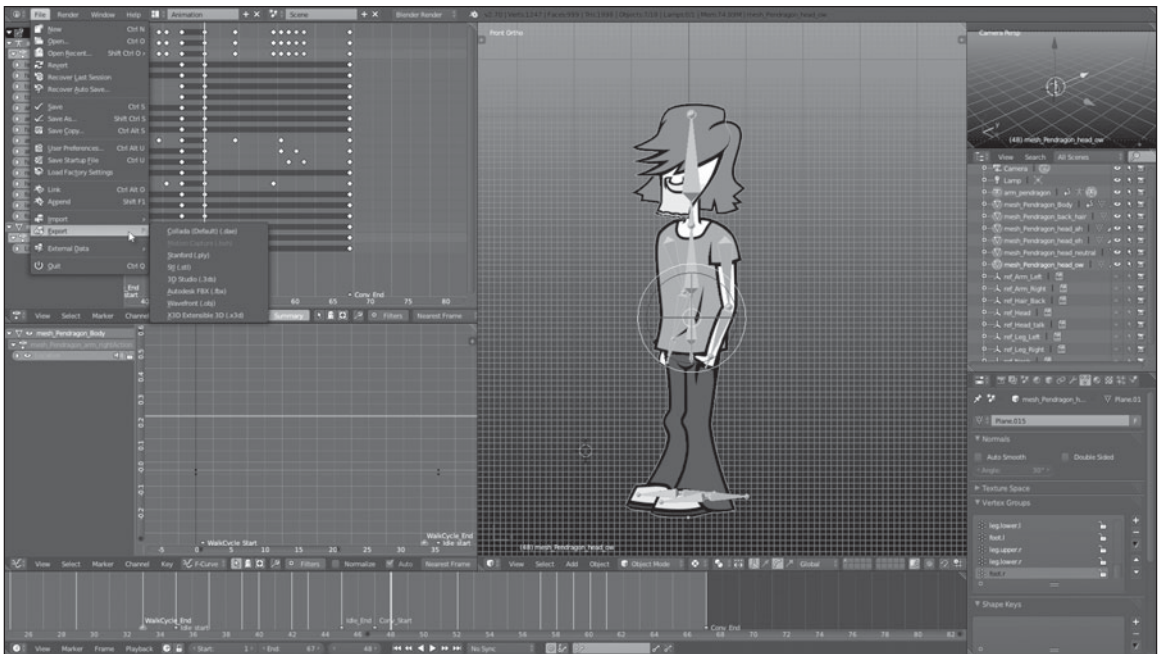


Figure 5.1
Preparing the Pendragon model for export.
Source: The Blender Foundation.

Note

The completed Pendragon model, including rigging and animation, is included in the eBook companion files in the Chapter05 folder. You can open the file and follow with this chapter, if you prefer.

STEP 19: EXPORTING OBJECTS FROM BLENDER

Unity supports several mesh formats for import. Indeed, you can make models and scenes in many formats—including MAX, MB, Blend, and more—and then import them effectively into Unity. Not all formats and workflows are created equal for in-game performance, however. Consequently, you must take special care when choosing a file format for exporting animated models from Blender.

So which file format should you use? Well, one you *shouldn't* use is the native Blend format. It's likely to contain additional, Blender-specific data that Unity simply doesn't understand or use, which is simply inefficient. Instead, export your models using either the Autodesk FBX format or the Open Standard DAE format. Both formats work with Unity and offer finer control over the kind of data Blender exports with your mesh files. This section explores mesh export with the popular and widely supported FBX format.

1. Select all mesh parts for Pendragon (head, legs, arms, etc.) and the animated armature. These objects represent the items to be exported. (You don't need to export objects like cameras, lamps, empties, references planes, and other helpers.)
2. Move the animation playhead in the Timeline to frame 1 (the start frame).
3. Ensure the animation end frame is set correctly. It should reflect the true, last frame of animation in the sequence. The Blender FBX Exporter uses the end frame value to determine the total amount of animation data to export to the FBX File.
4. Choose File > Export > Autodesk FBX. (See Figure 5.2.)
5. The FBX Exporter displays many options for controlling how the FBX file is exported, enabling you to include or exclude specific data from the scene. To export an animated character that's selected in the scene, configure the FBX Exporter as follows (see Figure 5.3):
 - Select the Selected Objects checkbox to limit the export to only the selected objects in this scene. In this way, you avoid exporting unnecessary data.
 - Shift-click the Armature and Mesh options. This enables you to export both armature and mesh objects. The armature is exported as bones, which feature important animation and structural data for your model.

- Select the Apply Modifiers checkbox to bake all modifier data into the mesh that's exported. This process works nondestructively. That is, after export, your mesh data in the Blender scene will remain unchanged.
- Select the Include Animation and Optimize Keyframes checkboxes.

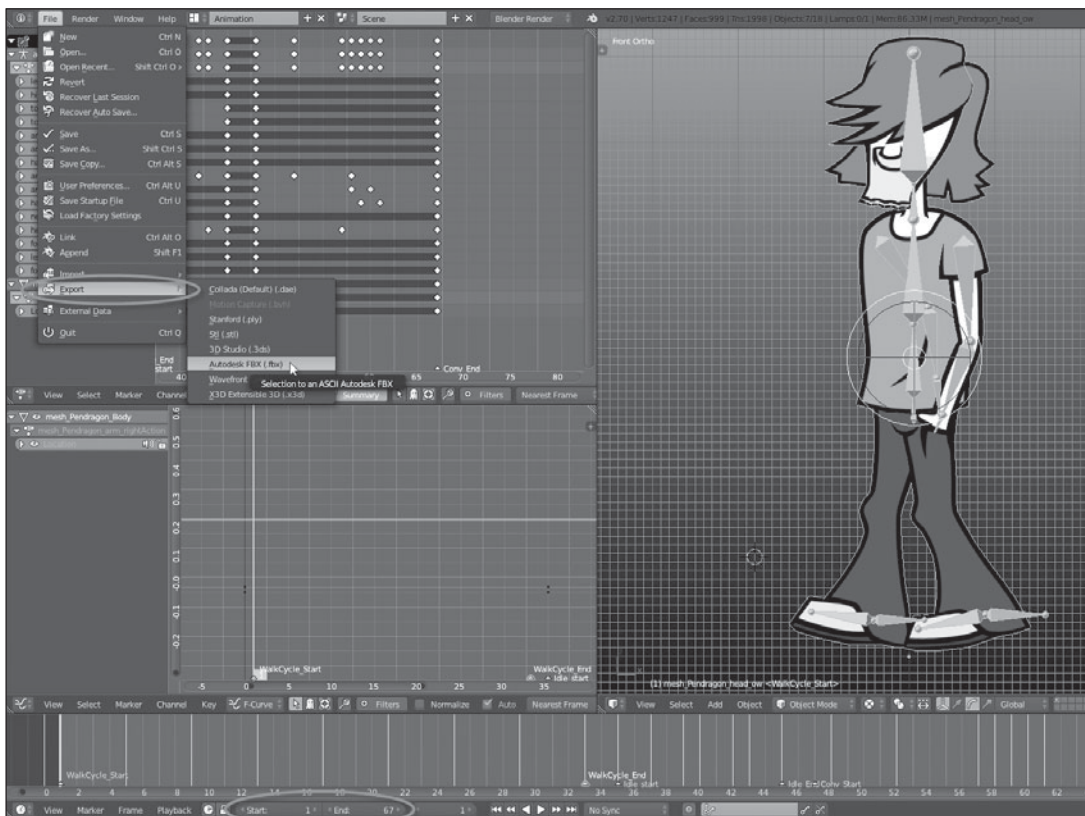


Figure 5.2
Accessing the Blender FBX Exporter.
Source: The Blender Foundation.

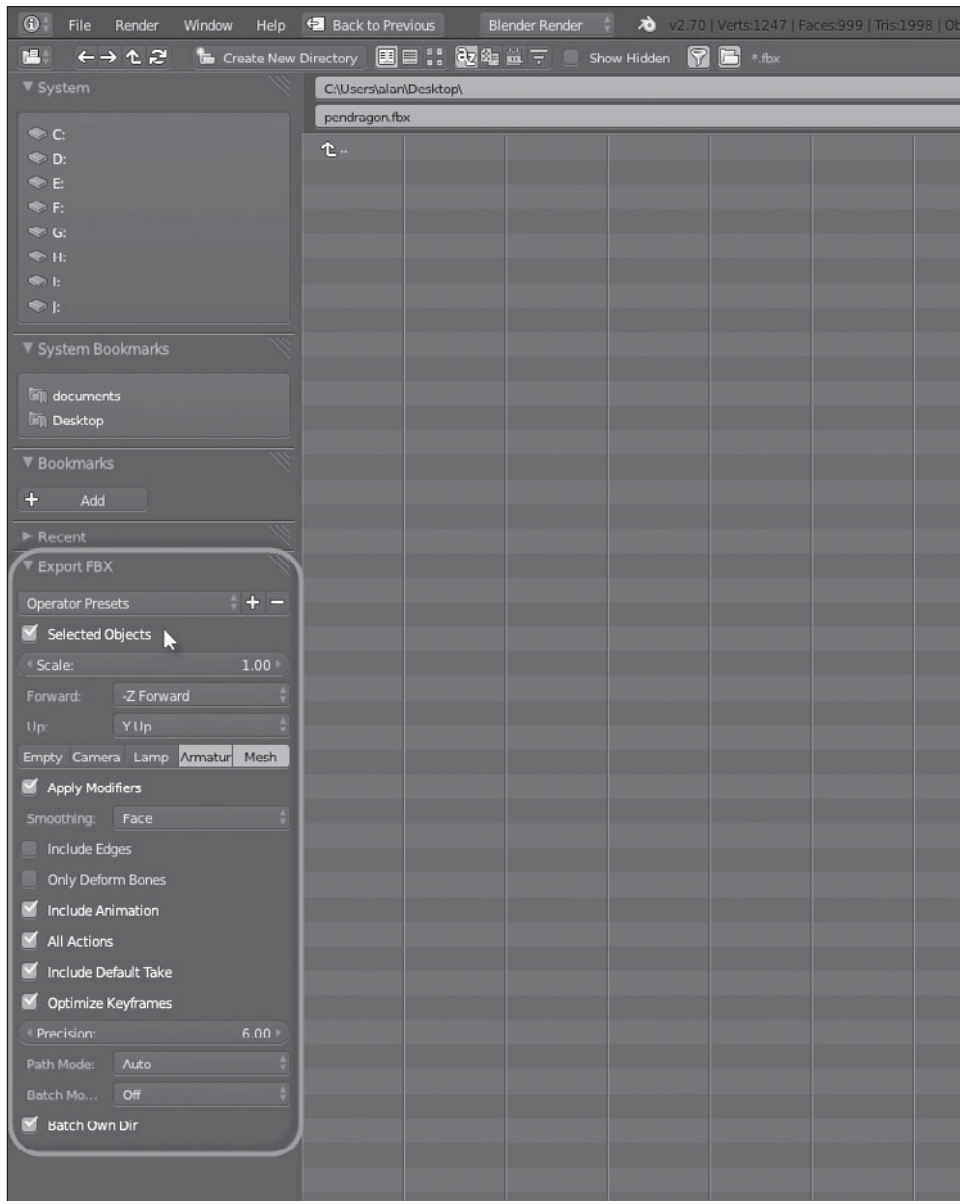


Figure 5.3

Using the FBX Exporter to export an animated mesh.

Source: The Blender Foundation.

6. After you set these options, click the Export FBX button in the top-right area of the Export FBX dialog box.

STEP 20: IMPORTING OBJECTS INTO UNITY

To import animated characters into Unity, you need two main files: the character mesh and animation data itself (included in the FBX file) and its associated texture map (which can be any valid image format—PNG, TGA, etc.). When you have both these files, you're ready to import your character!

Caution

For the texture map, avoid using the JPEG format due to its lossy compression.

1. First, you import the texture, and then import the mesh. This enables Unity to automatically detect and assign the texture to the mesh, allowing you to see the texture result in its thumbnail previews in the Project panel. To import both the texture and mesh files, simply drag and drop them from a Windows Explorer window or a Mac Finder into the Project panel. (See Figure 5.4.)

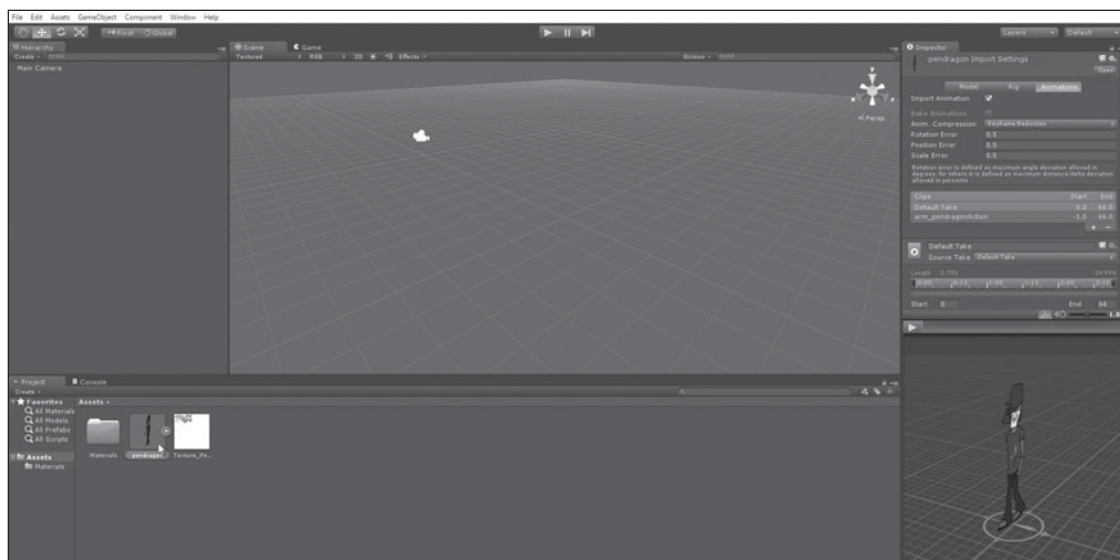


Figure 5.4

Importing an animated character from Blender into Unity.

Source: Unity Technologies.

2. In many cases (though not all), you'll want the imported character to remain unaffected by standard scene lights and lighting. For a cartoon style, the character is probably illuminated already as defined inside the texture. If that's the case, you can

disable scene lighting for the character material, making it self-illuminated. To do so, select the character material in the Project panel and change its shader type from Diffuse to Unlit > Texture, as shown in Figure 5.5. When you do, the character mesh will change appearance in the Preview panel, probably appearing brighter.

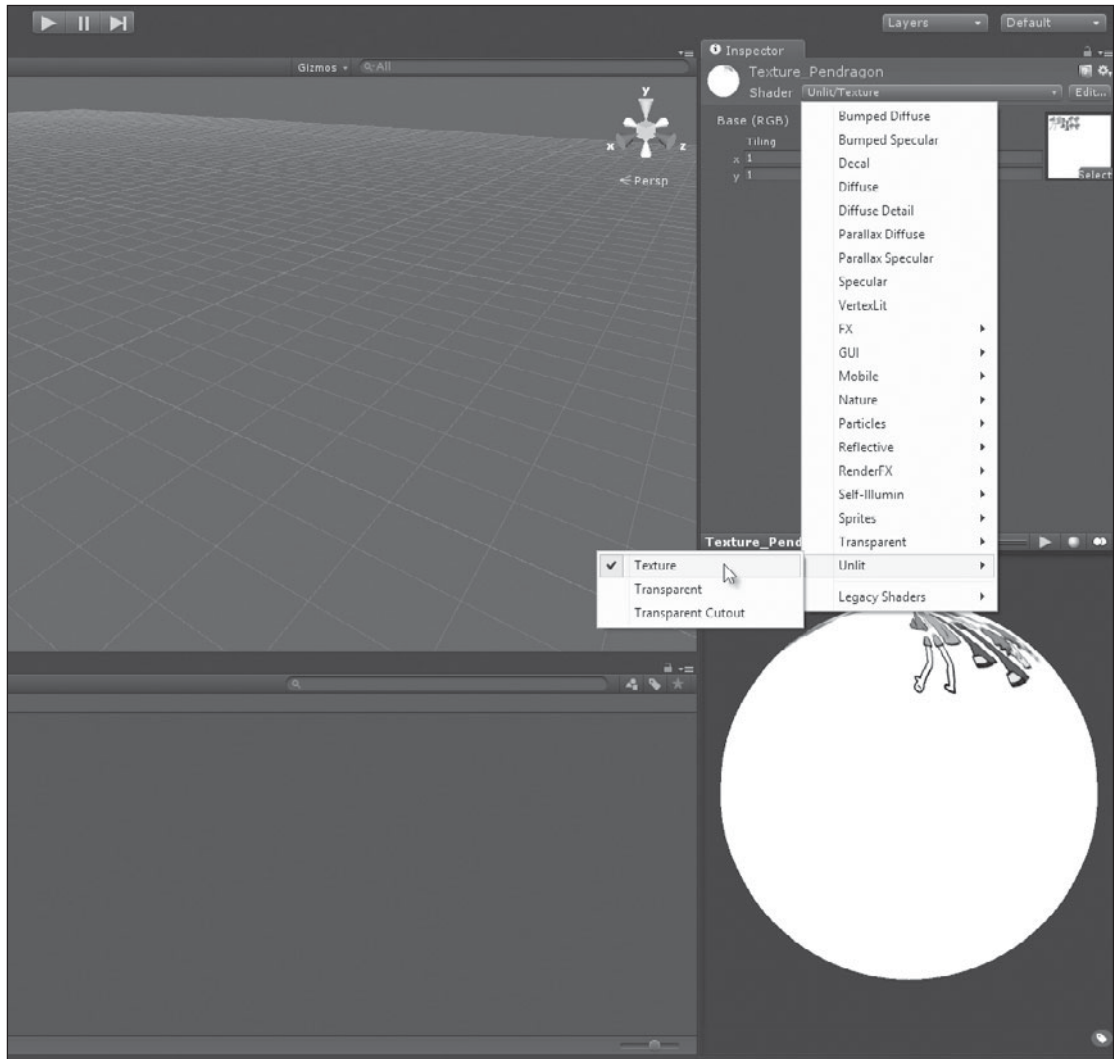


Figure 5.5
Tweaking the character shader to be self-illuminated.
Source: Unity Technologies.

3. Select the Pendragon mesh in the Project panel and click the right-pointing arrow on the mesh thumbnail. This expands the mesh contents to show all the imported components, including all mesh pieces, the armature bone data, and the animation data. (If one of these components is missing, return to Blender, select all the required objects, and use the FBX Exporter to reimport the mesh.) The complete Pendragon mesh will look something like Figure 5.6 in Unity.



Figure 5.6
Reviewing the imported mesh contents.
Source: Unity Technologies.

4. The default Scale Factor setting for the Pendragon mesh, found in the Object Inspector's Model tab, will likely be 0.01. (See Figure 5.7.) This value is not what you want for the mesh, however. Change the value from 0.01 to 1. Then drag and drop the mesh into the active scene for test purposes. Ascertain whether its size is right for your project, tweaking it as required by changing the Scale Factor setting in the Object Inspector. I settled on a Scale Factor of 8. Remember, the mesh you added to the scene is simply a test object for adjusting the Scale Factor setting to a more suitable value. When you've settled on a Scale Factor value, you can delete the object.

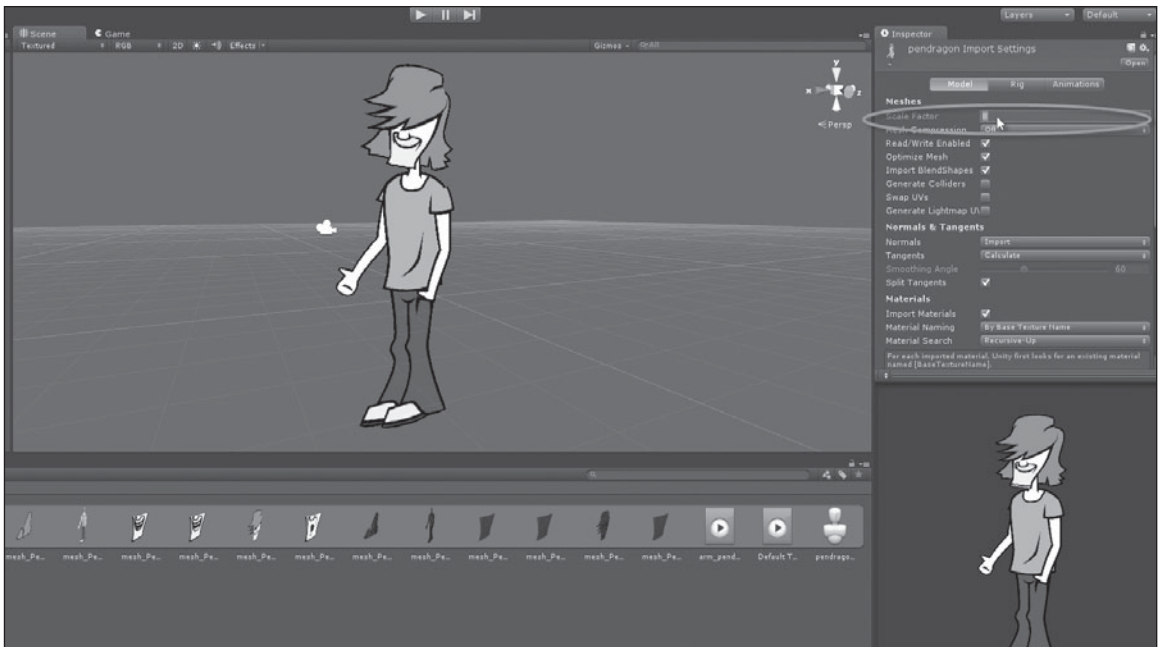


Figure 5.7

Changing the mesh's Scale Factor setting.

Source: Unity Technologies.

STEP 21: CONFIGURING THE MESH

Next, let's tweak the mesh settings even further. Follow these steps:

1. On the Model tab, deselect the Import BlendShapes, Generate Colliders, and Generate Lightmap UVs checkboxes. (See Figure 5.8.)

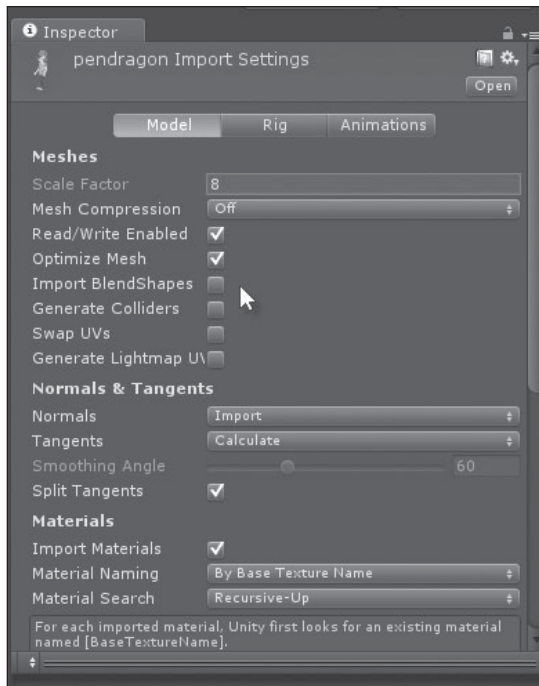


Figure 5.8
Configuring settings in the Model tab.

Source: Unity Technologies.

2. If your mesh needs colliders, consider using collider primitives, available from the Component > Physics menu. These perform faster than mesh colliders.
3. Click the Rig tab. Then select Generic from the Animation Type drop-down list. Leave all remaining settings at their defaults, as shown in Figure 5.9.

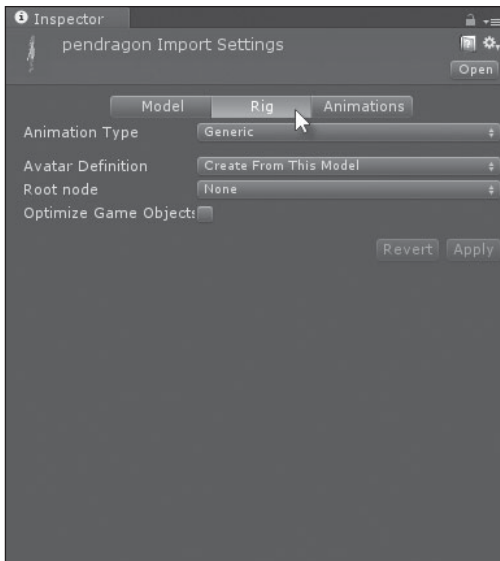


Figure 5.9
Configuring settings in the Rig tab.

Source: Unity Technologies.

4. Click the Animation tab. In this tab, Unity enables you to carve up the Timeline included in the file into separate animations. You can then play and loop these animations individually within Unity. Chapter 4, “Animation,” discussed how to create a walk animation for Pendragon, but the Pendragon file imported here also includes additional animations, such as a talk animation and an idle animation. These animations are included on the same animation Timeline. To define the complete walk cycle created in Chapter 4, enabling Unity to recognize it as a single animation sequence, start by selecting the default take in the Animation tab to show the animation settings in the file. (See Figure 5.10.)

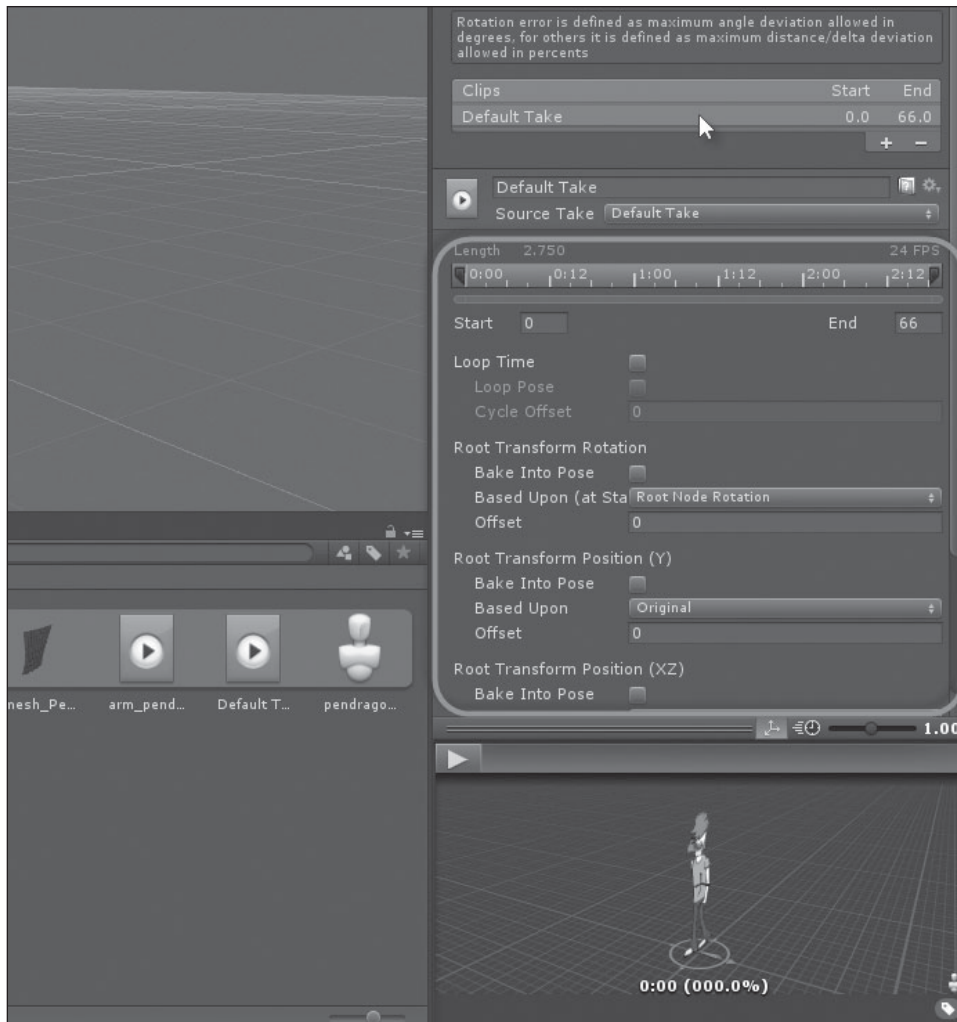


Figure 5.10
Previewing animation data.
Source: Unity Technologies.

- The complete Timeline included in the file spans from frames 0 to 66, but the walk-cycle animation is only a subset of these frames. The first frame of the walk cycle is 1 and the last frame is 33. Based on this, specify the start frame as 1 and the end frame as 32, not 33. This is because the animation must loop, repeating the first frame once completed. If frame 33, which is identical to frame 1, were included in the cycle, there would be a noticeable and undesirable pause where the two duplicate frames are played in sequence.

6. Select the Loop Time checkbox to mark the animation as a looping cycle. Figure 5.11 shows the settings for the walk animation.

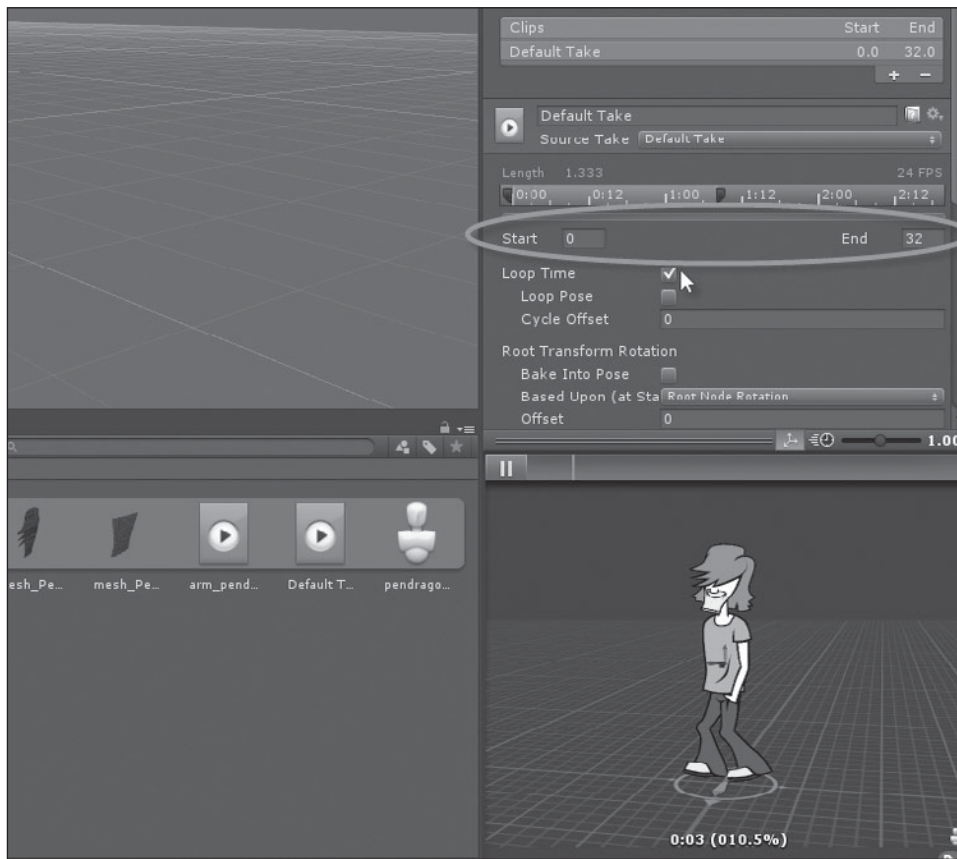


Figure 5.11
Defining the walk animation.

Source: Unity Technologies.

7. Click the Play button in the Object Inspector Preview panel to see the animation sequence in real time and make sure it looks as you intended.
8. If the animation looks as you intended, click the Apply button in the Object Inspector to save your changes. These changes are not written to the original FBX file, but are included in Unity-specific files as metadata for the asset. (If you need to revert to the original file, simply click the Revert button.)

As mentioned, in addition to the walk cycle, the Pendragon mesh also features a conversation animation and an idle animation. The conversation animation should play on a loop when Pendragon is speaking, and the idle animation should play when he's standing still. The idle animation spans frames 35 to 45, and the conversation animation spans frames 48 to 67. To configure the idle animation, follow these steps:

1. Click the plus (+) icon to create a new animation clip.
2. In the Source Box, type Idle to name the animation clip. (See Figure 5.12.)

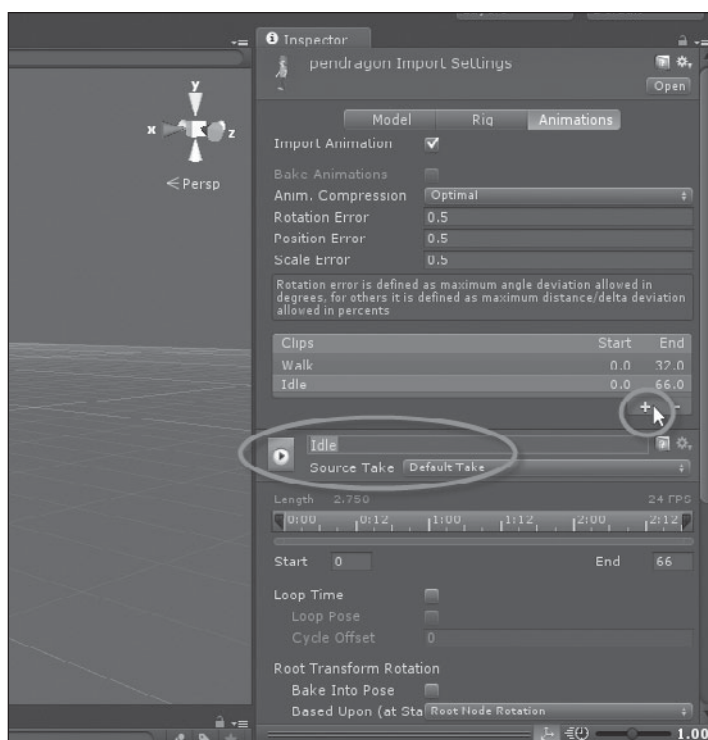


Figure 5.12

Creating a new animation clip for the idle animation.

Source: Unity Technologies.

3. Repeat the steps you performed with the walk animation, but change the start and end frames to 35 and 44, respectively. (Again, you must subtract 1 from the end frame to maintain a smooth looping cycle.)

To configure the conversation animation, repeat these steps, but name the clip “Conversation” and change the start and end frames to 48 and 65, respectively. Figure 5.13 shows all three animations—walk, idle, and conversation.

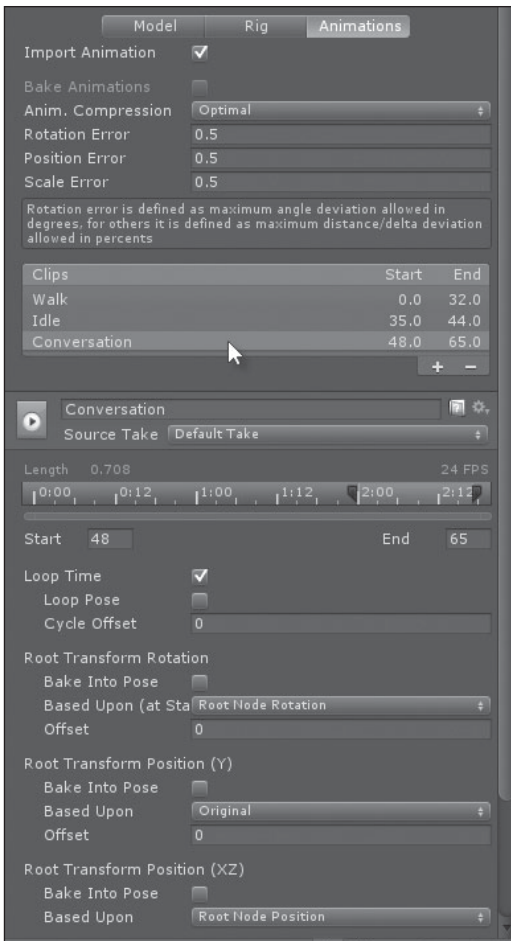


Figure 5.13
Creating the idle and conversation animations.

Source: Unity Technologies.

When you've created the animation clips for the Pendragon character, click Apply to save the changes. You can confirm that the animation clips have been added to the mesh by expanding it in Project panel and viewing its constituent pieces. In addition to the mesh and armature data, it should now contain animation clips—one clip for each separate and independent animation. (See Figure 5.14.)

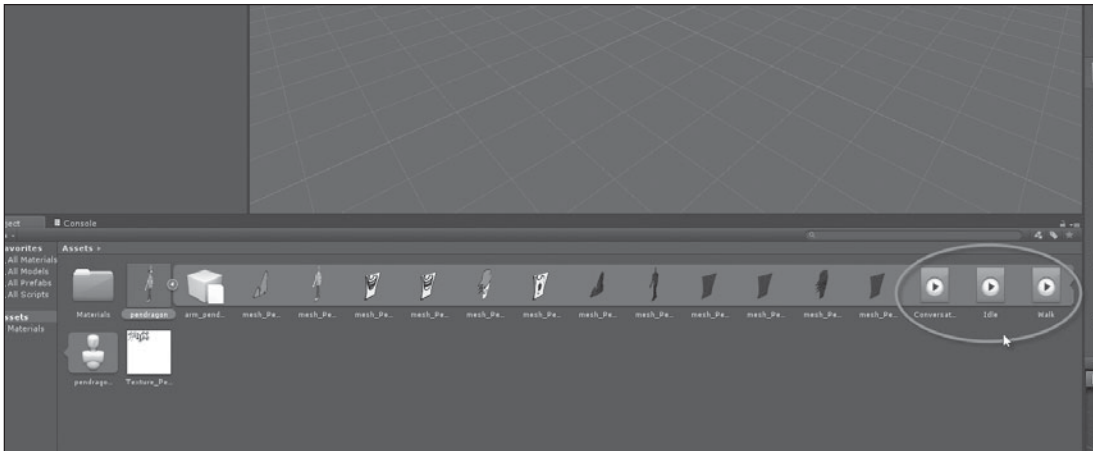


Figure 5.14
Mesh with animations added.
Source: Unity Technologies.

STEP 22: TESTING THE MESH

Here comes the moment of truth—the part where you test the animated mesh in-game to see how it works. Follow these steps:

1. In Unity, drag and drop the character mesh into the scene and position it in view of the scene camera (or else move the camera), as shown in Figure 5.15.

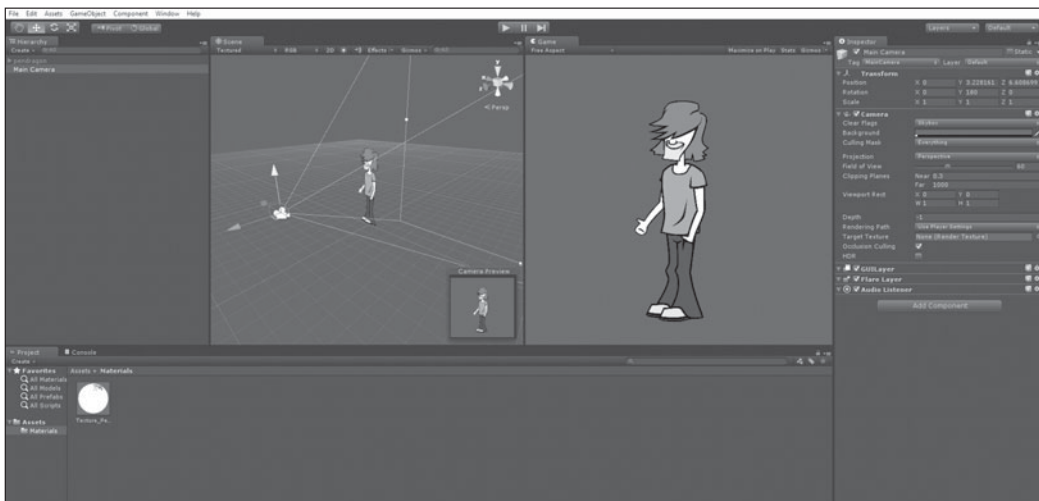


Figure 5.15
Adding a character mesh to the scene.
Source: Unity Technologies.

2. The default scene camera is configured as a perspective camera, which means its rendering feature set is optimized for 3D scenes and objects rather than 2D ones. You can fix this manually by using an orthographic camera, configured to a scale of 1 world unit to 1 pixel. To do this, first select the camera and change its Projection setting to Orthographic.
3. Set the Target Resolution in the Game tab toolbar to full HD (1920 × 1080) and change the Size setting in the Camera panel to 540 (which is half the vertical resolution—that is, 1,080/2). Note that you may need to scale up the Pendragon character to improve his visibility in the viewport. (See Figure 5.16.)

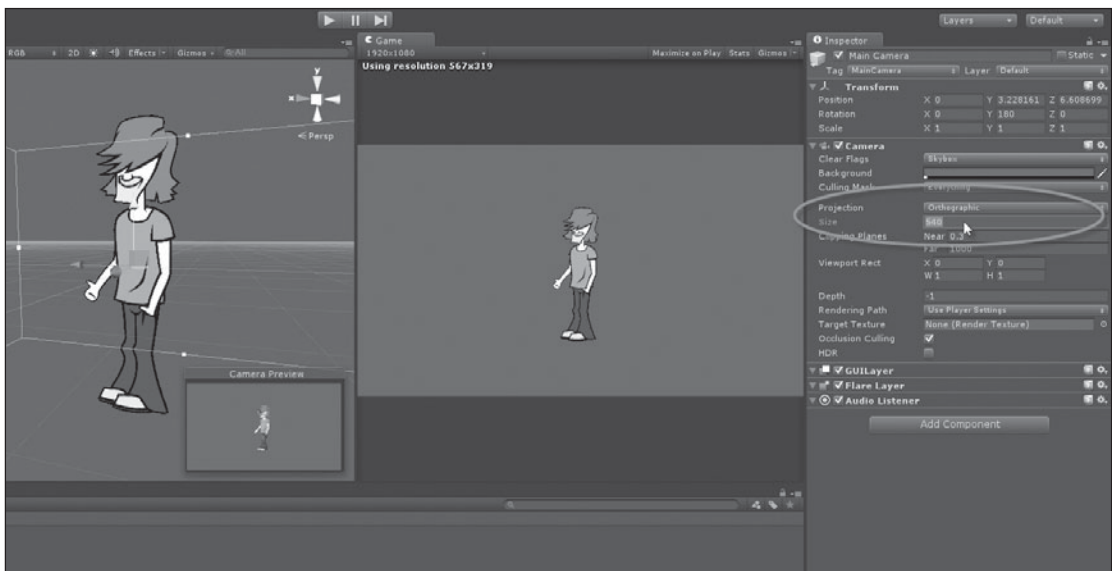


Figure 5.16

Viewing the 2D character from an orthographic camera.

Source: Unity Technologies.

4. You now have a “pixel perfect” view of the character onscreen. But if you click Play on the toolbar to run the game, you’ll see the character remains static and motionless. The animations simply don’t play. To fix this, you need to create a Mecanim state machine asset. To do so, right-click in the Unity Project panel and choose Create > Animator Controller, as shown in Figure 5.17. This creates a new Graph or State Machine asset in the project. This defines which animation should play on the character mesh in the level.

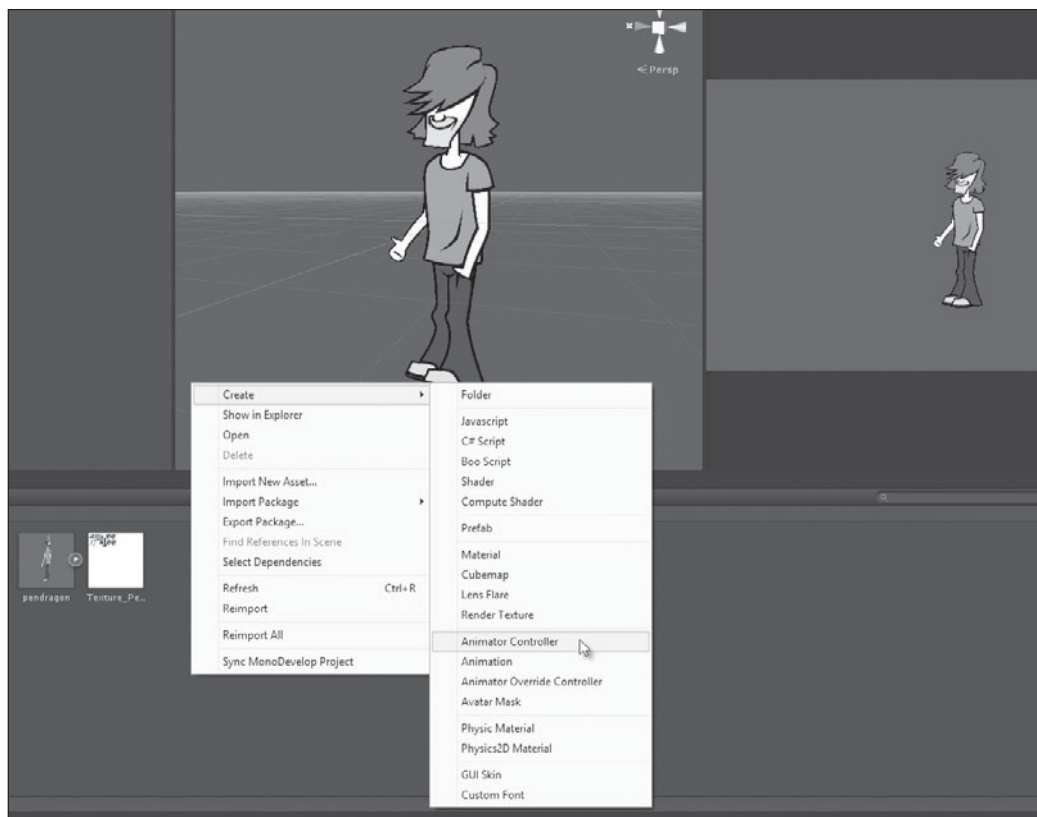


Figure 5.17
Creating an Animator Controller in the Unity project.
Source: Unity Technologies.

5. To view and edit the newly created Animator Controller asset, choose **Window > Animator** to open a new Animator window. Then double-click the Animator Controller asset in the Project panel to open it inside the Animator window. This window displays the contents of the Animator Controller. By default, the controller features only one node (or state), named “Any State.” On its own, this state will have no appreciable effect on the Pendragon mesh in the scene. (See Figure 5.18.)

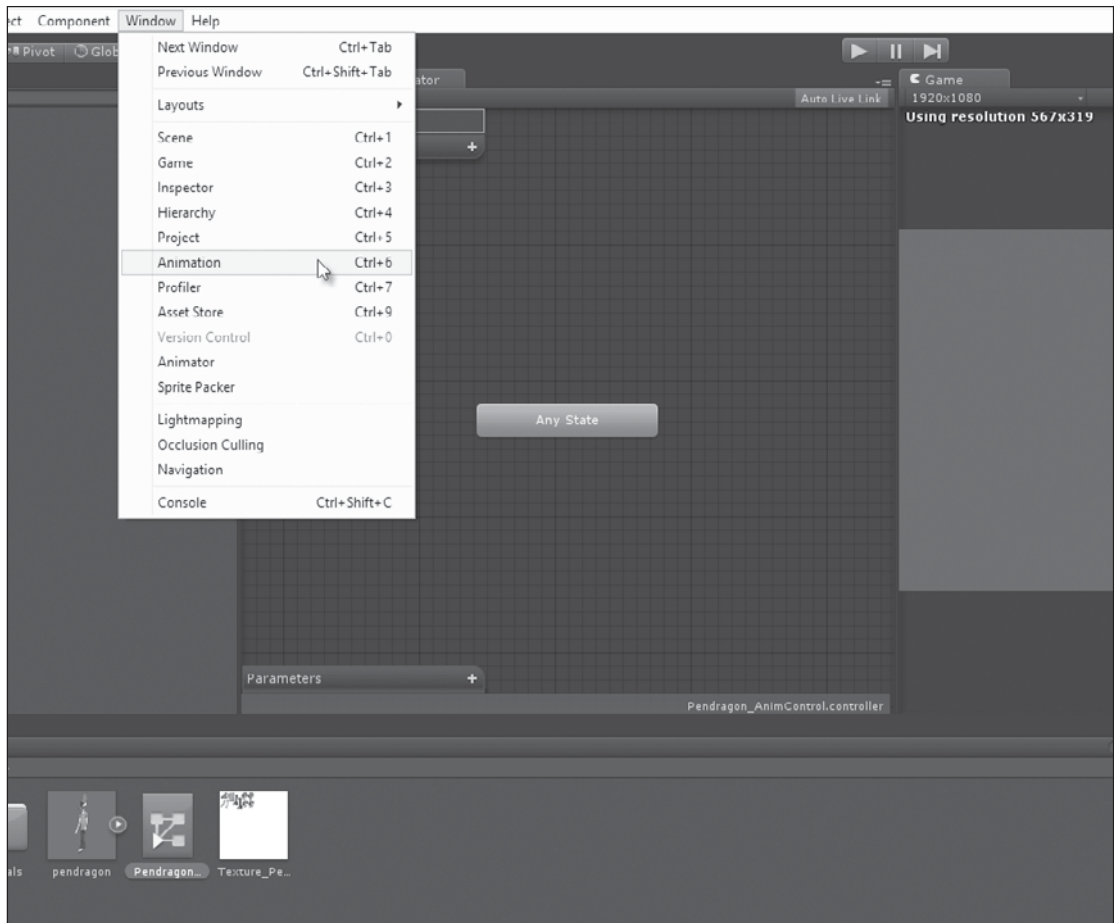


Figure 5.18

The animation graph with a default state.

Source: Unity Technologies.

6. To get the animation working for the Pendragon mesh, expand the mesh character pieces in the Project panel as before, by clicking the right-pointing arrow on the mesh thumbnail.
7. Click and drag the walk animation state from the mesh asset and drop it into the animation graph. The newly added animation appears as a state in the graph and is highlighted orange to indicate that it's the default state—that is, the state that will play automatically as gameplay begins. (See Figure 5.19.) If the animation doesn't appear in orange, right-click the state and choose Set As Default.

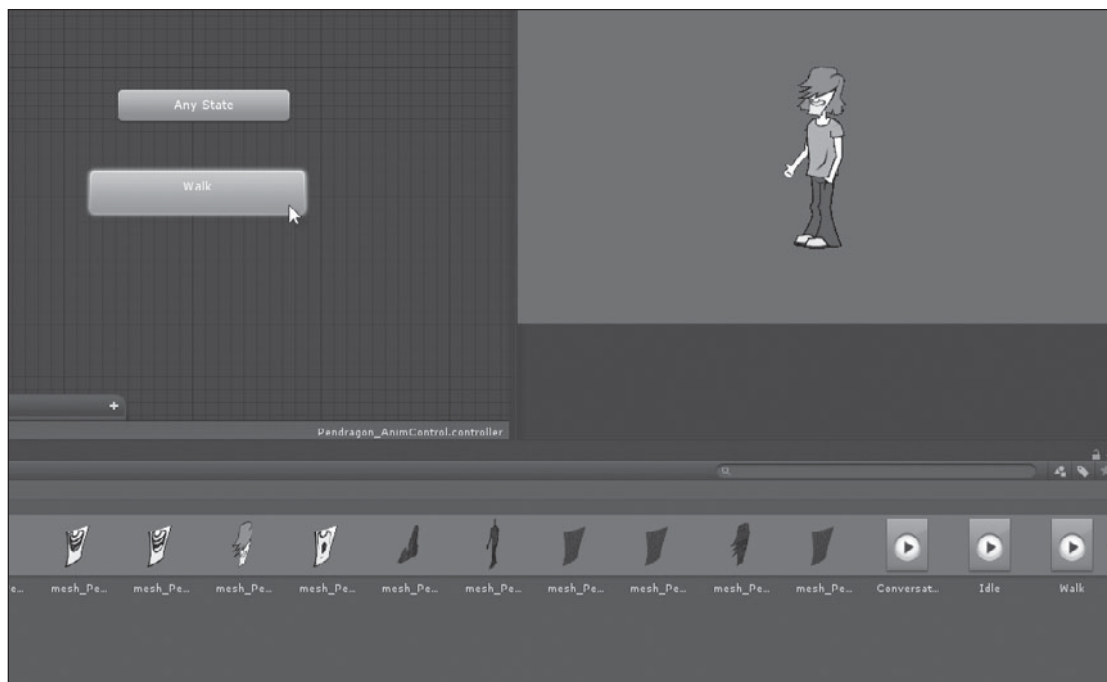


Figure 5.19

Setting the walk animation as the default state for the character.

Source: Unity Technologies.

8. The animation controller is now configured to play the walk animation as the game begins. To assign this controller to the Pendragon mesh in the scene, select the mesh and then drag and drop the animation controller asset from the Project panel into the Controller field for the Animator component in the Object Inspector. (See Figure 5.20.)

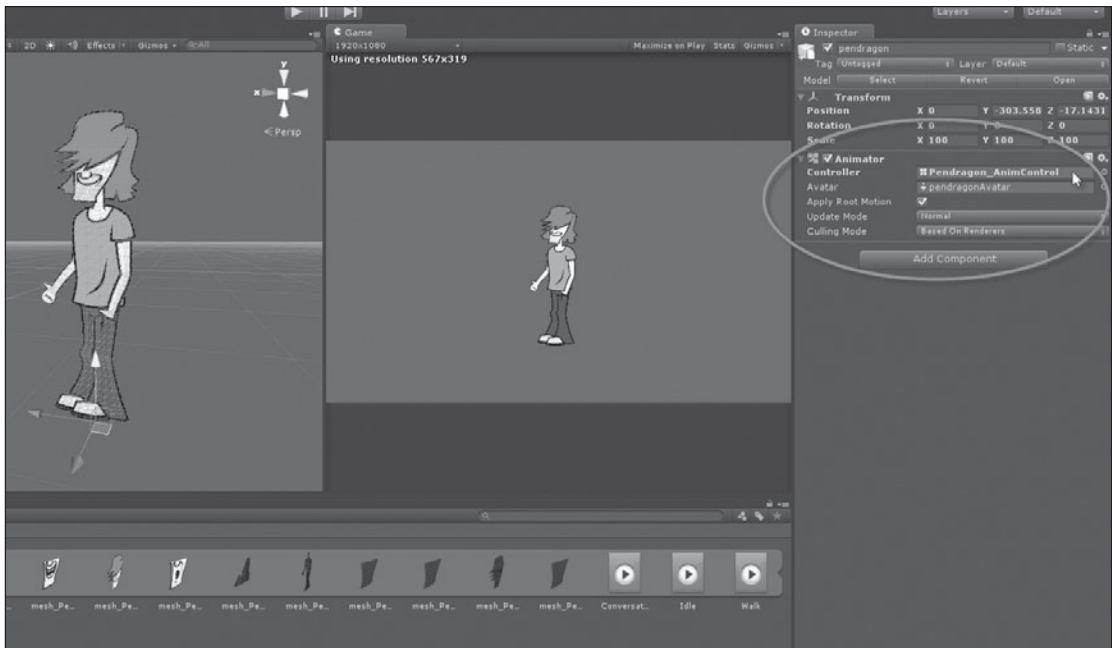


Figure 5.20

Assigning the animation controller to the character.

Source: Unity Technologies.

9. Run the game and watch the walk animation at work on the character in the viewport! Excellent work. You've just completed your first animated, real-time game character using a Blender-to-Unity workflow! (See Figure 5.21.)

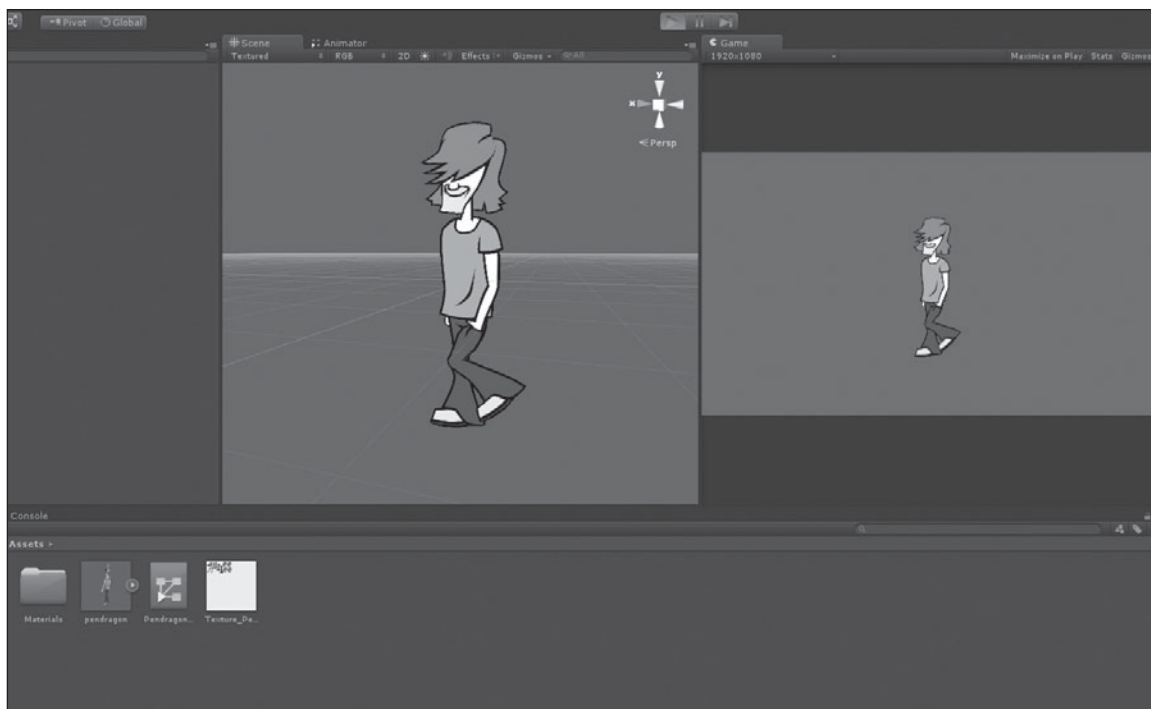


Figure 5.21

The character, up and running in Unity.

Source: Unity Technologies.

Note

The completed Unity project with an animated character can be found in the eBook companion files, in the Chapter05 folder.

CONCLUSION

You've come to the end of this course! In reaching this point, you've created an animated, game-compliant 2D character using a professional-grade workflow in Blender, a completely free application. The Pendragon character has been modeled, textured and UV-mapped, rigged, and animated to run a loopable walk-cycle that can be reused for any 2D character fitting the Pendragon armature. This character not only has the ability to move and rotate its limbs, like its arms and legs, but the limbs can deform and bend in a believable way.

Of course, in a short eBook such as this, it's not possible to cover every aspect of 2D animation. There's simply too much information for even the largest of books. But this eBook does get you going in the right direction on a strong foundation that supports a reliable workflow. Even more important, the workflow covered here is abstract and transferable, enabling you to rig and animate your own characters—even in other 3D tools that offer equivalent features to Blender. So with that said, I wish you every success in your 2D character creation adventures!